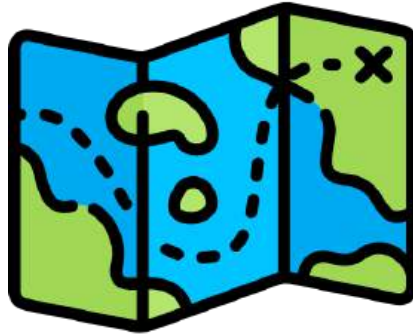


73.38 - Bases de Datos Espaciales y de Movilidad



Alumnos:

Iván Chayer, 61360

Thomas Mizrahi, 60154

Profesores:

Alejandro Ariel Vaisman



Índice

1. Introducción.....	3
2. GTFS Schedule.....	5
2.1 Descripción del estándar y de los archivos a utilizar.....	5
2.2 Carga de los datos estáticos con gtfs-via-postgres.....	7
2.3 Visualización de rutas.....	10
2.3.1 Visualización de todas las rutas.....	10
2.3.2 Visualización de las rutas por agencia.....	11
2.3.3 Visualización por tipo de ruta.....	13
2.4 División de rutas en segmentos con gtfs_functions.....	22
2.4.1 Rutas agregadas por segmentos.....	24
2.5 Análisis de la velocidad.....	29
2.5.1 Velocidad promedio por segmento.....	29
2.5.2 Comparación de la velocidad: días hábiles vs. fin de semana.....	33
2.6 Identificación de duplicación de rutas.....	37
2.6.1 Elección de las zonas a analizar.....	37
2.6.2 Definición del método de detección de hotspots.....	38
2.6.3 Cálculo de viajes por celda y detección de hotspots.....	41
2.6.4 Visualización de hotspots.....	44
2.6.5 Detección de duplicadas por buffer de las paradas.....	51
2.6.6 Exploración de resultados de rutas duplicadas.....	54
2.6.7 Conclusiones sobre la duplicación de rutas.....	57
2.7 Análisis de cobertura del transporte público en colegios de la ciudad.....	59
2.7.1 Carga de los datos para realizar el análisis.....	60
2.7.2 Cobertura del transporte público por tipo de ruta.....	63
2.7.3 Top 5 colegios con menor accesibilidad.....	68
2.7.4 Top 5 colegios con mayor accesibilidad.....	71
3. GTFS Real Time.....	74
3.1 Procesamiento del feed de las posiciones de los vehículos.....	76
3.2 Procesar datos de vehículos a partir del feed.....	81
3.3 Cargar las posiciones de los vehículos depuradas en PostgreSQL.....	84
3.4 Visualización de las trayectorias reales.....	89
3.4.1 Visualización de las trayectorias sin map matching.....	90
3.4.2 Map matching usando Valhalla.....	95
3.4.2.1 Implementación del cliente Valhalla.....	97
3.4.2.2 Visualización del bus trip de la sección 3.4.1.....	101
3.4.2.3 Visualización de todos los bus trips.....	103
3.5 La problemática del armado de los bus trips “ajustados”.....	108
3.5.1 Modificando el cliente Valhalla para parseo de tracepoints.....	111
3.5.2 Algoritmo de interpolación temporal.....	113
3.5.3 Carga de los bus trips ajustados.....	115
3.5.4 Visualización de los bus trips ajustados.....	119
3.6 Construcción de segmentos.....	121

3.6.1 Preprocesamiento de los trips ajustados.....	121
3.6.2 Creación de la tabla trip_stops.....	122
3.6.3 Creación de la tabla trip_segments.....	129
4. Comparación de datos planificados y en tiempo real.....	131
4.1 Comparación de velocidades.....	131
4.1.1 Análisis cuantitativo.....	133
4.1.2 Análisis espacial.....	134
4.2 Comparación de la demora.....	138
4.2.1 Demora a nivel parada.....	138
4.2.2 Demora a nivel segmento.....	140
4.2.3 Demora de vehículos puntuales a nivel parada y segmentos.....	143
5. Conclusión.....	146
X. Referencias.....	147

1. Introducción

Este proyecto corresponde al trabajo final de la materia Bases de Datos Espaciales y de Movilidad, cuyo objetivo principal es analizar el sistema de transporte público utilizando datos en formato GTFS (General Transit Feed Specification). En particular, se trabajó con datos de la ciudad de Boston, ubicada en el estado de Massachusetts. A partir del procesamiento de datos públicos de transporte, se busca evaluar distintos aspectos del sistema.

El informe se estructura en varias etapas. En la Sección 2, se aborda el procesamiento de datos estáticos (GTFS Schedule), incluyendo la obtención, organización y carga de los datos. Una vez cargados, se realizan visualizaciones de las rutas planificadas, se agrupan por segmentos y se identifican zonas con alta densidad de recorrido. Además, se analiza la velocidad planificada en distintos momentos de la semana y se hace un análisis de la cobertura del sistema de transporte público frente a los colegios de la ciudad.

La Sección 3 se centra en los datos en tiempo real (GTFS Real-Time), detallando el proceso de recolección, depuración y análisis. Para ello, se describen las técnicas utilizadas para obtener la información, así como la implementación de técnicas de *map matching* e interpolación temporal de trayectorias, con el fin de reconstruir las trayectorias reales de los vehículos a partir de su posición reportada del *feed GTFS*.

En la Sección 4, se presenta un análisis comparativo entre lo planificado (GTFS Schedule) y lo observado (GTFS Real-Time), incluyendo la comparación de velocidades y el cálculo de delays a nivel de segmento y paradas.

Algunos desarrollos están basados en el Capítulo 9 y 11 del libro de la materia, que sirvió como guía metodológica¹.

¹ Sakr, M., Vaisman, A., & Zimányi, E. (2025). Mobility Data Science: From Data to Insights. Springer.

Para complementar el informe, se incluyen recursos adicionales que permiten reproducir el trabajo para incluso facilitar futuras extensiones del análisis. Algunos recursos, como los datos estáticos utilizados para la Sección 2, se encuentran disponibles en este [link de Google Drive](#). Los *scripts* utilizados para el proyecto se encuentran en un [repositorio de GitHub](#). Todos ellos están implementados en Python, por lo que se recomienda crear un entorno virtual e instalar las dependencias definidas en el archivo `requirements.txt` siguiendo el README del repositorio.

Los scripts de Python ubicados en la carpeta `scheduled` deben ejecutarse desde el directorio raíz del proyecto, utilizando la siguiente sintaxis:

```
(bdeym) > python scheduled/<folder>/<script_name>.py
```

De manera similar, los scripts ubicados en la carpeta `realtime` también deben ejecutarse desde la raíz del proyecto, pero en este caso utilizando la opción `-m` de Python para hacer referencia al módulo.

```
(bdeym) > python -m realtime.<folder>.<script_name>
```

2. GTFS Schedule

2.1 Descripción del estándar y de los archivos a utilizar

GTFS Schedule (General Transit Feed Specification) es un formato utilizado para representar información sobre los servicios de transporte público planificados. Básicamente, permite compartir información sobre rutas, horarios, paradas y otros datos relacionados en un formato estandarizado. Está compuesto por un conjunto de archivos CSV que describen distintos aspectos del sistema de transporte de, por ejemplo, una ciudad. Estos archivos suelen distribuirse comprimidos en un único archivo zip.

En su forma más básica, un conjunto de datos GTFS Schedule está compuesto por 7 archivos: `agency.txt`, `routes.txt`, `trips.txt`, `stops.txt`, `stop_times.txt`, `calendar.txt` y `calendar_dates.txt`. Junto con este conjunto básico de archivos, también pueden incluirse archivos adicionales (opcionales) para proporcionar información sobre otros elementos del servicio, como tarifas, figuras, etc. Actualmente existen más de 10 archivos opcionales que amplían los elementos básicos mencionados anteriormente. La fuente de referencia oficial para todos los archivos del conjunto de datos GTFS Schedule se encuentra en este [link](#), la cual brinda información detallada sobre los requisitos de cada uno de los elementos presentes en los archivos que lo componen.

Para el caso particular de Boston, los datos fueron descargados desde el portal oficial de la Massachusetts Bay Transportation Authority (MBTA), a través de este [enlace](#). El archivo zip original contiene un total de 32 archivos, aunque no todos forman parte del estándar obligatorio del formato GTFS. Siguiendo la [documentación oficial de MBTA para GTFS Schedule](#), se observa que varios de esos archivos son específicos de MBTA o no están contemplados en GTFS Basic, mientras que otros son opcionales. Para este proyecto, se decidió trabajar únicamente con los 7 archivos básicos definidos por GTFS ,

complementados con dos archivos opcionales: `shapes.txt` y `transfers.txt`. Esta selección de archivos se encuentra en un nuevo zip denominado `data.zip`, disponible en la carpeta de Google Drive que recomendamos descargar dentro de la carpeta `scheduled` del repositorio clonado. Una vez descargado, sugerimos descomprimir el contenido dentro de una subcarpeta llamada `gtfs` para facilitar los pasos posteriores. A continuación, presentamos una breve descripción de cada uno de los archivos incluidos.

- **agency.txt**: Contiene información sobre las agencias de transporte que operan los servicios incluidos en el *dataset*.
- **calendar.txt**: Contiene información sobre las fechas de servicio definidas a través de un calendario semanal, indicando fechas de inicio y fin.
- **calendar_dates.txt**: Proporciona excepciones al servicio especificado en `calendar.txt`, como feriados o eventos especiales.
- **routes.txt**: Describe las líneas de transporte ofrecida por las agencias. Una línea agrupa varios recorridos que se presentan a los usuarios como un mismo servicio.
- **shapes.txt**: Contiene información para representar los trayectos que siguen los vehículos.
- **stop_times.txt**: Contiene información sobre los horarios de llegada y salida del vehículo en cada parada para cada recorrido.
- **stops.txt**: Contiene información sobre las paradas y estaciones donde los vehículos recogen o dejan pasajeros.
- **transfers.txt**: Contiene información sobre cómo deben hacerse los transbordos entre líneas en los puntos de conexión.
- **trips.txt**: Describe los recorridos de cada línea. Un recorrido es una secuencia de dos o más paradas que se realizan durante un período de tiempo determinado.

Ahora bien, para poder llevar a cabo el análisis del sistema, es necesario combinar la información contenida en cada uno de los archivos. Para facilitar esta tarea, existen herramientas que automatizan la carga y vinculación de los archivos dentro de una base de datos geoespacial (PostgreSQL con PostGIS), como veremos a continuación.

2.2 Carga de los datos estáticos con *gtfs-via-postgres*

En la sección anterior descomprimos el archivo `data.zip` descargado de MBTA, guardamos los archivos dentro de la carpeta `scheduled/gtfs` y analizamos los diferentes archivos que provee el *dataset*. En este paso, nuestro objetivo es cargar esa información en una base de datos PostgreSQL con soporte geoespacial a través de PostGIS.

Haremos uso de [gtfs-via-postgres](#), una herramienta de código abierto desarrollada para facilitar la importación de datos GTFS Schedule a bases de datos PostgreSQL con PostGIS. Está escrita en *Node.js* y puede instalarse fácilmente usando el gestor de paquetes [npm](#). También existen [binarios precompilados](#) disponibles. Es importante aclarar que la herramienta requiere PostgreSQL 14 o superior y tener la extensión PostGIS instalada (por ejemplo, usando StackBuilder). Además, toda operación que se ejecute con esta herramienta, debe ser realizada a través de un superusuario de PostgreSQL (como el usuario `postgres`). Para nuestro proyecto, vamos a instalar la herramienta usando `npm` de la siguiente forma:

```
npm install -g gtfs-via-postgres
```

Una vez instalada la herramienta, abrimos una terminal en el directorio `scheduled` y seguimos los siguientes pasos dependiendo de nuestro sistema operativo.

WINDOWS

```
> Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass  
> .\load_gtfs_data.ps1
```

El *script* de Powershell `load_gtfs_data.ps1` ejecuta los comandos necesarios para llamar a la herramienta correctamente.

LINUX

```
> export PGUSER="postgres"  
> export PGPASSWORD="postgres"  
> export PGHOST="localhost"  
> export PGPORT="5432"  
> psql -d postgres -c "DROP DATABASE IF EXISTS mbtagtfs;"  
> psql -d postgres -c "CREATE DATABASE mbtagtfs;"  
> export PGDATABASE="mbtagtfs"  
> npm exec -- gtfs-to-sql --require-dependencies -- gtfs/*.txt | sponge  
| psql -b
```

La importación de los datos tomará un tiempo. Además de una tabla para cada archivo txt, la herramienta crea las siguientes vistas:

- **service_days:** aplica `calendar_dates` a `calendar` para mostrar todos los días de operación para cada servicio definido en `calendar`.
- **arrivals_departures:** aplica `stop_times` y `frequencies` a `trips` y `service_days` para mostrar todas las llegadas/salidas en cada parada con fechas y horarios. También resuelve el ID y nombre de la estación principal de cada parada.
- **connections:** aplica `stop_times` y `frequencies` a `trips` y `service_days`, como `arrivals_departures`, pero da pares de salida (en la parada A) y llegada (en la parada B).
- **shapes_aggregated:** agrega los puntos individuales de `shapes` en un `LineString` de PostGIS.

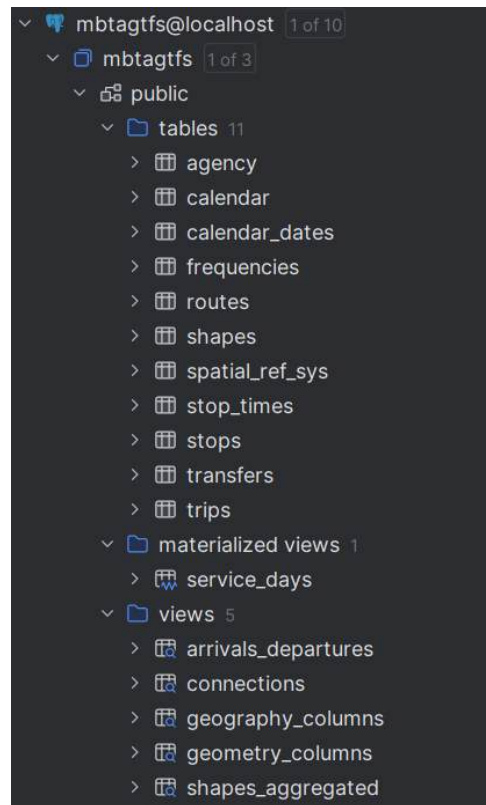


Figura 1: Tablas y vistas creadas por *gtfs-via-postgres* en PostgreSQL.

2.3 Visualización de rutas

En esta sección se presentan distintas visualizaciones de las rutas planificadas del sistema de transporte de la ciudad de Boston. Utilizaremos los *scripts* que se encuentran en `schedule/visualizations` del repositorio. En la subsección 2.3.1 se muestran todas las geometrías correspondientes a las rutas incluidas en la vista `shapes_aggregated` generada en la sección anterior por la herramienta *gtfs-via-postgres*. Posteriormente, se agrupan y visualizan estas rutas según agencia (subsección 2.3.2) y el tipo de ruta (subsección 2.3.3).

2.3.1 Visualización de todas las rutas

Si ejecutamos el *script* `map_individual_lines.py` se obtienen todas las tuplas de la vista `shapes_aggregated` y, usando la librería de Folium, se genera un `html` que podemos visualizar en nuestro navegador. A no ser que se especifique lo contrario, las figuras que se verán a lo largo de este documento se generarán utilizando dicha librería.

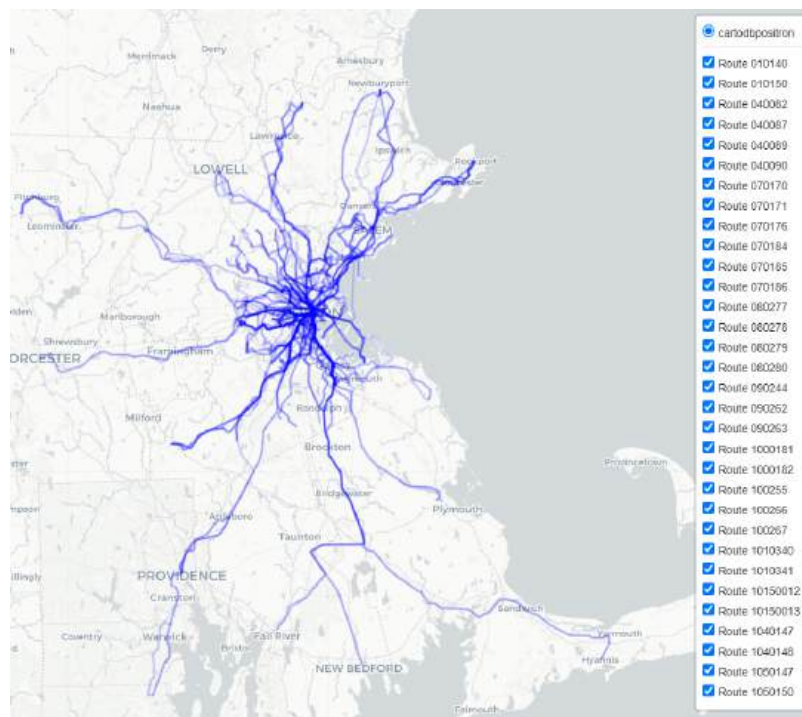


Figura 2: Visualización de todas las rutas planificadas.

A partir de la Figura 2, puede observarse que el servicio de transporte público no se restringe únicamente al núcleo urbano de Boston, sino que se extiende hacia una amplia región metropolitana que abarca ciudades y localidades periféricas como Worcester, Providence, Fitchburg, New Bedford y Plymouth, entre otras. Esta extensa cobertura evidencia que el sistema está diseñado con un enfoque regional, integrando tanto zonas urbanas como suburbanas. Además se puede observar que el *dataset* contiene rutas marítimas.

2.3.2 Visualización de las rutas por agencia

A continuación queremos ver cómo se distribuyen las rutas las diferentes agencias de nuestro *dataset*. En nuestro caso tenemos solamente 2, MBTA y Cape Cod Regional Transit Authority según la tabla *agency*.

```
SELECT r.route_id,
       r.route_type,
       sa.shape_id,
       sa.shape,
       a.agency_id,
       a.agency_name
FROM shapes_aggregated sa
     JOIN trips t ON sa.shape_id = t.shape_id
     JOIN routes r ON t.route_id = r.route_id
     JOIN agency a ON r.agency_id = a.agency_id
GROUP BY sa.shape_id, sa.shape, a.agency_id, a.agency_name,
         r.route_type, r.route_id
```

Esa consulta nos permite obtener el *shape_id* asociado a cada agencia y devolviendo una sola fila por combinación única de los campos agrupados. Dicha consulta se ejecuta dentro del *script* `map_by_agencies.py` que genera un mapa con las rutas por cada agencia.

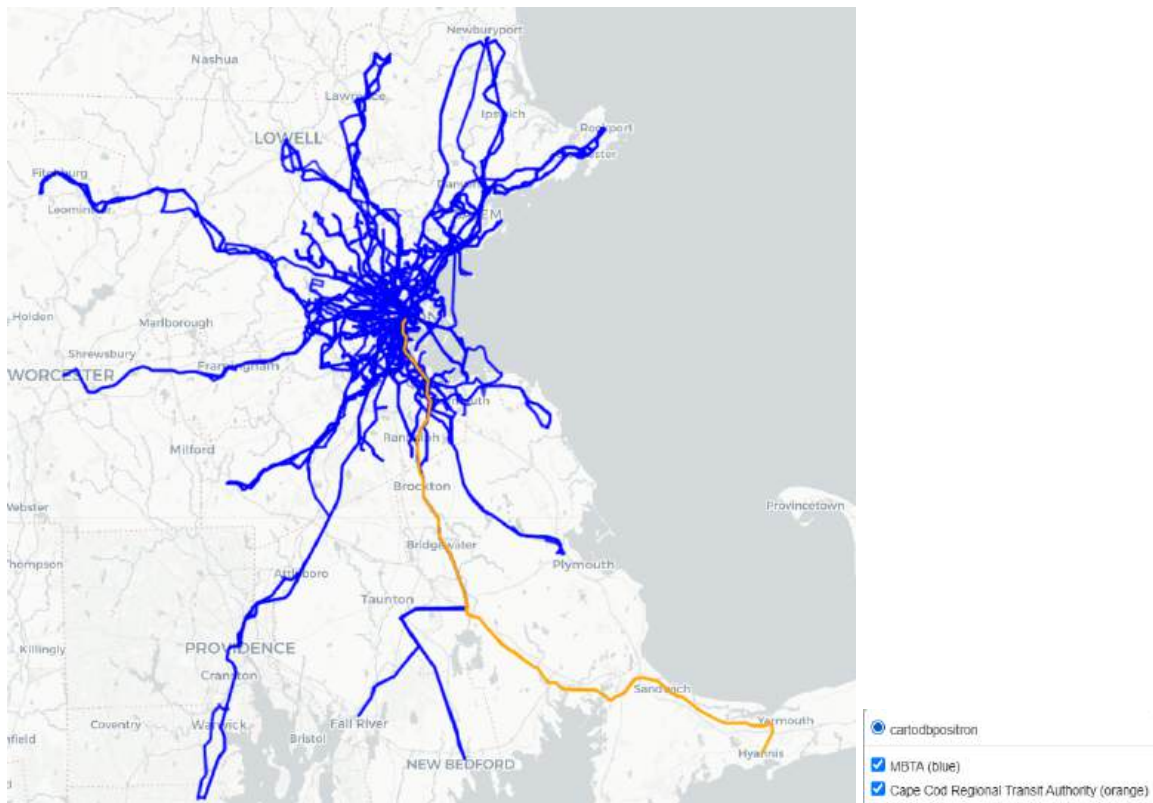


Figura 3: Visualización de las rutas por agencia.

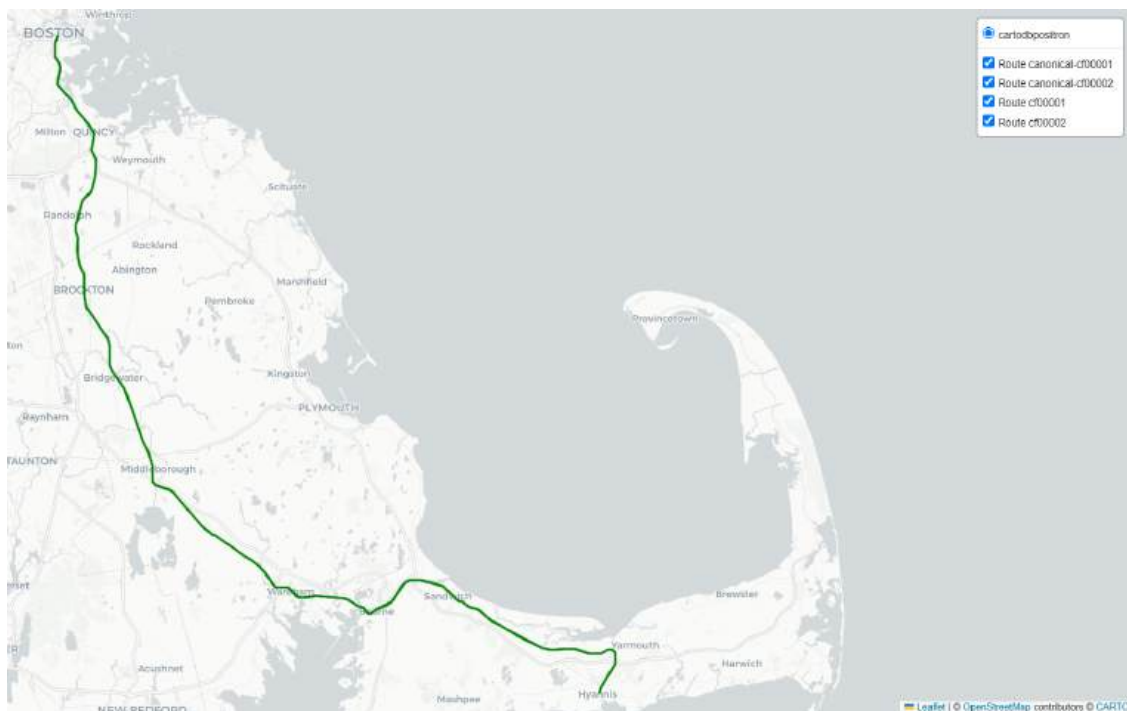


Figura 4: Visualización de las rutas de la agencia Cape Cod Regional Transit Authority.

Como podemos observar en la Figura 3, la mayoría se corresponden a la agencia MBTA mientras que parecería ser que una sola corresponde a Cape Cod Regional Transit Authority. A partir de la Figura 4, se observa que las rutas dentro de la Cape Cod Regional Transit Authority comparten exactamente el mismo trazado geográfico, es decir, siguen un mismo recorrido físico.

2.3.3 Visualización por tipo de ruta

Veamos ahora la siguiente consulta para analizar los diferentes tipos de ruta.

```
-- https://gtfs.org/documentation/schedule/reference/#routetxt

SELECT
CASE r.route_type
  WHEN '0' THEN 'Tram / Light rail'
  WHEN '1' THEN 'Subway / Metro'
  WHEN '2' THEN 'Rail (long-distance)'
  WHEN '3' THEN 'Bus'
  WHEN '4' THEN 'Ferry'
  WHEN '5' THEN 'Cable tram'
  WHEN '6' THEN 'Aerial lift'
  WHEN '7' THEN 'Funicular'
  WHEN '11' THEN 'Trolleybus'
  WHEN '12' THEN 'Monorail'
  ELSE 'Other / Unknown'
END AS route_type_name,
COUNT(DISTINCT t.shape_id) AS num_shapes
FROM trips t
JOIN routes r ON t.route_id = r.route_id
GROUP BY route_type_name
ORDER BY num_shapes DESC
```

	route_type_name ▾	num_shapes ▾
1	Bus	846
2	Rail (long-distance)	77
3	Ferry	38
4	Subway / Metro	28
5	Tram / Light rail	28

Figura 5: Resultado de la consulta.

El resultado de la Figura 5 muestra la cantidad de recorridos por tipo de ruta según el estándar básico de GTFS Schedule. Se observa que el bus es el predominante y que el subte/metro y el tren ligero son los menos predominantes. Utilizando como base la consulta utilizada en la sección anterior, podemos modificarla agregando el nuevo campo `route_type_name` para graficar las rutas agrupadas por tipos de rutas a través del *script* `map_by_route_type.py`.

```
SELECT r.route_id,
       r.route_type,
       CASE r.route_type
         WHEN '0' THEN 'Tram / Light rail'
         WHEN '1' THEN 'Subway / Metro'
         WHEN '2' THEN 'Rail (long-distance)'
         WHEN '3' THEN 'Bus'
         WHEN '4' THEN 'Ferry'
         WHEN '5' THEN 'Cable tram'
         WHEN '6' THEN 'Aerial lift'
         WHEN '7' THEN 'Funicular'
         WHEN '11' THEN 'Trolleybus'
         WHEN '12' THEN 'Monorail'
         ELSE 'Other / Unknown'
       END AS route_type_name,
       sa.shape_id,
       sa.shape
FROM shapes_aggregated sa
     JOIN trips t ON sa.shape_id = t.shape_id
     JOIN routes r ON t.route_id = r.route_id
```

```
GROUP BY sa.shape_id, sa.shape, r.route_type, r.route_id
```

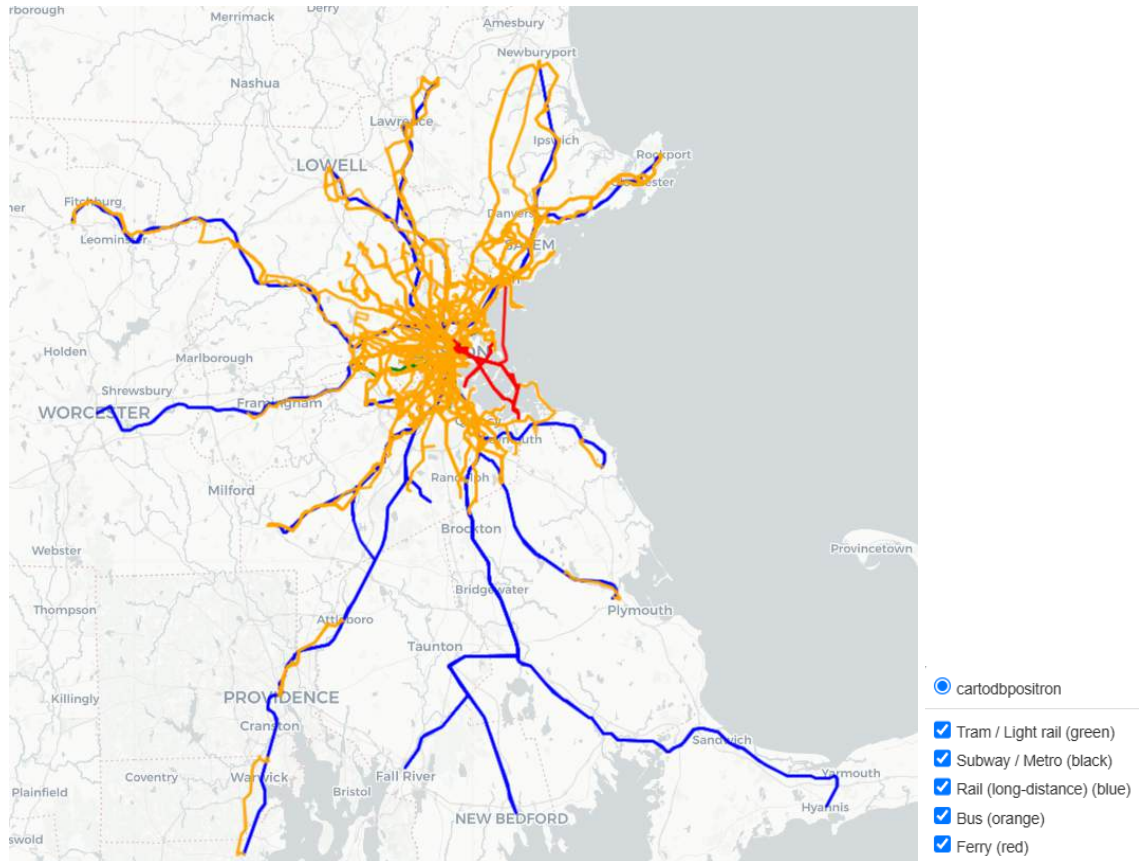


Figura 6: Visualización de las rutas por tipo de ruta.

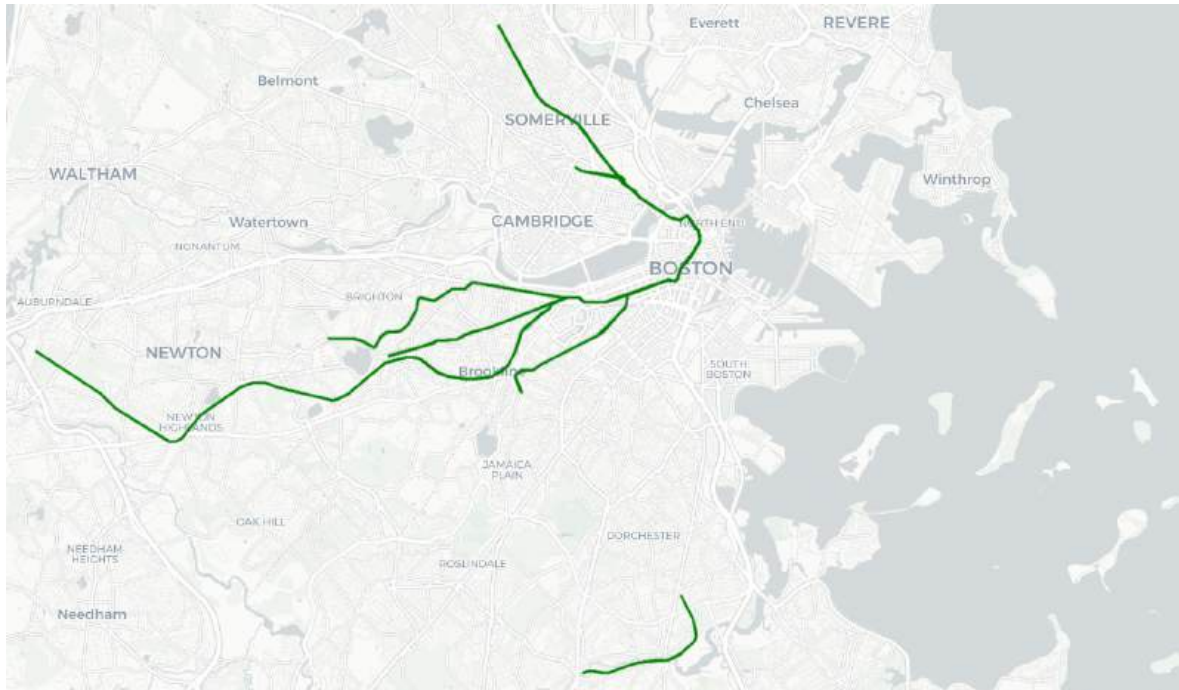


Figura 7: Visualización de las rutas de tipo Tram / Light Rail.

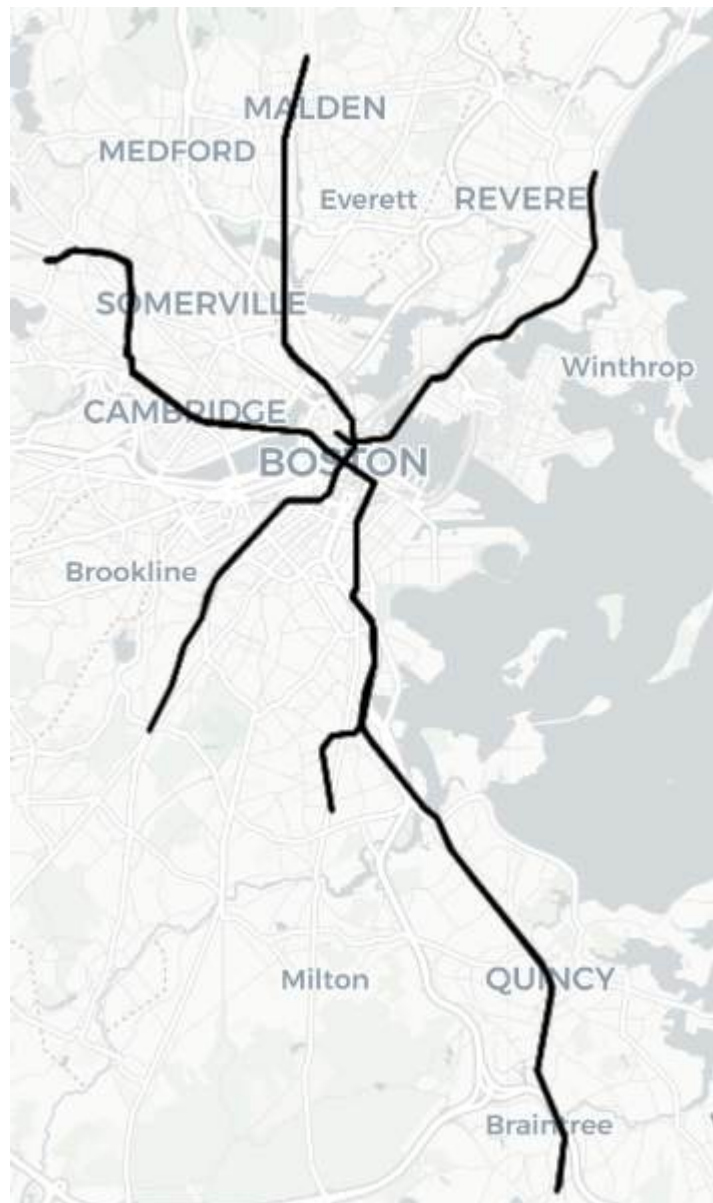


Figura 8: Visualización de las rutas de tipo Subway / Metro.

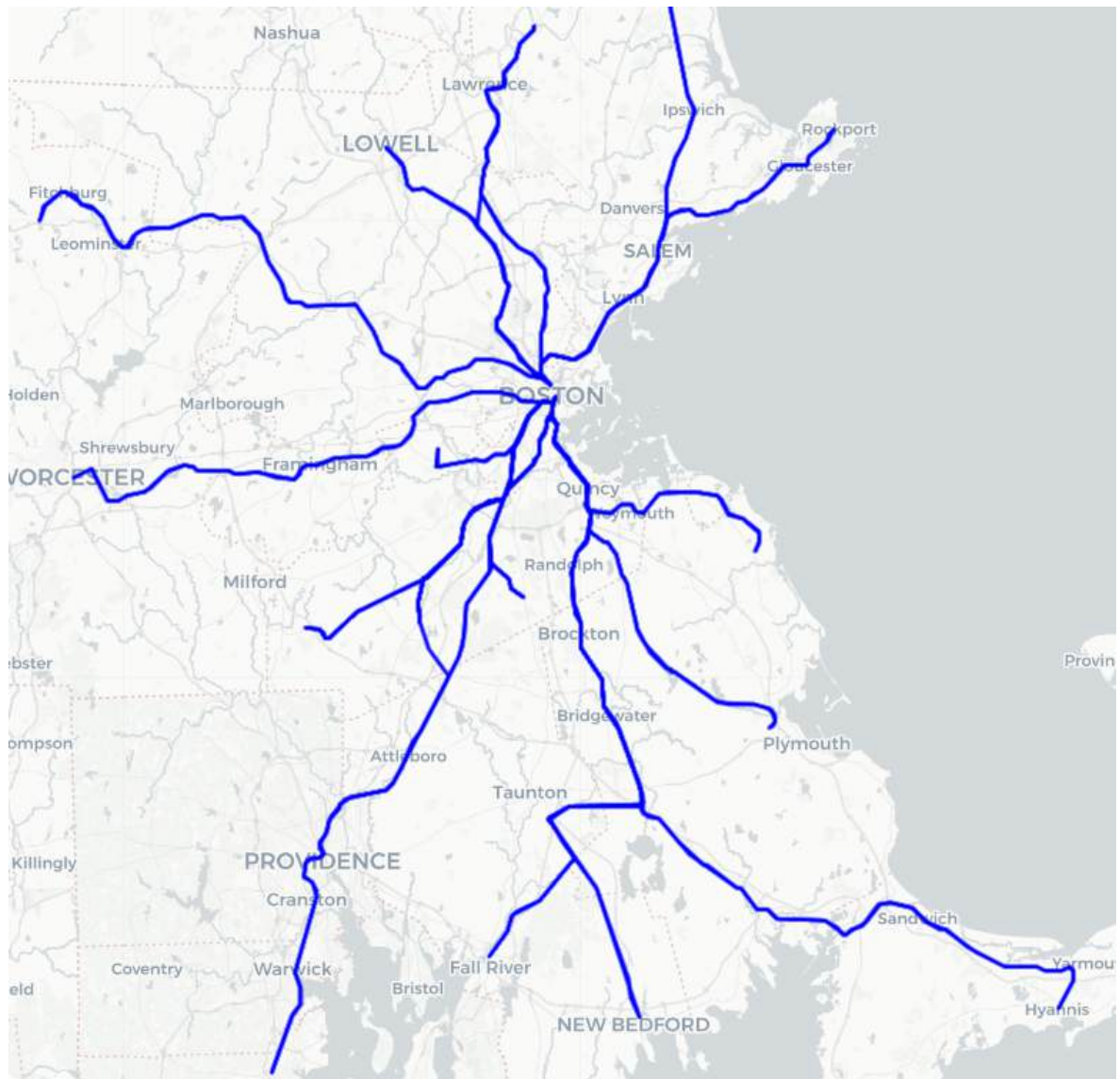


Figura 9: Visualización de las rutas de tipo Rail Long Distance.

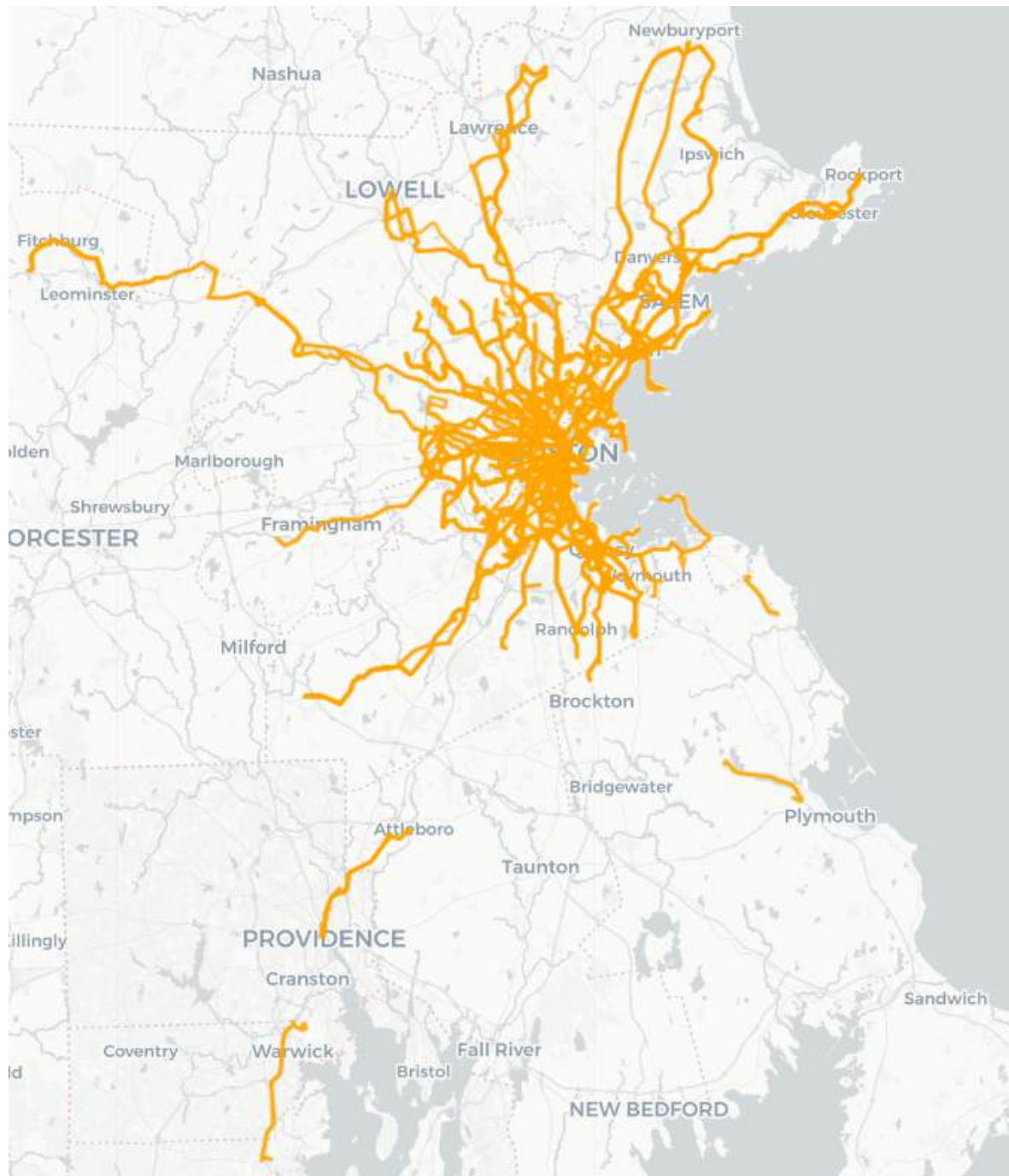


Figura 10: Visualización de las rutas de tipo Bus.

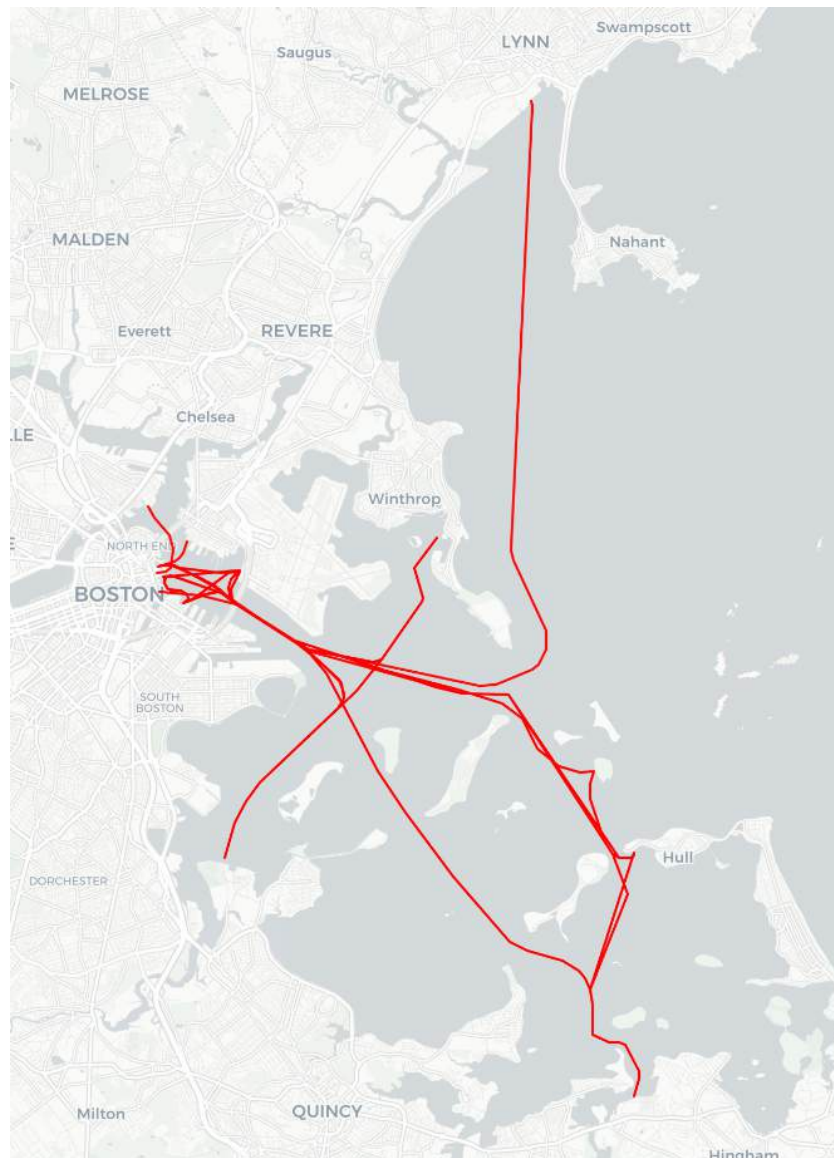


Figura 11: Visualización de las rutas de tipo Ferry.

En las Figuras 6 a 11 podemos ver las rutas por tipo de rutas. La Figura 6 y 10 refleja lo que obtuvimos en la Figura 5, donde tenemos más tipos de rutas de buses. En la Figura 7, al sur del centro de Boston, podemos visualizar una ruta de un tranvía muy famoso por su antigüedad llamado [Mattapan Trolley](#) que va de Mattapan a Ashton (y viceversa) y permite conectar con varias [líneas de transporte](#). En la Figura 8 se muestran las rutas de tipo Subway / Metro. Se observa una estructura radial que conecta el centro de Boston con las zonas más pobladas del área metropolitana, como Cambridge, Quincy y Malden. En la

Figura 9 se observan las rutas clasificadas como Rail Long Distance. Estas líneas se extienden mucho más allá del centro urbano, conectando Boston con ciudades periféricas como Worcester, Providence, Lowell y Plymouth. Esto refuerza su rol como infraestructura interurbana, pensada para cubrir trayectos de media y larga distancia. En la Figura 10 se muestra la red de rutas de Bus, la cual aparece mucho más ramificada y densa en comparación con el resto. Se despliega en forma de abanico desde el centro de Boston, alcanzando tanto zonas urbanas como suburbanas. Su densidad sugiere que complementa a los sistemas ferroviarios, cubriendo trayectos más cortos y zonas no alcanzadas por el metro o el tren. En la Figura 11 se visualiza las rutas de tipo Ferry. Se destaca la conexión entre el centro de Boston y diversas localidades costeras como Hull, Hingham y Lynn, lo que evidencia el rol del transporte fluvial en la articulación de zonas donde otros modos de transporte terrestre serían menos eficientes.

2.4 División de rutas en segmentos con *gtfs_functions*

En esta sección se busca descomponer las rutas del sistema de transporte en segmentos, es decir, tramos entre dos paradas consecutivas. Para eso, vamos a utilizar una librería llamada [gtfs_functions](#) que provee funciones para manipular datos GTFS Schedule. La misma se puede instalar usando [pip](#), el package manager de Python, para versiones de Python mayores o iguales a la 3.8. La misma se encuentra en el archivo `requirements.txt` mencionado en la introducción. Para esta sección, estaremos utilizando los scripts que se encuentran dentro de la carpeta `scheduled/segments`.

Para crear los segmentos, colocar el archivo `data.zip` dentro de la carpeta `scheduled/segments` y ejecutar el *script* `create_segments.py`. Luego de ejecutarlo, el resultado debería ser el de la Figura 12.

```
ivanc@ichayer-G7TASH2 MINGW64 ~/Desktop/BDM/bdeym/scheduled/segments (main)
$ python create_segments.py
INFO:root:Getting segments...
INFO:root:Reading "stop_times.txt".
INFO:root:get trips in stop_times
INFO:root:accessing trips
INFO:root:Reading "routes.txt".
INFO:root:Reading "trips.txt".
INFO:root:Reading "calendar.txt".
INFO:root:Reading "calendar_dates.txt".
INFO:root:The busiest date/s of this feed or your selected date range is/are: ['2025-07-04'] with 24948 trips.
INFO:root:In the case that more than one busiest date was found, the first one will be considered.
INFO:root:In this case is 2025-07-04.
INFO:root:Reading "stop_times.txt".
INFO:root:trips is defined in stop_times
INFO:root:Reading "stops.txt".
INFO:root:computing patterns
INFO:root:Reading "shapes.txt".
INFO:root:Projecting stops onto shape...
INFO:root:Interpolating stops onto shape...
INFO:root:Sorting shape points and stops...
INFO:root:segments_df: 18470, geometry: 18470
Table 'segments' successfully saved to database 'mbtagtfs'.
```

Figura 12: Resultado de correr *create_segments.py*.

El *script* utiliza la librería mencionada para cargar los datos de Boston y extraer los segmentos de las rutas, que luego se almacenan en la base de datos. El objeto Feed se inicia con la ruta al archivo zip y un rango de fechas. Este objeto devuelve un

GeoDataFrame que contiene los segmentos, definidos como los tramos entre dos paradas consecutivas en un viaje. El resultado es una tabla llamada `segments` dentro de la base de datos `mbtagtfs`, que contiene todos los segmentos de transporte planificados para el año 2025.

Para ver un resultado a modo de ejemplo, podemos graficar con el *script* `visualize_universities_segment.py` (Figura 13) la ruta del Bus 8 que va de Mt Vernon St @ South Point Dr a Kenmorede pasando por la universidad de Harvard (marcador azul) y otras universidades (marcador rojo) de la siguiente manera.

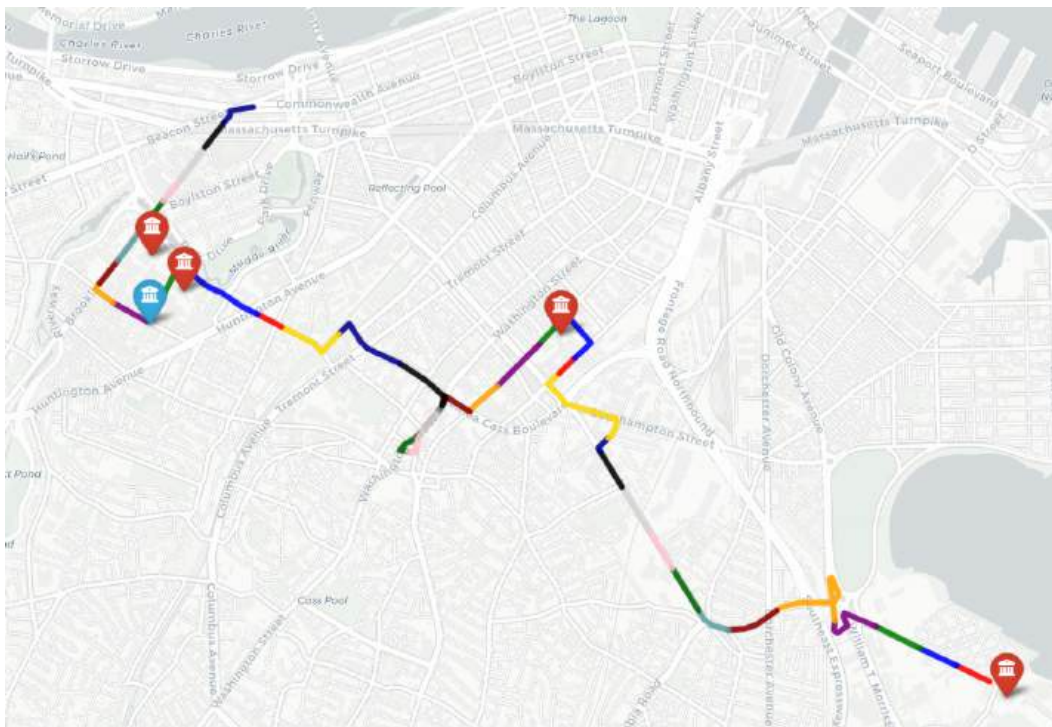


Figura 13: Segmentos de la ruta Mt Vernon St @ South Point Dr a Kenmore.

2.4.1 Rutas agregadas por segmentos

Para calcular la cantidad de viajes por segmento, utilizamos la tabla `segments`, y la vista `connections`. Al unir ambas, fue posible contar cuántos viajes recorren cada segmento (*number_of_trips*) y las rutas distintas que utilizan ese tramo (*routes*). Esta información fue almacenada en una vista materializada enfocados únicamente en la semana del 7 al 13 de julio.

```
CREATE MATERIALIZED VIEW segment_trip_stats AS
SELECT c.from_stop_id,
       c.from_stop_name,
       c.to_stop_id,
       c.to_stop_name,
       s.geometry,
       COUNT(trip_id) AS number_of_trips,
       STRING_AGG(DISTINCT c.route_long_name, ',') AS routes
FROM segments s
      JOIN connections c
        ON s.route_id = c.route_id
         AND s.direction_id = c.direction_id
         AND s.start_stop_id = c.from_stop_id
         AND s.end_stop_id = c.to_stop_id
WHERE c.date BETWEEN '2025-07-07' AND '2025-07-13'
GROUP BY c.from_stop_id, c.from_stop_name,
         c.to_stop_id, c.to_stop_name,
         s.geometry
```

A partir de esta vista, podemos obtener un resumen general de los segmentos: cuántos se generaron, cuántos viajes tienen y cuál es el mínimo y máximo de viajes registrados por tramo.

```
SELECT
  COUNT(*) AS total_segments, -- 8212
  ROUND(AVG(number_of_trips), 2) AS average_trips_per_segment, -- 483.75
  MIN(number_of_trips) AS min_trips_per_segment, -- 1
  MAX(number_of_trips) AS max_trips_per_segment -- 7500
FROM segment_trip_stats
```

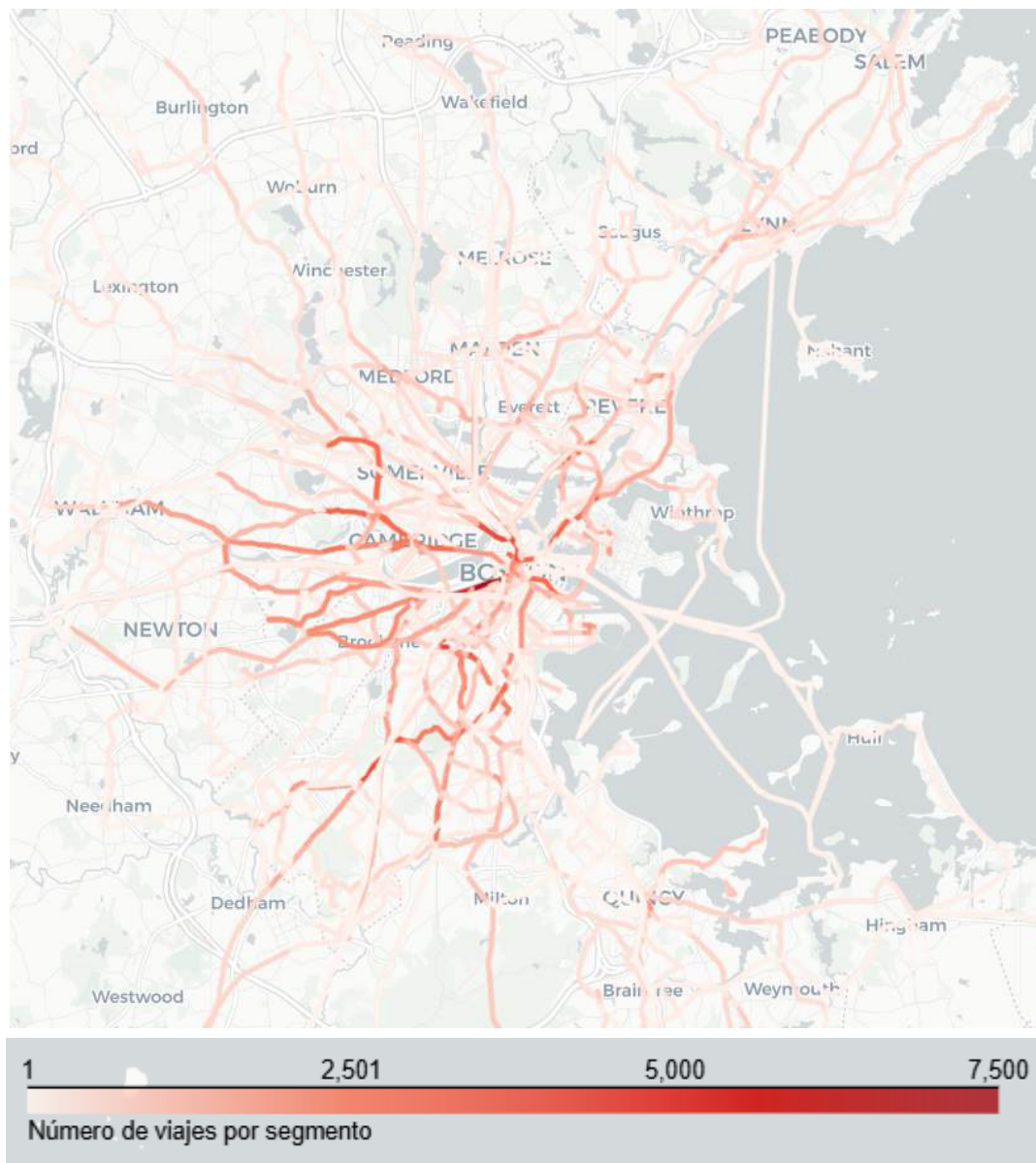


Figura 14: Visualización del número de viajes por segmento entre el 7 de julio y 13 de julio de 2025

Con el *script* `visualize_aggregate_trips_per_segment.py` obtenemos la Figura 14, en la que se observa que la mayor concentración de itinerarios planificados por segmento se encuentra en el área central de Boston, donde los trazos presentan un color rojo intenso que indica una alta frecuencia de servicios programados. Esta densidad refleja la priorización del transporte público en la zona central de la ciudad.

A medida que se avanza hacia las zonas periféricas, los segmentos muestran tonalidades más claras, lo que indica una menor cantidad de servicios planificados.

Con la siguiente consulta, obtenemos los segmentos con la mayor cantidad de viajes (Figura 15).

```
SELECT *  
FROM segment_trip_stats  
WHERE number_of_trips = (SELECT MAX(number_of_trips) FROM  
segment_trip_stats)
```

	from_stop_id	from_stop_name	to_stop_id	to_stop_name	number_of_trips
1	70157	Arlington	70155	Copley	7500
2	70159	Boylston	70157	Arlington	7500

Figura 15: Resultados de la consulta anterior

Podemos visualizar en Python el resultado (Figura 17) ejecutando el *script* `visualize_max_aggragated_trips_per_segment.py`. Análogamente, podemos obtener los segmentos con menos trips (Figura 16) ejecutando el *script* `visualize_min_aggragated_trips_per_segment.py`

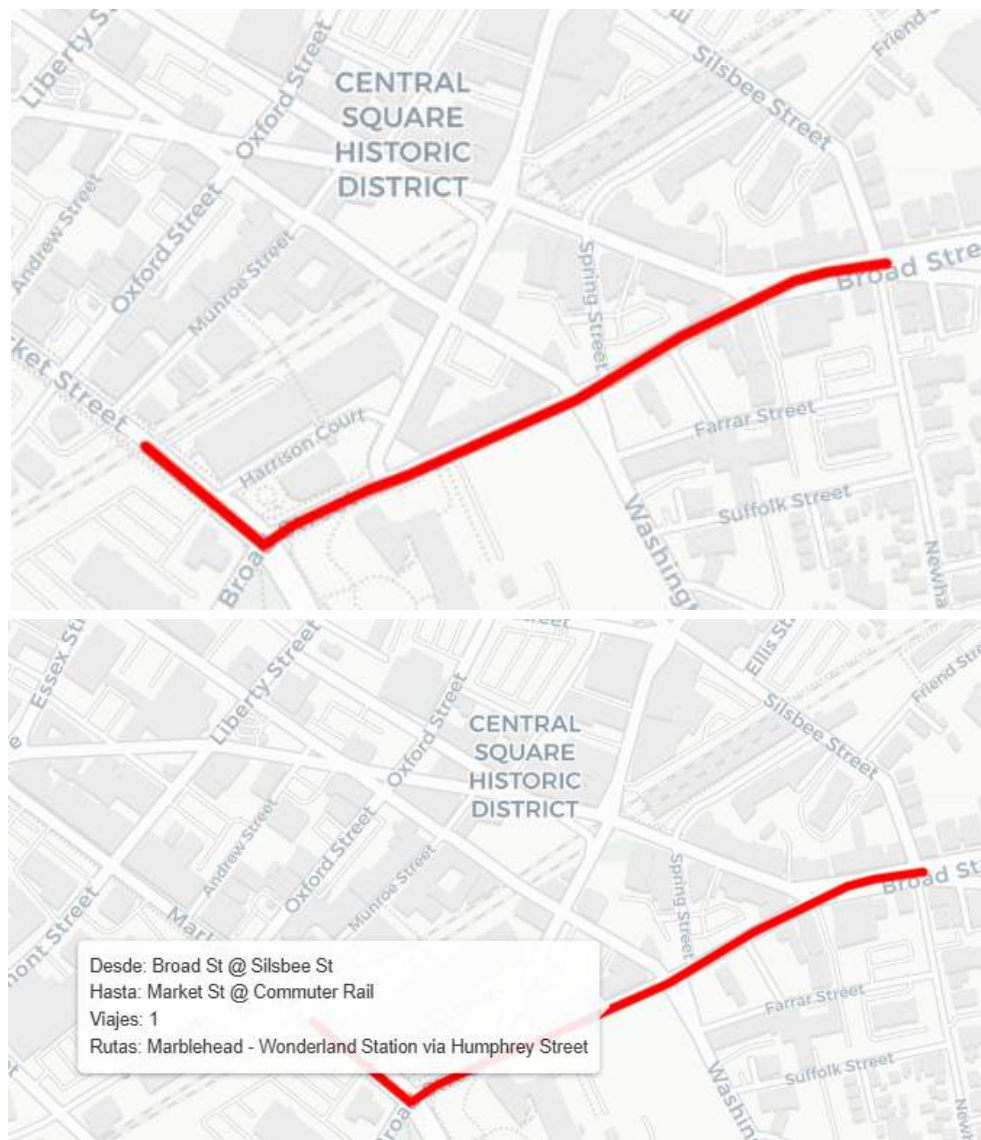


Figura 16: Segmento con la menor cantidad de viajes



Figura 17: Segmentos con la mayor cantidad de viajes

A primera vista con la Figura 17, nos hace sentido que los segmentos mostrados sean los más transitados según el itinerario, ya que conecta áreas densamente pobladas y de alta actividad comercial como Back Bay, Chinatown, Downtown Boston y Financial District, incluyendo zonas clave como parques. Además, estos tramos son compartidos por todas las variantes de la línea verde de Boston, lo cual creemos que aumenta el volumen de viajes acumulados en esos segmentos.

2.5 Análisis de la velocidad

Los datos estáticos contienen, de forma implícita, estimaciones de velocidad promedio para cada segmento que obtuvimos en la sección anterior, derivadas de los horarios planificados y las distancias entre paradas. Estas velocidades reflejan supuestos basados en datos históricos, y pueden variar según la hora del día, el tipo de jornada (laboral o no) o la época del año. Son fundamentales para predecir los tiempos de llegada a cada parada y para modelar el funcionamiento general del sistema.

En esta sección, nos proponemos extraer y analizar esas velocidades planificadas. Visualizaremos la velocidad promedio por segmento para la segunda semana de julio de 2025 y, posteriormente, compararemos los valores obtenidos entre los días laborables y los fines de semana. Trabajaremos con los scripts que se encuentran dentro de la carpeta `scheduled/velocity`.

2.5.1 Velocidad promedio por segmento

En el análisis que se presenta a continuación, buscamos extraer los datos de velocidad promedio por segmento, según el horario planificado. La siguiente consulta crea una vista materializada calculando la velocidad promedio en kilómetros por hora por segmento para la semana del 7 de julio al 13 de julio de 2025. El cálculo se basa en la diferencia de tiempo entre la llegada del vehículo a una parada y su salida desde la parada anterior, excluyendo así el tiempo que el transporte permanece detenido en la parada.

```
CREATE MATERIALIZED VIEW avg_segment_speed_kmh AS
SELECT AVG(s.distance_m / EXTRACT(EPOCH FROM (c.t_arrival -
c.t_departure))) * 3.6) AS avg_speed_kmh,
       s.distance_m,
       c.from_stop_id,
       c.from_stop_name,
       c.to_stop_id,
       c.to_stop_name,
```

```
s.geometry
FROM connections c
  JOIN segments s ON
    c.route_id = s.route_id AND
    c.direction_id = s.direction_id AND
    c.from_stop_id = s.start_stop_id AND
    c.to_stop_id = s.end_stop_id
WHERE c.date BETWEEN '2025-07-07' AND '2025-07-13'
  AND EXTRACT(EPOCH FROM (c.t_arrival - c.t_departure)) > 0
GROUP BY c.from_stop_id,
  c.from_stop_name,
  c.to_stop_id,
  c.to_stop_name,
  s.geometry,
  s.distance_m
```

A partir de la siguiente consulta, obtenemos un resumen general de los segmentos a utilizar durante el período analizado: cuántos se registraron, cuál fue la velocidad promedio y los valores mínimo y máximo de velocidad promedio observados por segmento.

```
SELECT
  COUNT(*) AS total_segments, -- 7794
  AVG(avg_speed_kmh) AS avg_speed, -- 17.05
  MIN(avg_speed_kmh) AS min_avg_speed, -- 0.28
  MAX(avg_speed_kmh) AS max_avg_speed -- 103.17
FROM avg_segment_speed_kmh
```

Luego, mediante la ejecución del *script* `visualize_avg_velocity.py`, es posible generar una representación visual de la velocidad promedio planificada a la que se recorren los distintos tramos de la red durante ese intervalo temporal.

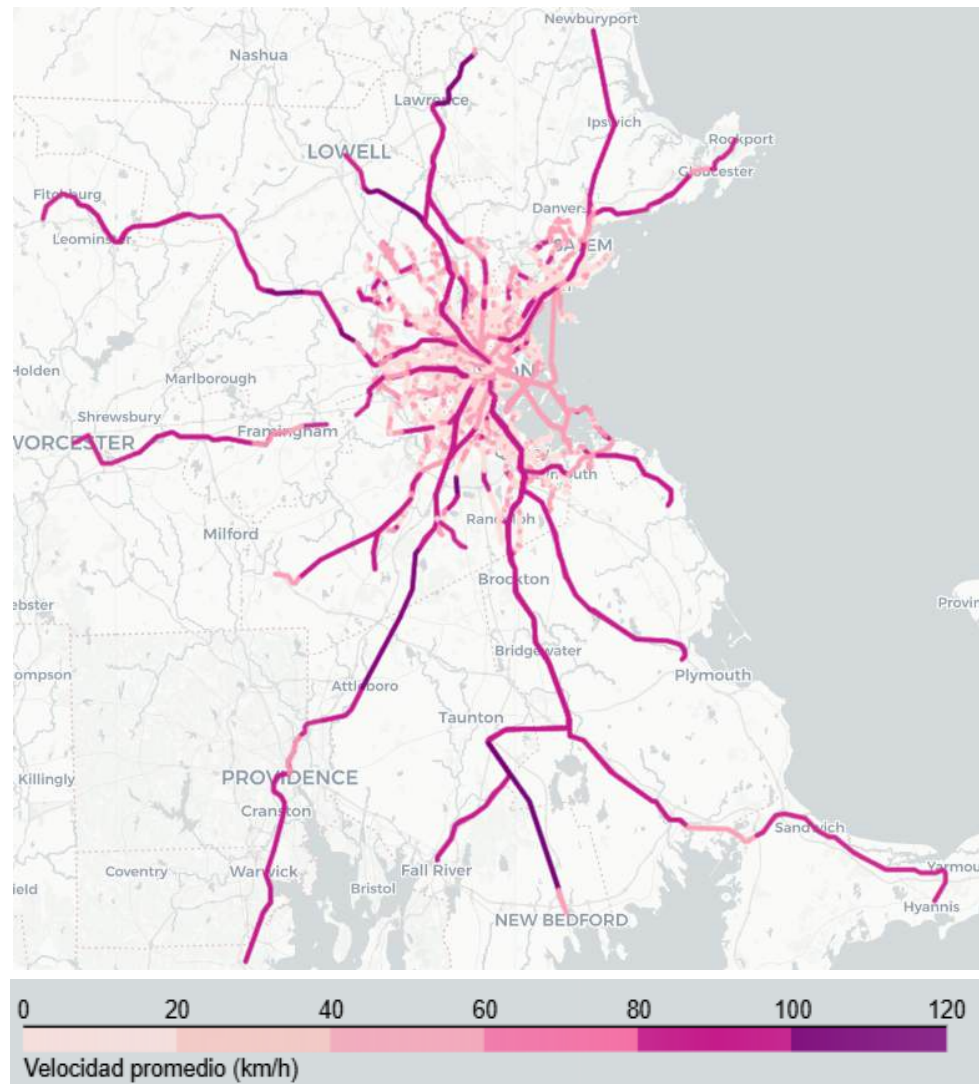


Figura 18: Gráfico de la velocidad promedio (km/h) por segmentos entre el 7 y 13 de julio.

De la Figura 18 podemos deducir que los tramos con velocidades más altas (superiores a 80 km/h), representados en tonos rojos intensos, se concentran mayoritariamente en los extremos del mapa, alejados del núcleo urbano de Boston y las que se encuentran en el núcleo central, corresponden a subtes o trenes de larga distancia (Figura 8 y 9). En contraste, los tramos con velocidades promedio entre 0 a 70 km/h predominan en la zona central de la ciudad, donde se superponen líneas de autobuses (Figura 10) y en zonas con alta densidad demográfica.

Podemos ver en qué segmento ocurre la velocidad promedio máxima (Figura 19) ejecutando el script `visualize_max_avg_velocity.py`. Podemos observar que ese tramo se corresponde con la ruta de un tren de larga distancia de la Figura 9.

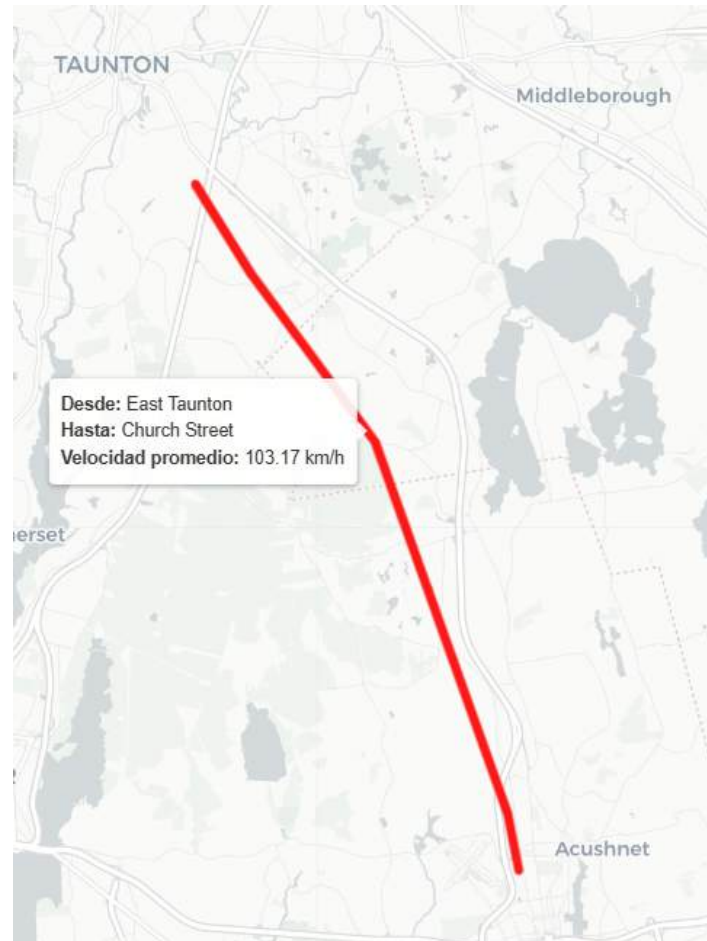


Figura 19: Segmento con velocidad promedio más alta

2.5.2 Comparación de la velocidad: días hábiles vs. fin de semana

Nuestro objetivo ahora es comparar el comportamiento del transporte, en términos de su velocidad promedio, entre los días hábiles y el fin de semana de la segunda semana de julio.

Como primer paso, se creó una vista materializada que estima la velocidad en km/h para cada segmento recorrido por los vehículos a partir de los tiempos de llegada y salida registrados.

```
CREATE MATERIALIZED VIEW segment_speed_kmh AS
SELECT (s.distance_m / EXTRACT(EPOCH FROM (c.t_arrival - c.t_departure))
* 3.6) AS speed,
       s.distance_m,
       c.from_stop_id,
       c.from_stop_name,
       c.t_departure,
       c.to_stop_id,
       c.to_stop_name,
       c.t_arrival,
       s.geometry
FROM connections c
      JOIN segments s ON
c.route_id = s.route_id AND
c.direction_id = s.direction_id AND
c.from_stop_id = s.start_stop_id AND
c.to_stop_id = s.end_stop_id
WHERE c.date BETWEEN '2025-07-07' AND '2025-07-13'
AND EXTRACT(EPOCH FROM (c.t_arrival - c.t_departure)) > 0
```

A partir de esta vista, podemos agrupar y ordenar los datos por tipo de día (hábil o fin de semana) y por hora, calculando la velocidad promedio en cada franja horaria con la siguiente consulta:

```
SELECT CASE EXTRACT(DOW FROM t_arrival AT TIME ZONE 'America/New_York')
        WHEN 0 THEN 'Weekend'
        WHEN 6 THEN 'Weekend'
```

```

        ELSE 'Weekday'
    END AS day_type,
    EXTRACT(HOUR FROM t_arrival AT TIME ZONE 'America/New_York') AS
hour,
    COUNT(*) AS num_segments,
    AVG(speed) AS avg_speed
FROM segment_speed_kmh
WHERE t_arrival >= '2025-07-07 00:00:00 America/New_York' AND t_arrival
< '2025-07-13 00:00:00 America/New_York'
GROUP BY day_type, hour
ORDER BY day_type, hour

```

Esta consulta se utiliza en el *script* `weekend_vs_weekday.py` que permite generar las Figuras 20 a 22.

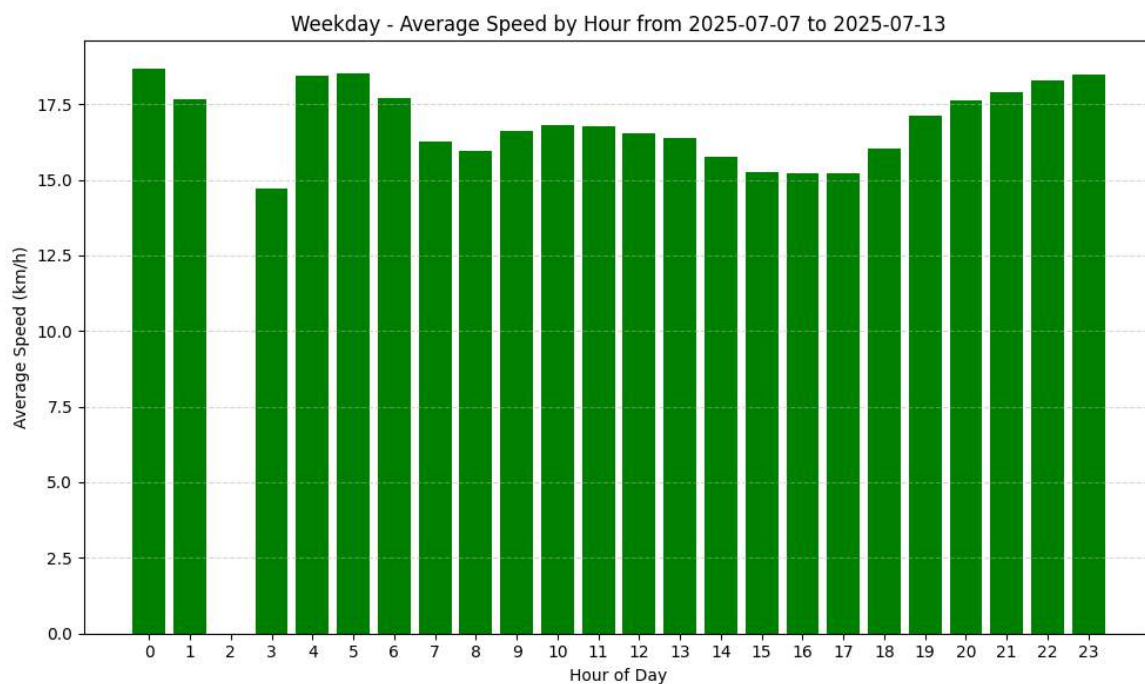


Figura 20: Velocidad promedio durante los días hábiles de la semana

En la Figura 20, se observa una distribución horaria estable en la mayoría de las franjas, con valores que comúnmente oscilan entre los 15 y los 19 km/h. Las horas con mayor velocidad promedio se concentran alrededor de la medianoche y de las 5 de la mañana, momentos en los que circula una menor cantidad de vehículos y, posiblemente, con menor densidad de tránsito. Sin embargo, destaca la ausencia total de registros a las

2:00 AM, lo cual se visualiza como una barra completamente vacía. Este fenómeno probablemente se deba a la suspensión del servicio durante esa franja horaria, una práctica común en muchas redes de transporte urbano durante los días laborables. En general, el comportamiento es regular y sin variaciones abruptas, lo que sugiere un sistema con velocidades bastante homogéneas en promedio a lo largo del día.

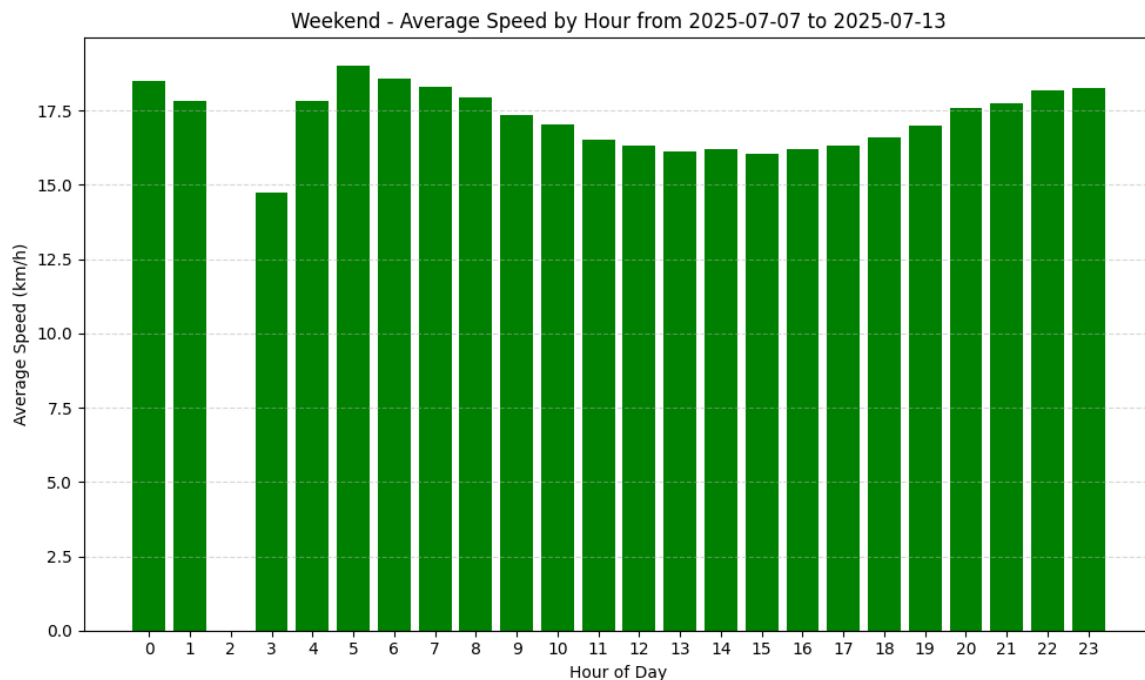


Figura 21: Velocidad promedio durante el fin de semana.

En la Figura 21, se identifican velocidades significativamente más elevadas durante la madrugada, en particular entre las 0:00 y las 5:00 AM, con valores de hasta 19.5 km/h. Esta tendencia puede explicarse por la menor congestión vehicular durante esas horas, así como por una posible menor cantidad de paradas o demoras. Además, podemos ver que los fines de semana tampoco tiene registros a las 2:00 AM, al igual que los días hábiles. Luego del amanecer, la velocidad disminuye levemente y a lo largo del día varía levemente entre los 16 y 18 km/h.

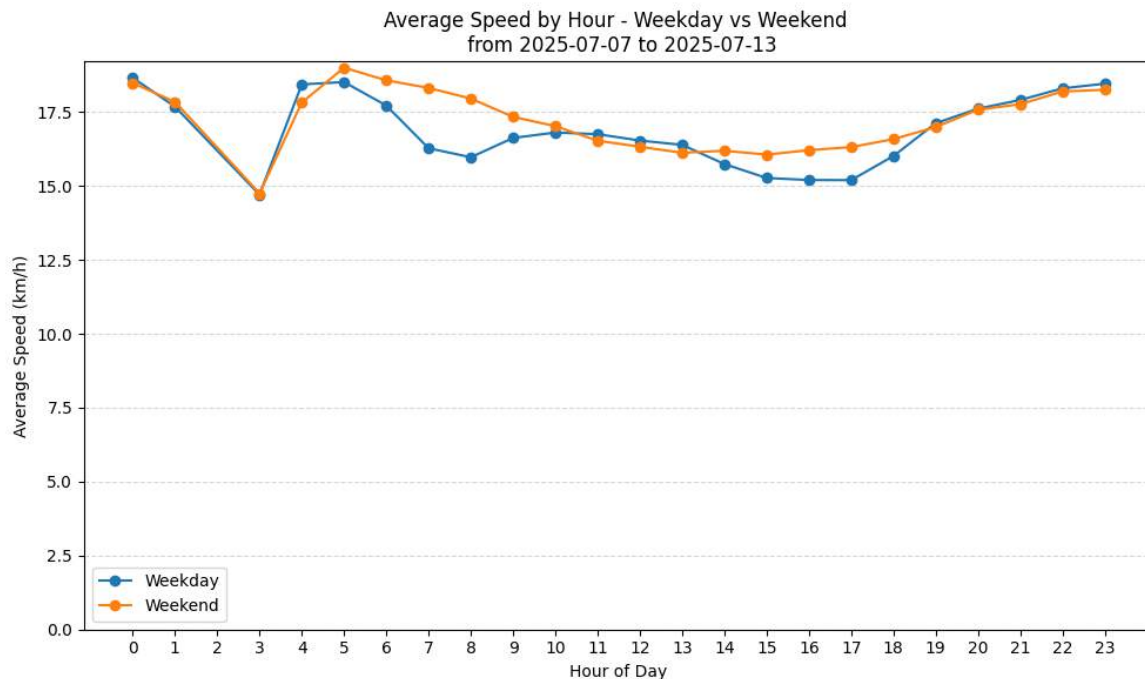


Figura 22: Comparación de la velocidad promedio entre los días hábiles y el fin de semana

La Figura 22 permite comparar de forma directa las diferencias en las velocidades promedio entre los días de semana y los fines de semana para cada hora del día. En líneas generales, no se observan grandes diferencias excepto en dos franjas horarias; entre las 5:00 AM y las 9:00 AM, y entre las 14:00 PM y las 18:00 PM.

Estas franjas horarias se interseccionan con la hora pico, cuando las personas salen de sus casas para ir a trabajar a la mañana, comenzando su jornada a la mañana para luego terminarla a la tarde, volviendo a sus casas. Las personas realizan este movimiento los días de semana, pero no los fines de semana. Esta correlación sugiere que la razón de esta diferencia es no solo la mayor carga al transporte público en estos horarios, sino también la mayor congestión de la vía por vehículos privados (que no son parte de nuestro *dataset*). En el tiempo intermedio, durante la jornada laboral, las diferencias son una vez más mínimas dado que no hay tanto movimiento. Teniendo en cuenta estas franjas, la diferencia de velocidad entre los días de semana y los fines de semana sigue siendo pequeña, con la mayor diferencia siendo de solamente 2 km/h a las 8:00 AM.

2.6 Identificación de duplicación de rutas

En esta sección, nos proponemos encontrar rutas duplicadas. Esto es, rutas por las que circulan varios viajes al mismo tiempo, y por ende existe la posibilidad de eliminar una de estas. Vamos a estar restringiendo el análisis al jueves 10 de julio de 2025, entre las 15:00 PM y las 16:00 PM, dado que este es un horario de mayor movimiento durante un día de semana. Todos los *scripts* de esta sección están en `scheduled/administrative` y `scheduled/duplicates`.

2.6.1 Elección de las zonas a analizar

Para comenzar, decidimos restringir el conjunto de los datos sobre los que vamos a trabajar a aquellas zonas con mayor densidad de viajes. El propósito de esto es, por un lado nos permite enfocarnos en las rutas que probablemente tengan duplicación, y por el otro nos limita el tamaño del *dataset* que estamos procesando. Para poder decidir cómo realizar la división, decidimos importar un *dataset* de áreas administrativas de Boston y sus alrededores.

Encontramos que el [sitio del gobierno del estado de Massachusetts](#) contiene data layers de distintos tipos, como información ambiental, infraestructura, y en particular [áreas administrativas](#). Descargamos el shapefile, lo que nos trae una carpeta zip con múltiples archivos. El que nos es de interés es `TOWNSSURVEY_POLY.shp`, que según el sitio nos indica contiene polígonos (otros archivos pueden representar los límites administrativos mediante líneas o puntos).

Para importar este archivo a nuestra base de datos, podemos abrir una terminal en la carpeta descargada y ejecutar los siguientes comandos:

```
> shp2pgsql -I -s 26986 TOWNSSURVEY_POLY.shp > insert.sql  
> psql -d mbtagtfs -c "\i insert.sql;"
```

Esto creará una tabla `townsurvey_poly` con las distintas municipalidades de Massachusetts con las columnas, entre otras, `gid`, `town`, `county`, `island` (0 o 1), y `geom`, que es la geometría del área. Para dibujar estas geometrías, preparamos el archivo `map_administrative_areas.py`, cuyo resultado podemos ver a continuación en la Figura 23.

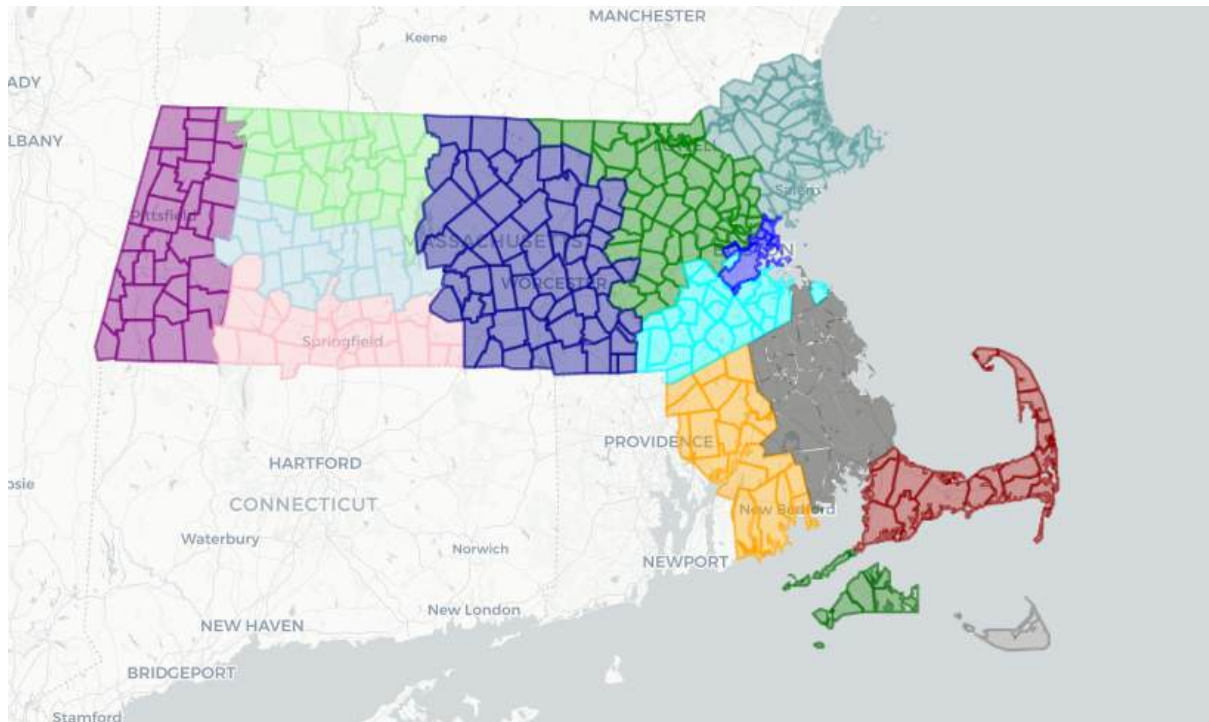


Figura 23: Áreas administrativas de Massachusetts

Estos datos se extienden mucho más allá que la zona en la que estamos trabajando, nosotros nos limitaremos a Boston y sus alrededores y excluimos las islas. El query que trae las zonas de interés es el siguiente:

```
SELECT * FROM townsurvey_poly
WHERE town = 'BOSTON' AND island = 0
```

Esto nos trae la mayor parte de la zona azul.

2.6.2 Definición del método de detección de hotspots

Con las zonas a analizar decididas, nuestro siguiente objetivo es encontrar *hotspots*, o zonas por las que pasan varios viajes al mismo tiempo. Nuestra primera iteración en esto

consistía en utilizar las paradas, contando la cantidad de viajes que pasan por cada parada en el rango horario de interés, pero al visualizar las paradas como se ve en la Figura 24, encontramos que muchas veces las paradas se encuentran cercanas entre sí. Podemos ver en la figura como por ejemplo, en Haverhill Street (un pequeño tramo entre Canal Street y I-93 S), se encuentra un cluster de más de 50 paradas a lo largo de una misma cuadra.

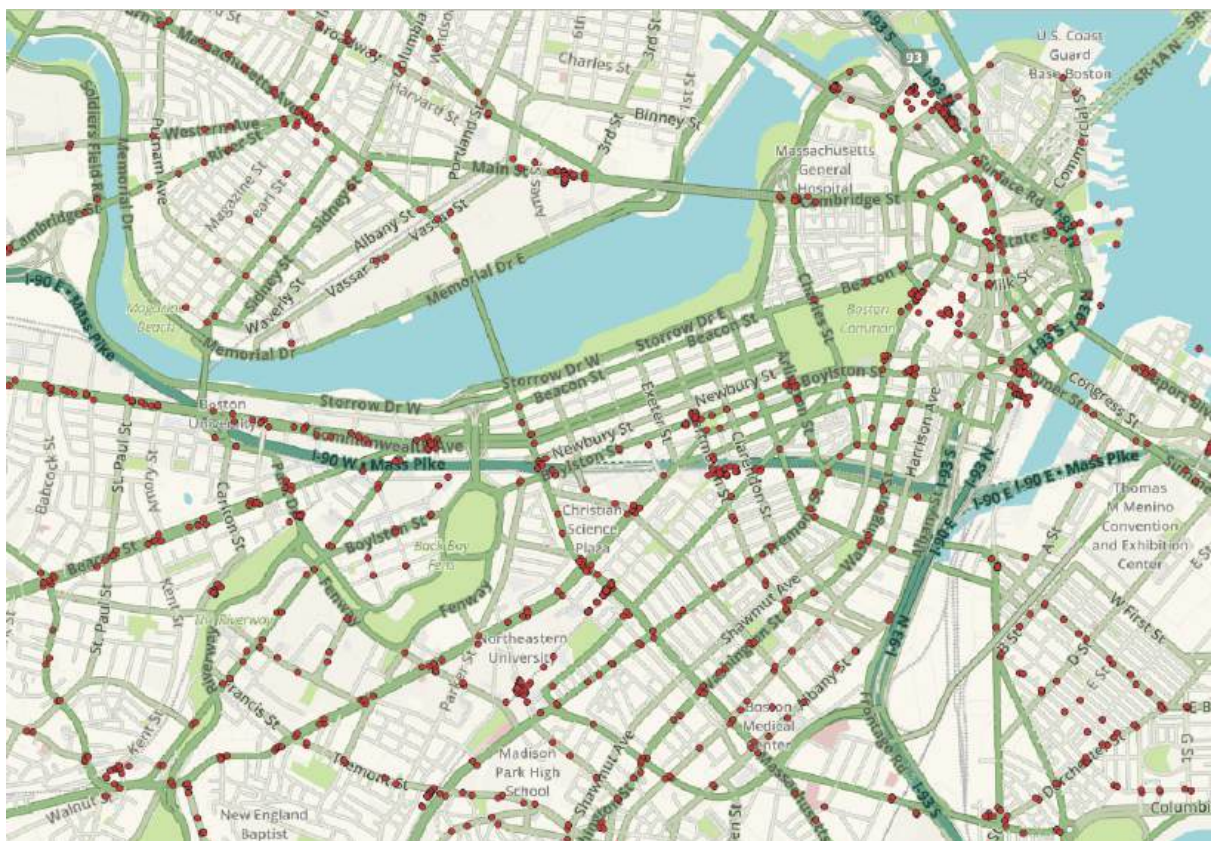


Figura 24: Visualización de todas las paradas de la tabla *stops*, enfocada en el centro de Boston.

Este tipo de casos se deberían agrupar, tal que si pasa solo un viaje por cada una de estas paradas, podamos considerar como que en realidad pasaron más de 50 viajes en esta zona, por ende decidimos descartar esta idea.

Segundo consideramos utilizar el análisis de frecuencia de viajes por segmento, sin embargo esto también lo descartamos dado que no sería capaz de detectar viajes que pasen por calles adyacentes y paralelas. Entonces avanzamos con una tercera idea, que

consiste en dividir el área analizar en subsecciones formando una grilla, y luego analizar la cantidad de viajes que pasan por cada celda de la grilla.

PostGIS contiene la función `ST_SquareGrid(integer, geometry)` para realizar exactamente esto, los parámetros siendo el tamaño de cada celda y una geometría que indique el área en el que se genera la grilla. El resultado es una tabla con una grilla que cubre un área cuadrada alrededor de la geometría especificada. Nosotros nos quedaremos solo con las celdas que se interseccionan con nuestro área de análisis. El query para generar la grilla es el siguiente:

```
WITH area_to_analyze AS (  
  SELECT ST_Union(geom) AS geom  
  FROM townssurvey_poly  
  WHERE town = 'BOSTON' AND island = 0  
) ,  
grid_full AS (  
  SELECT geom, i, j  
  FROM ST_SquareGrid(  
    750,  
    (SELECT a.geom FROM area_to_analyze a)  
  ) AS sg(geom, i, j)  
)  
SELECT g.i, g.j, ST_Transform(g.geom, 4326) AS geo4326  
FROM grid_full g, area_to_analyze a  
WHERE ST_Intersects(g.geom, a.geom)
```

Este query se encuentra dentro del archivo `map_grid_empty.py`, que nos permite generar la visualización de la grilla que se muestra en la figura 25. Notar que estamos usando la función `ST_Transform()` para convertir la geometría entre distintos SRIDs, la razón para esto es que la geometría de los límites administrativos de Massachusetts se encuentra en [SRID 26986, Massachusetts Mainland](#), mientras que Folium precisa trabajar en [SRID 4326, WGS84](#). Trabajaremos en el SRID de Massachusetts, pero convertiremos los resultados a WGS84 al momento de pasar los datos a Folium.

Elegimos un tamaño de grilla de 750x750 metros (Figura 25) dado que consideramos que esta es una distancia aceptable para que un peatón camine hacia la parada de su viaje. Con la grilla establecida, el siguiente paso es calcular la cantidad de viajes que pasan por cada celda de la grilla en nuestra franja horaria de análisis.

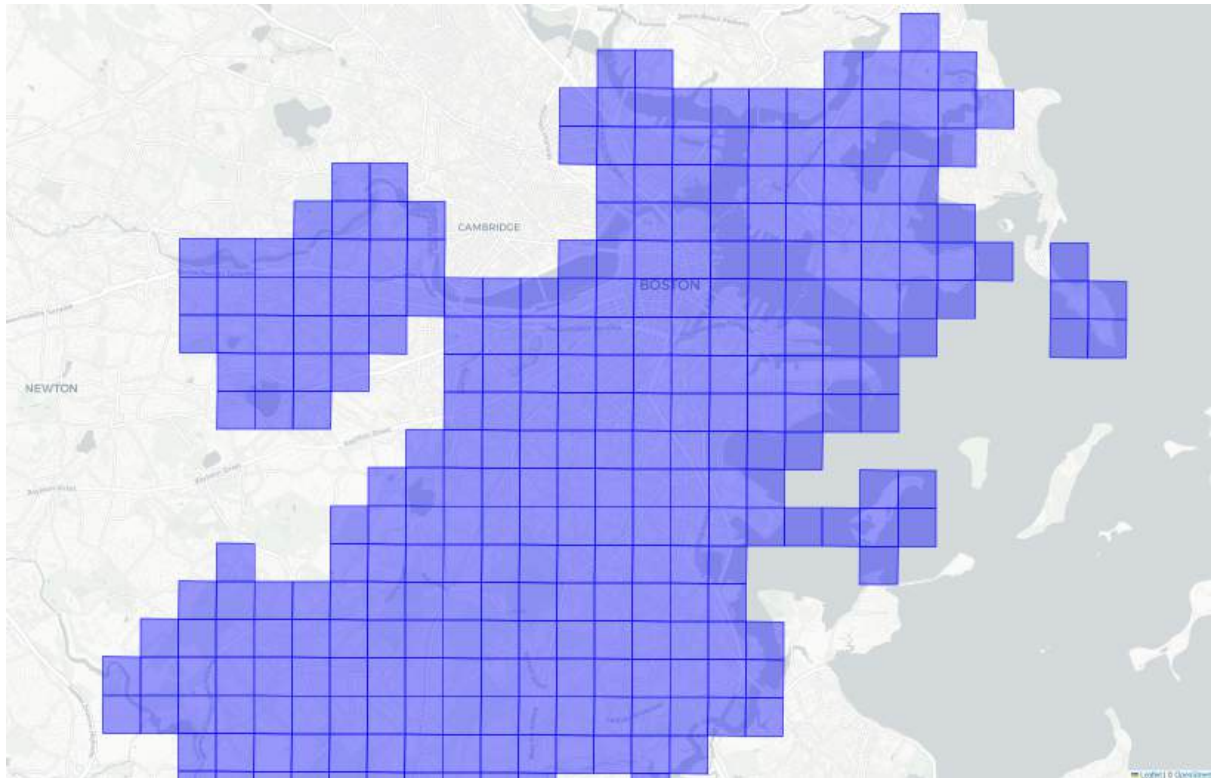


Figura 25: Visualización de la grilla con celdas de 750 metros de lado sobre Boston.

2.6.3 Cálculo de viajes por celda y detección de hotspots

Para calcular la cantidad de viajes que pasa por cada celda, no podemos simplemente tomar la geometría (o “*shape*”) de cada viaje de la vista *shapes_aggregated*, dado que esto incluiría el viaje de punta a punta y no lo restringe a nuestra franja horaria a analizar. Para esto, comenzamos por crear una tabla de ayuda con los viajes restringidos, cada uno con una geometría que incluya los segmentos que forman parte del viaje entre las 15:00 PM y las 16:00 PM. También filtramos los viajes que no estén activos el Jueves 10 de

Julio, según indique el calendario del servicio asociado. El query para crear esta tabla es el siguiente:

```
CREATE TABLE trips_active_restricted AS (
  SELECT
    t.*,
    ST_Union(ST_Transform(ST_SetSRID(s.geometry, 4326), 26986)) AS
    geom_restricted
  FROM trips t
    JOIN stop_times st ON t.trip_id = st.trip_id
    JOIN segments s ON s.route_id = t.route_id
      AND s.start_stop_id = st.stop_id
    JOIN calendar c ON c.service_id = t.service_id
  WHERE st.departure_time >= '15:00'::interval
    AND st.arrival_time <= '16:00'::interval
    AND '2025-07-10'::date >= c.start_date
    AND '2025-07-10'::date <= c.end_date
    AND c.thursday::text = 'available'
  GROUP BY t.trip_id
)
```

En este query, buscamos para cada viaje los segmentos dentro de la franja horaria y los juntamos en una sola geometría; *geom_restricted*. Notemos que la tabla *segments*, creada en la Sección 2.4 con la herramienta *gtfs_functions*, tiene su geometría en SRID 0, pero los valores realmente son SRID 4326. De todos modos, para simplificar el procesamiento de estos datos, lo convertimos al SRID 26986 de Massachusetts.

La tabla resultante tiene 1464 viajes. Podemos visualizar estos utilizando el script `map_trips_restricted.py`, cuyo resultado podemos ver en la Figura 26. De ahora en adelante, acotamos el procesamiento de viajes a solamente los 1464 de esta tabla.

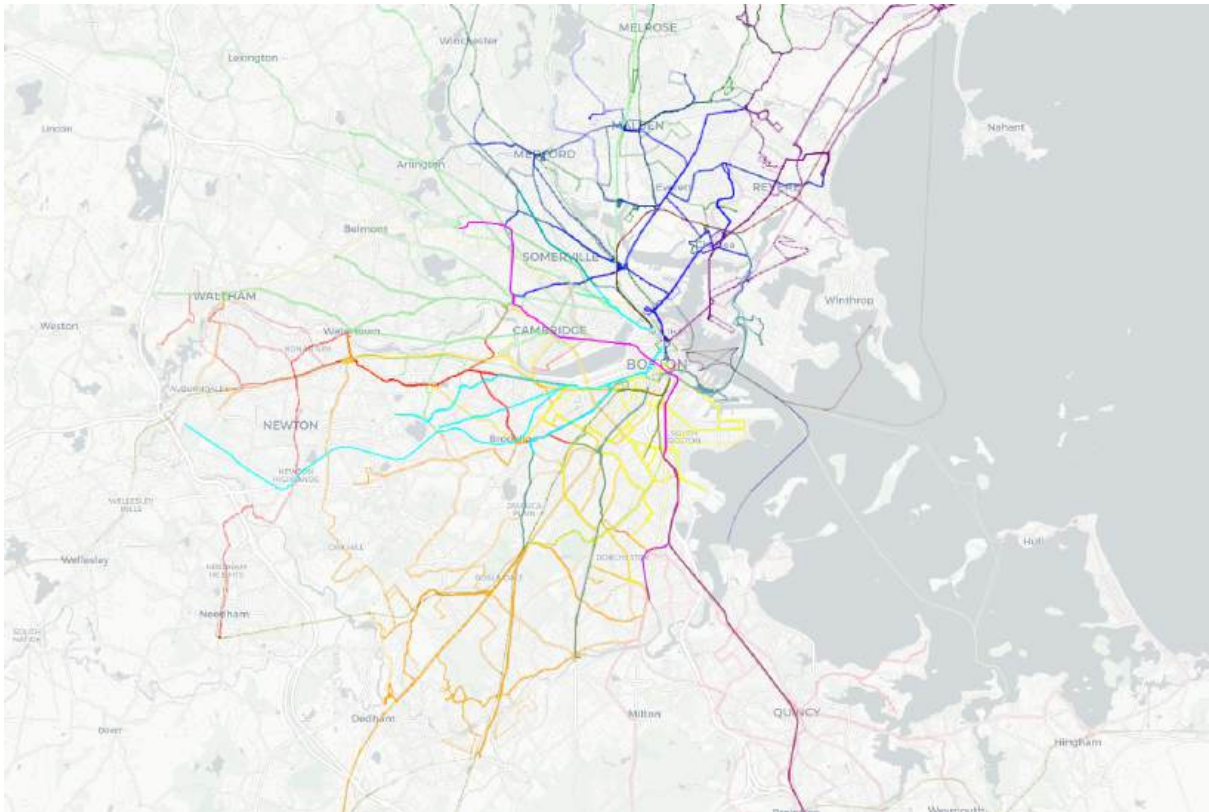


Figura 26: Visualización de la tabla *trips_active_restricted*.

Teniendo los viajes restringidos creados, pasamos a crear una tabla para la grilla, con columnas para contar la cantidad de viajes y un *array* con los IDs de dichos viajes:

```
CREATE TABLE route_count_grid (
  i          BIGINT NOT NULL,
  j          BIGINT NOT NULL,
  geom       GEOMETRY NOT NULL,
  trip_count BIGINT,
  trip_ids   TEXT[],
  PRIMARY KEY (i, j)
);
```

Primero insertamos la grilla en la tabla, filtrando las zonas de interés definidas y sin contar los viajes todavía:

```
WITH area_to_analyze AS (
  SELECT ST_Union(geom) AS geom
  FROM townssurvey_poly
```

```

WHERE town = 'BOSTON' AND island = 0
),
grid_full AS (
  SELECT geom, i, j
  FROM ST_SquareGrid(
    750,
    (SELECT a.geom FROM area_to_analyze a)
  ) AS sg(geom, i, j)
)
INSERT INTO route_count_grid (i, j, geom)
SELECT g.i, g.j, g.geom
FROM grid_full g, area_to_analyze a
WHERE ST_Intersects(g.geom, a.geom)

```

Para acelerar la siguiente operación de cálculo de viajes por celda, creamos un índice GiST sobre las celdas de la grilla:

```

CREATE INDEX grid_geom_gist_idx ON route_count_grid USING GIST (geom)

```

Y por último, calculamos los viajes que intersectan cada celda y llenamos los datos restantes de la tabla:

```

UPDATE route_count_grid g SET (trip_count, trip_ids) = (
  SELECT COUNT(*), ARRAY_AGG(t.trip_id)
  FROM trips_active_restricted t
  WHERE ST_Intersects(g.geom, t.geom_restricted)
)

```

2.6.4 Visualización de hotspots

Ahora procedemos a crear visualizaciones para los *hotspots*. Utilizaremos grillas de distintos tamaños para poder tener una mejor idea del movimiento del tráfico, pero el subsiguiente análisis de duplicados lo realizaremos con el tamaño de 750.

Creamos el script `map_grid_heatmap.py`, que genera un mapa interactivo en base a la tabla `route_count_grid` creada anteriormente. Este mapa permite hacer click en cada celda para visualizar las coordenadas (i, j) que la identifican y la cantidad de viajes.

Comencemos por visualizar la grilla de tamaño 250x250 metros. En la Figura 27 vemos el resultado, que nos muestra cómo resaltan las arterias de tránsito principales. Las celdas con mayor cantidad de viajes son la $(i, j) = (944, 3602)$ en el centro de Boston con 149 viajes, seguida por la $(i, j) = (927, 3578)$, que se encuentra no en el centro de Boston sino significativamente más al suroeste entre las calles Arborway y Washington Street, con 138 viajes. Hay una gran cantidad de celdas que tienen un *trip_count* de 0, mostrando una falta de transporte público en los suburbios.

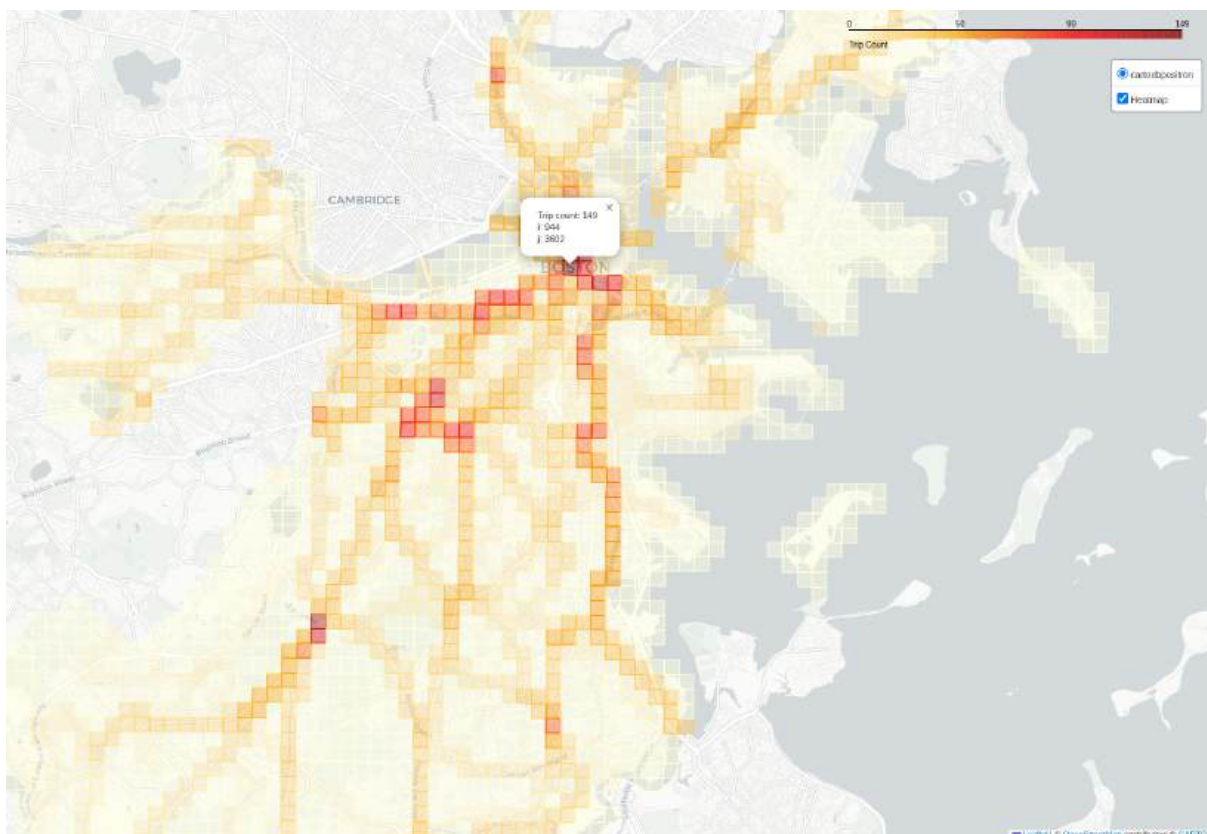


Figura 27: Visualización de hotspots con una grilla de 250x250.

Si disminuimos el tamaño de la grilla a 100x100 metros, podemos ver en la Figura 28 como resaltan mejor las distintas rutas individuales.

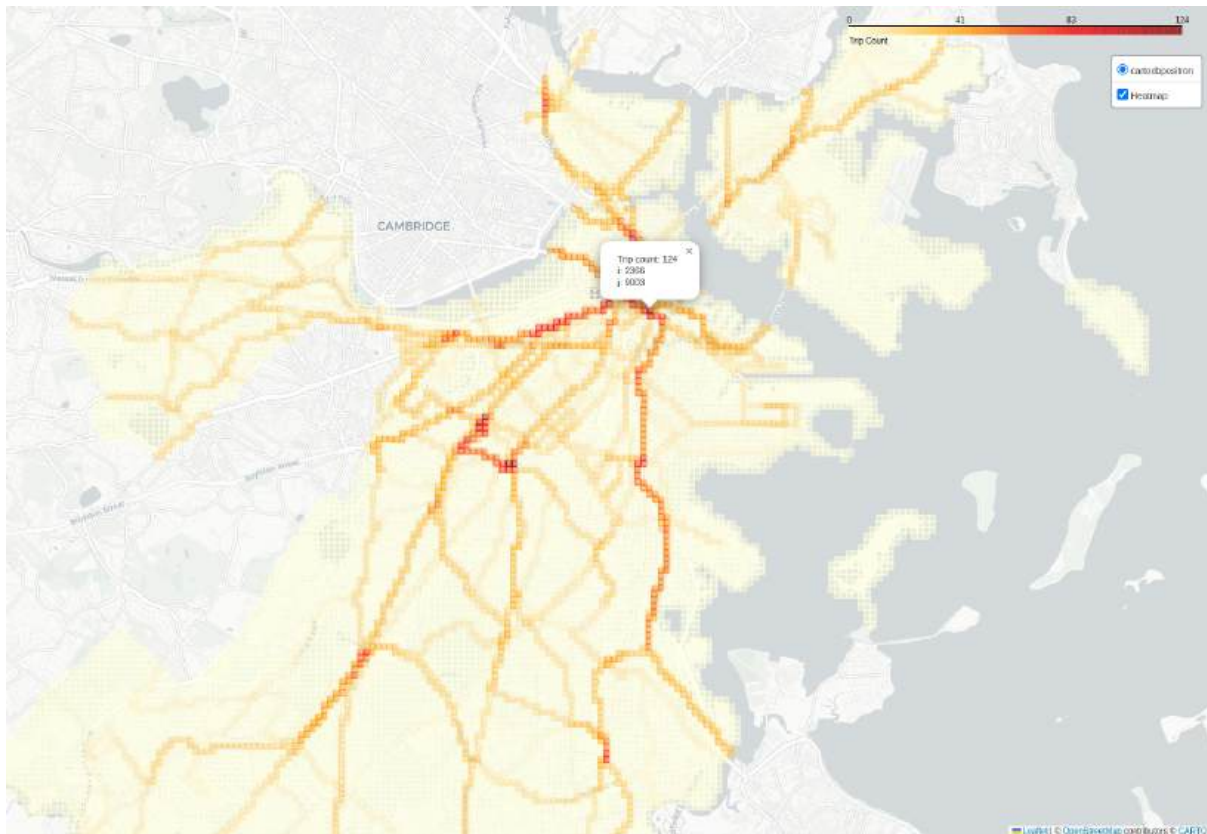


Figura 28: Visualización de hotspots con una grilla de 100x100.

La celda con mayor *trip_count* sigue siendo en el centro, con coordenadas $(i, j) = (2366, 9003)$, en una ubicación más levemente al sureste que el principal hotspot anterior. Esta variación significativa nos indica que nos conviene incrementar el tamaño de la grilla.

En la Figura 29, mostramos la grilla con celdas de 750 metros. La celda predominante ahora es la de coordenadas $(i, j) = (315, 1200)$, con 179 viajes y ubicada en el mismo lugar que la celda predominante de la grilla con tamaño 250. Esto nos indica que 750 es un buen tamaño para continuar. Tomaremos entonces las 5 celdas más transitadas, que podemos obtener con el siguiente query, cuyo resultado se indica en la figura 30.

```
SELECT i, j, trip_count, trip_ids
FROM route_count_grid
ORDER BY trip_count DESC;
```

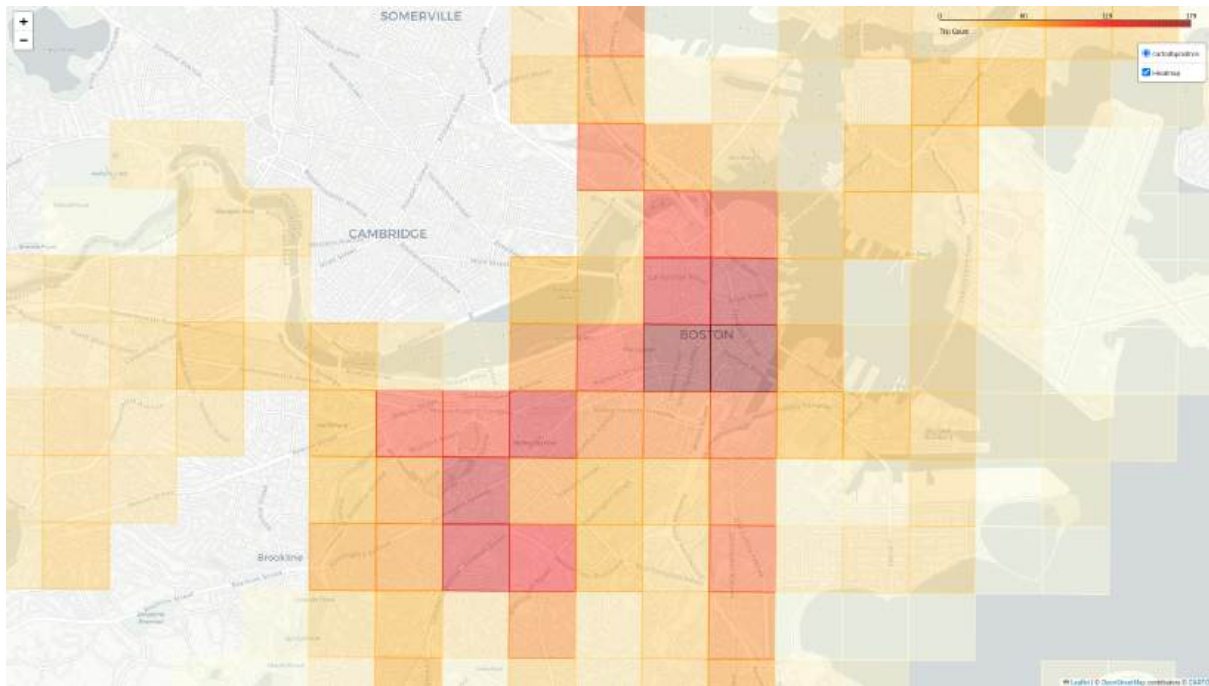


Figura 29: Visualización de hotspots con una grilla de 750x750.

	i	j	trip_count	trip_ids
1	315	1200	179	{69111007,69111010,69111014,69111017,69111114,69111117,69111138,6911...
2	314	1200	176	{69111314,69111346,69111356,69111378,69111379,69111383,69111398,6911...
3	315	1201	156	{69329331,69329356,69329422,69329428,69329434,69329439,69329440,6932...
4	311	1198	149	{69110538,69110540,69110541,69110542,69110543,69110558,69110574,6911...
5	309	1192	140	{69110558,69110574,69110599,69110601,69110620,69110621,69110640,6911...
6	311	1197	139	{69110768,69110771,69110772,69110773,69110774,69110775,69110778,6911...
7	314	1201	138	{69329331,69329356,69329422,69329428,69329434,69329439,69329440,6932...
8	312	1199	138	{69110538,69110540,69110541,69110542,69110543,69110558,69110568,6911...
9	312	1197	123	{69110768,69110771,69110772,69110773,69110774,69110775,69110778,6911...
10	314	1202	108	{69329331,69329356,69329422,69329428,69329433,69329434,69329439,6932...

Figura 30: Resultado de la búsqueda de las celdas más predominantes.

Notemos que por más que las celdas más significativas sumen 800 viajes, hay cientos de repetidos por lo que el número real de viajes en este área es de 549:

```
WITH trip_id_arrays AS (
  SELECT trip_ids FROM route_count_grid
  ORDER BY trip_count DESC LIMIT 5
)
SELECT DISTINCT unnest(trip_ids) FROM trip_id_arrays;
```

Podemos visualizar estos viajes usando el *script* `map_high_traffic_trips.py`, que los agrupa en FeatureGroups por ID de servicio.

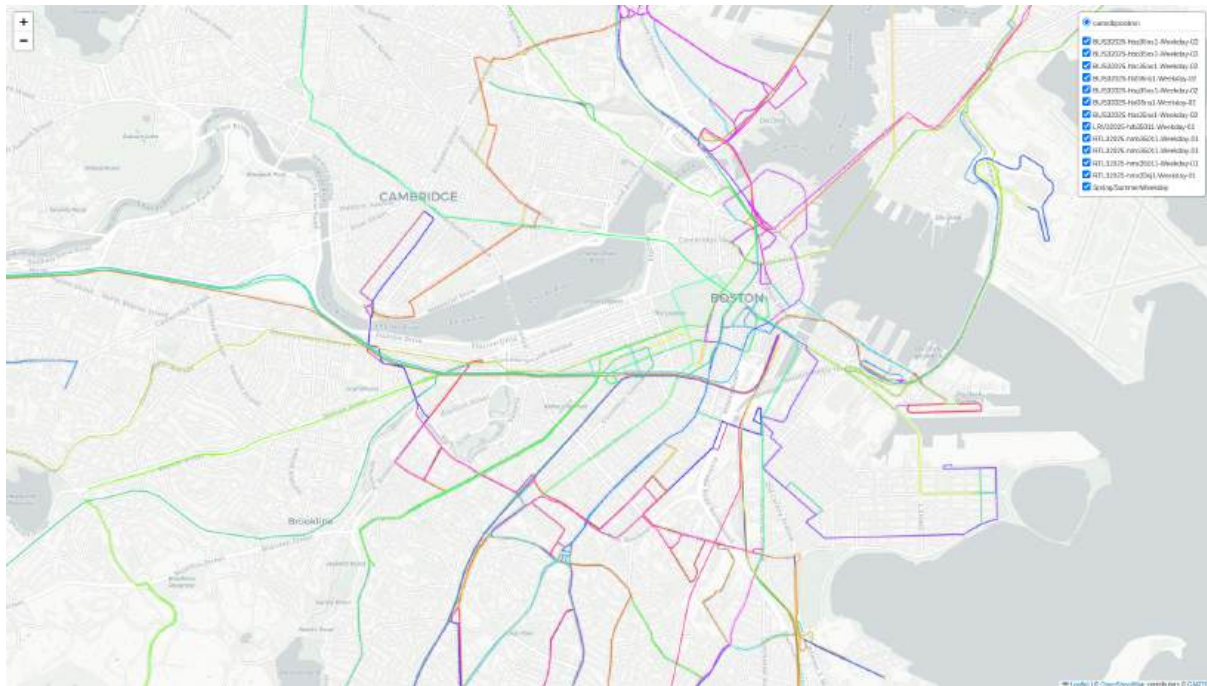


Figura 31: Visualización de los viajes que pasan por las celdas predominantes.

Lo primero que notamos al ver esta imagen es los servicios que incluye. En particular, el servicio *Spring/SummerWeekday* consiste de varios trenes de larga distancia (con `route_type` = 2) saliendo de la estación *South Station*. Esto nos indica que los trenes de larga distancia están conectados a las zonas de mayor concentración de viajes, lo que habla de la buena planificación del transporte público de Boston.

Lo segundo que notamos es los servicios de buses con nombres similares, que al realizar mayor investigación encontramos que múltiples de estos servicios pueden estar encargados de la misma línea o ruta. Por ejemplo, la ruta (o `route_id`) 22 es servida por ambos *BUS32025-hba35ns1-Weekday-02* y *BUS32025-hbc35ns1-Weekday-02* (notar la diferencia de “hba” contra “hbc”).

Lo tercero que notamos, al mirar más de cerca, es que hay líneas que aparentan ser duplicadas siguiendo casi el mismo camino, como por ejemplo las líneas violeta y naranja

que se muestra en la Figura 32, pero debemos recordar que estas consisten de la ida y la vuelta de una misma ruta. En este caso particular, es la ruta 450 del servicio *BUS32025-hbl35ns1-Weekday-02*, y dado que son viajes con dirección opuesta no podemos considerarlos como duplicados.

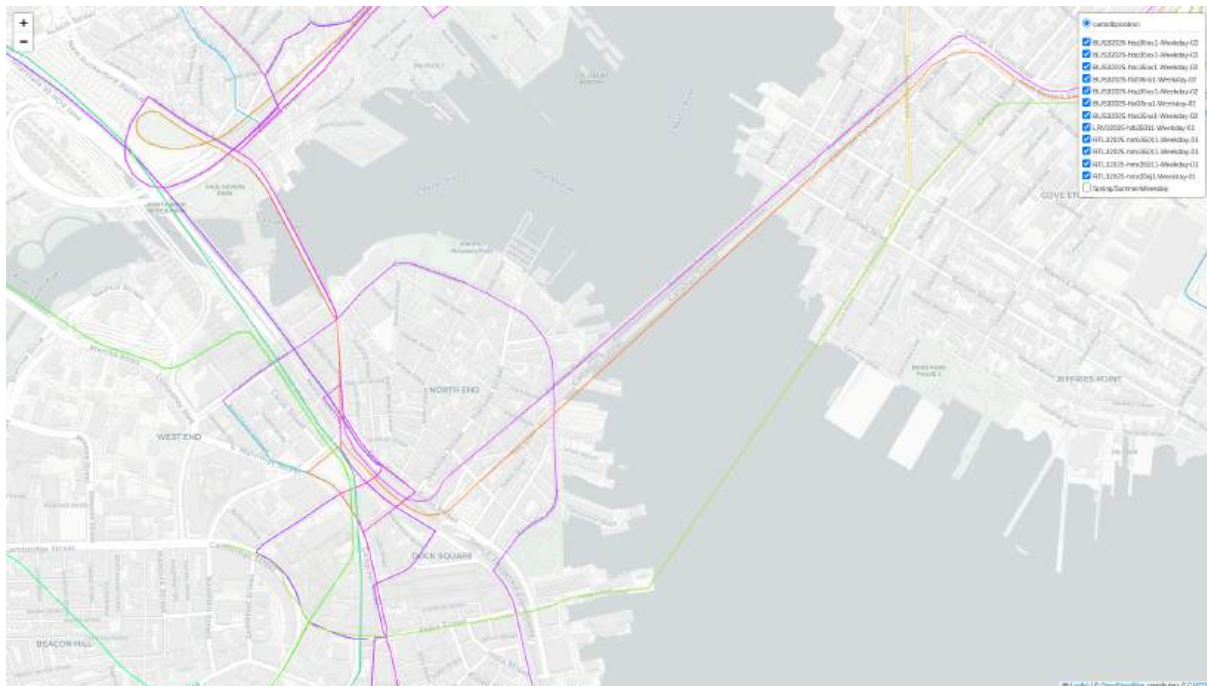


Figura 32: Ida y vuelta para la ruta 450 servicio *BUS32025-hbl35ns1-Weekday-02*.

Por último, podemos encontrar lo que parecen rutas duplicadas siguiendo líneas paralelas y verificando que no pertenezcan a la misma ruta. En la Figura 33, observamos los trips con IDs 69329770 y 69110621, que ambos pasaron por *Huntington Avenue* durante el rango horario, pero pertenecen a rutas y servicios distintos (ruta *Green-E* del servicio *LRV32025-hlb35011-Weekday-01* y ruta 39 del servicio *BUS32025-hbs35ns1-Weekday-02* respectivamente). En la Figura 34, observamos otro posible par duplicado con las rutas 22 y 44 ambas del servicio *BUS32025-hbc35ns1-Weekday-02*, que comparten significativa cantidad de sus recorridos a lo largo de *Columbus Avenue*.

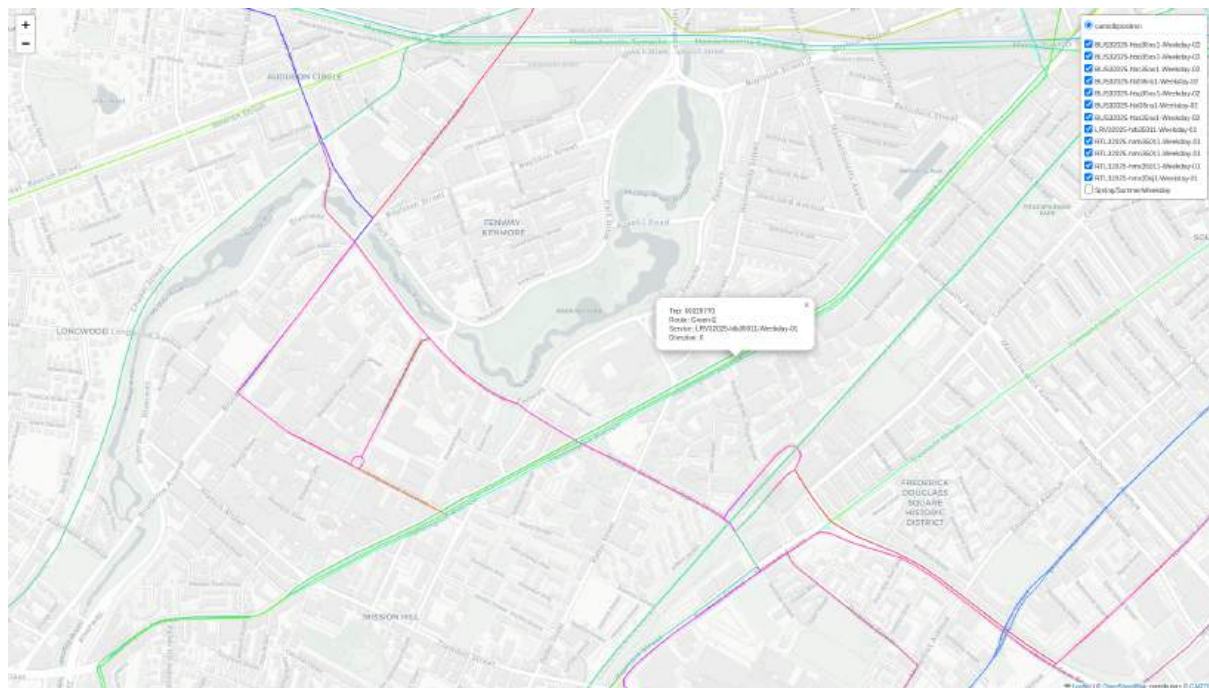


Figura 33: Visualización de las rutas *Green-E* y 39 a lo largo de *Huntington Avenue*.

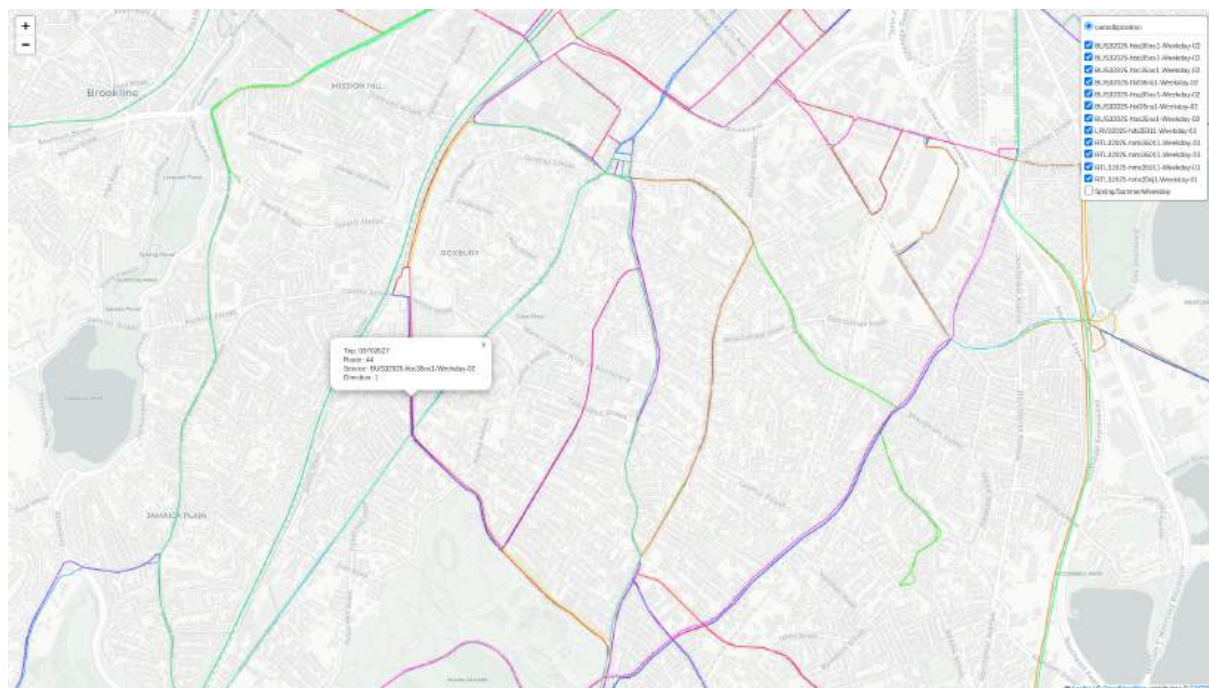


Figura 34: Visualización de rutas 22 y 44 ambas del servicio *BUS32025-hbc35ns1-Weekday-02*

Estas visualizaciones sin embargo no nos alcanzan para concluir si estas rutas son duplicadas, por ende decidimos agregar un último paso de filtrado fino a nuestro análisis para determinar mejor si es posible eliminar una de las rutas como duplicada.

2.6.5 Detección de duplicadas por buffer de las paradas

Algo que no estuvo presente en las visualizaciones del análisis de duplicación hasta ahora son las paradas. Sin embargo, estas son un punto importante, dado que no tiene sentido marcar como duplicadas dos líneas que pasan por la misma calle, pero donde una es una línea express sin paradas mientras la otra tiene una parada cada 500 metros. Para analizar esto, decidimos considerar el área a la que cada ruta brinda servicio, calculando esta como un buffer sobre las paradas.

La técnica consiste en generar para cada ruta una geometría de tipo MULTIPPOINT que junte todos los puntos de sus paradas. Luego se usa ST_Buffer para generar un polígono o multi-polígono con el área que se encuentra a 750 metros de cualquier parada de la ruta. Por último, podemos buscar pares de rutas cuyos buffers se intersectan, calcular el área de la intersección y comparar si este área es mayor a un porcentaje del área del buffer de cada una de las dos rutas del par.

Comenzamos por generar una tabla que agregue los puntos de paradas y su buffer:

```
CREATE TABLE high_traffic_trips_with_stops_geom AS (
  WITH trip_id_arrays AS (
    SELECT trip_ids FROM route_count_grid
    ORDER BY trip_count DESC LIMIT 5
  ),
  cte AS (
    SELECT
      t.*,
      ST_Collect(ST_Transform(ST_SetSRID(s.stop_loc,
4326)::geometry, 26986)) AS stops_geom
    FROM trips t
    JOIN stop_times st ON t.trip_id = st.trip_id
    JOIN stops s ON s.stop_id = st.stop_id
    WHERE t.trip_id IN (
      SELECT unnest(trip_ids) FROM trip_id_arrays
    )
    GROUP BY t.trip_id
```

```

)
SELECT
    cte.*,
    ST_Buffer(cte.stops_geom, 750) AS stops_geom_buffer,
    ST_Area(ST_Buffer(cte.stops_geom, 750)) AS
stops_geom_buffer_area
FROM cte
)

```

Notar el uso de la función de agregación ST_Collect, que junta los puntos en un MULTIPOINT, en vez de ST_Union, que realiza procesamiento para simplificar la geometría uniendo áreas que se intersectan, dado que no precisamos dicha simplificación para puntos.

Para acelerar las siguientes operaciones, creamos un índice sobre la geometría del buffer:

```

CREATE INDEX traffic_trips_with_stops_geom_idx ON
high_traffic_trips_with_stops_geom USING GIST (stops_geom_buffer)

```

Y finalmente, utilizamos el siguiente query para encontrar tuplas en la tabla high_traffic_trips_with_stops_geom cuyos buffers se intersectan en por lo menos el 60% del área del más pequeño. Decidimos comparar contra el área del más pequeño y no el más grande para poder encontrar casos donde una línea sirve a un subconjunto de otra, y elegimos 60% empíricamente en base a la cantidad de tuplas del resultado de la query.

```

WITH trip_pairs AS (
    SELECT
        t1.trip_id AS trip1_id,
        t2.trip_id AS trip2_id,
        t1.shape_id AS trip1_shape_id,
        t2.shape_id AS trip2_shape_id,
        t1.service_id AS trip1_service_id,
        t2.service_id AS trip2_service_id,
        t1.route_id AS trip1_route_id,

```



```

        t2.route_id AS trip2_route_id,
        t1.trip_headsign AS trip1_headsign,
        t2.trip_headsign AS trip2_headsign,
        t1.direction_id AS trip1_direction,
        t2.direction_id AS trip2_direction,
        t1.stops_geom AS trip1_stops_geom,
        t2.stops_geom AS trip2_stops_geom,
        t1.stops_geom_buffer AS trip1_stops_geom_buffer,
        t2.stops_geom_buffer AS trip2_stops_geom_buffer,
        t1.stops_geom_buffer_area AS trip1_stops_geom_buffer_area,
        t2.stops_geom_buffer_area AS trip2_stops_geom_buffer_area,
        ST_Intersection(t1.stops_geom_buffer, t2.stops_geom_buffer) AS
buffer_intersection,
        ST_Area(ST_Intersection(t1.stops_geom_buffer,
t2.stops_geom_buffer)) AS buffer_intersection_area,
        sa1.shape AS trip1_shape,
        sa2.shape AS trip2_shape
    FROM high_traffic_trips_with_stops_geom t1
    JOIN high_traffic_trips_with_stops_geom t2
    ON t1.trip_id < t2.trip_id
    AND ST_Intersects(t1.stops_geom_buffer, t2.stops_geom_buffer)
    AND t1.service_id != t2.service_id
    AND t1.route_id != t2.route_id
    AND t1.direction_id = t2.direction_id
    JOIN shapes_aggregated sa1 ON sa1.shape_id = t1.shape_id
    JOIN shapes_aggregated sa2 ON sa2.shape_id = t2.shape_id
)
SELECT DISTINCT ON (tp.trip1_service_id, tp.trip1_route_id,
tp.trip2_service_id, tp.trip2_route_id)
    (tp.buffer_intersection_area * 100 /
LEAST(tp.trip1_stops_geom_buffer_area,
tp.trip2_stops_geom_buffer_area)) AS intersection_percentage,
    tp.*
FROM trip_pairs tp
WHERE (tp.buffer_intersection_area * 100 /
LEAST(tp.trip1_stops_geom_buffer_area,
tp.trip2_stops_geom_buffer_area)) > 60

```

Este query es usado por el script `map_duplicate_routes.py` para generar un mapa interactivo que nos permite explorar los resultados. Este produjo apenas 62 tuplas, por ende al ser relativamente pocos resultados podemos manualmente explorarlos uno por

uno para entender cada caso y verificar si efectivamente son duplicados o no. En la siguiente subsección, analizamos algunos de los casos particulares que surgieron de esta query.

2.6.6 Exploración de resultados de rutas duplicadas

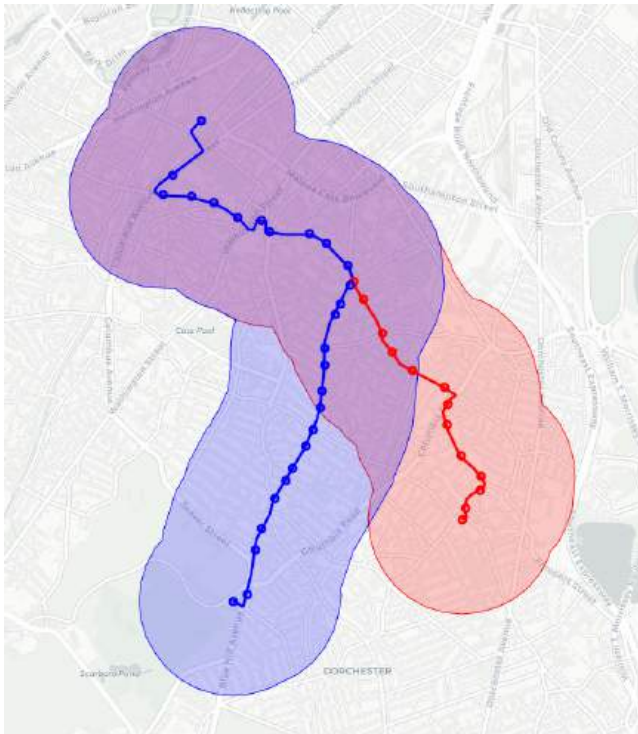


Figura 35: Rutas 15 y 45 (66.9%)

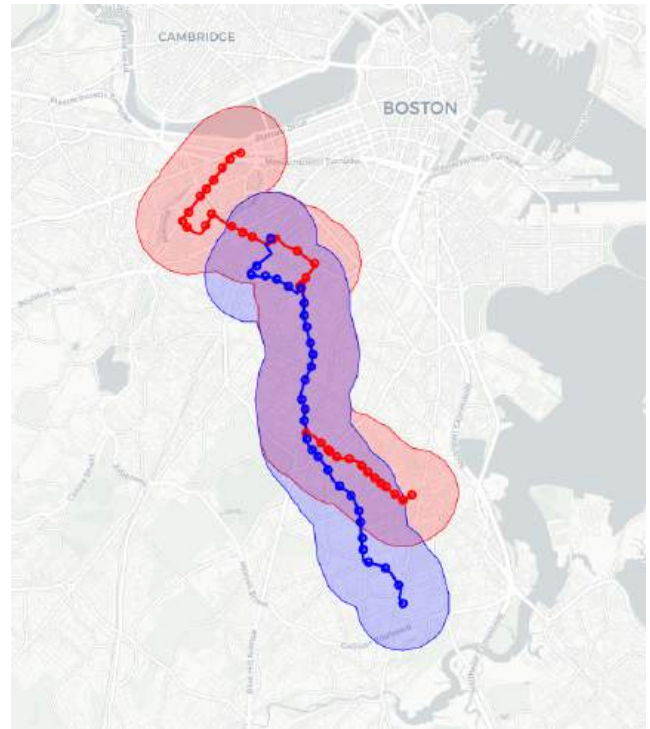


Figura 36: Rutas 19 y 23 (68.9%)

Notar que la primera ruta mencionada siempre es la roja y la segunda es la azul.

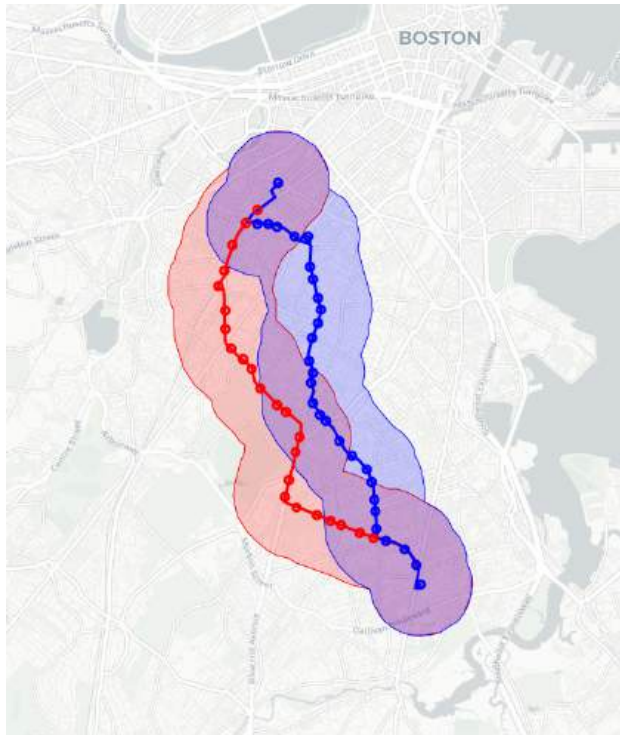


Figura 37: Rutas 22 y 23 (68.0%)

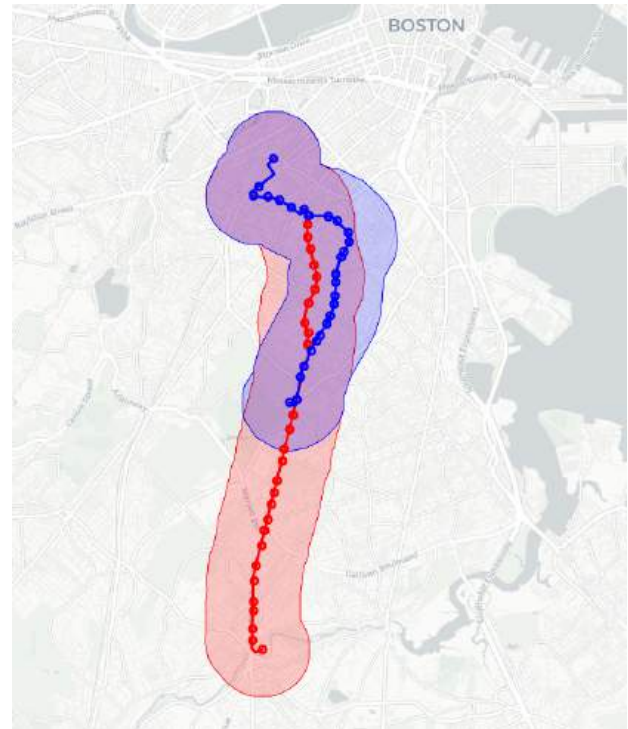


Figura 38: Rutas 28 y 45 (86.2%)

En las Figuras 35 a 38 tenemos distintos casos de rutas relevadas por la query como posibles duplicadas. La Figura 35 muestra las rutas 15 y 45 con un 66.9% de intersección, donde se ve como alrededor de la mitad norte de cada una de estas rutas es compartida, sin embargo una persona que utilice las partes sur de estas rutas no las consideraría como duplicadas. En la Figura 36 vemos las rutas 19 y 23 que comparten una significativa sección en el medio, pero igualmente son disjuntas en sus extremos.

La Figura 37 muestra un ejemplo distinto de dos rutas, la 22 y la 23, que comienzan y terminan en las mismas paradas, compartiendo 5 de las paradas en sus extremos, pero sin embargo difieren significativamente en el recorrido que toman en el medio. La Figura 38 es lo más cercano que llegamos hasta ahora al objetivo planteado en esta sección, que era el de encontrar rutas que se puedan eliminar como duplicadas, mostrando como el recorrido de la ruta 45 puede ser casi completamente reemplazado por el recorrido existente de la ruta 28.

Algo a tener en cuenta es que las rutas no precisan superponerse para poder considerarse duplicadas, sinó que sus buffers se deben intersectar. En la Figura 39 y 40 tenemos un ejemplo de dos rutas, posiblemente duplicadas, que no se interseccionan.

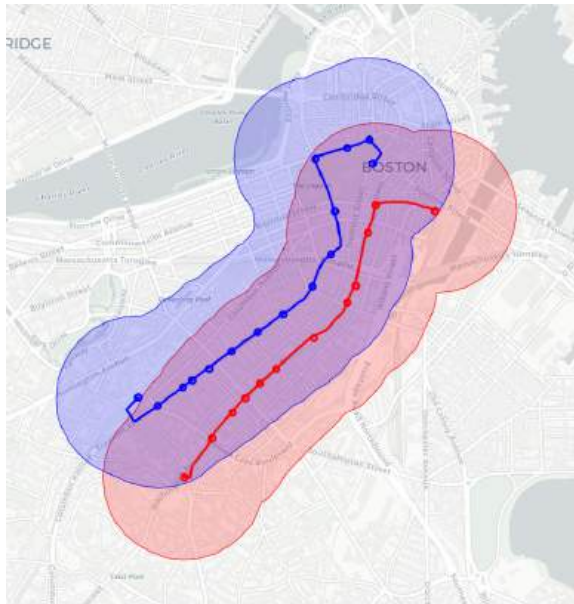


Figura 39: Rutas 751 y 43 (62.6%)

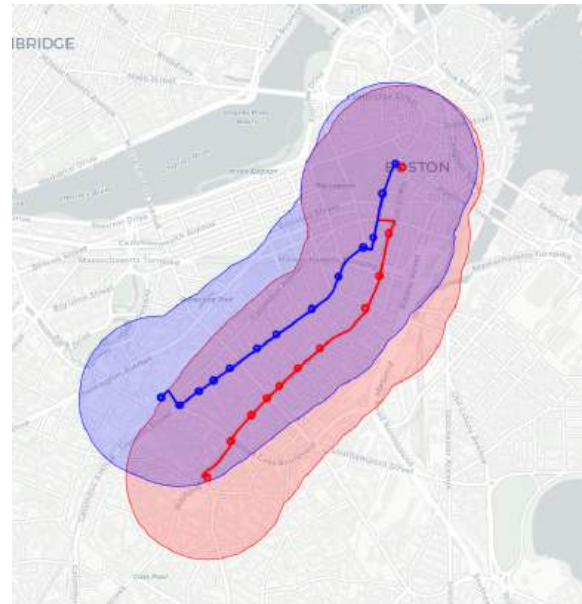


Figura 40: Rutas 749 y 43 (77.0%)

Los pares de rutas 751-43 y 749-43, mostrados en las Figuras 39 y 40 respectivamente, podrían considerarse como duplicadas dado que recorren caminos paralelos y en la misma dirección, pero remover una de ellas implicaría que los viajeros de dicha ruta tengan que posiblemente caminar unas cuerdas más para poder utilizar la otra.

Por el otro lado, la query también arrojó resultados poco significativos. En la Figura 41 vemos como las rutas 37 y *CR-Needham* fueron marcadas como duplicadas. Sin embargo, por más que se superponen significativamente, esta comparación es entre un bus y un tren de larga distancia, lo que significa que sirven propósitos distintos en la infraestructura de transporte público, por ende ninguna de estas rutas puede reemplazar a la otra. Se podría actualizar la query para solo permitir comparar rutas de distinto tipo, pero esto asumiría que el tipo de vehículo define el propósito de la ruta lo que puede no ser universalmente cierto (por ejemplo, los tipos “Bus” o “Trolleybus” podrían cumplir el mismo propósito) en un sistema de transporte público.

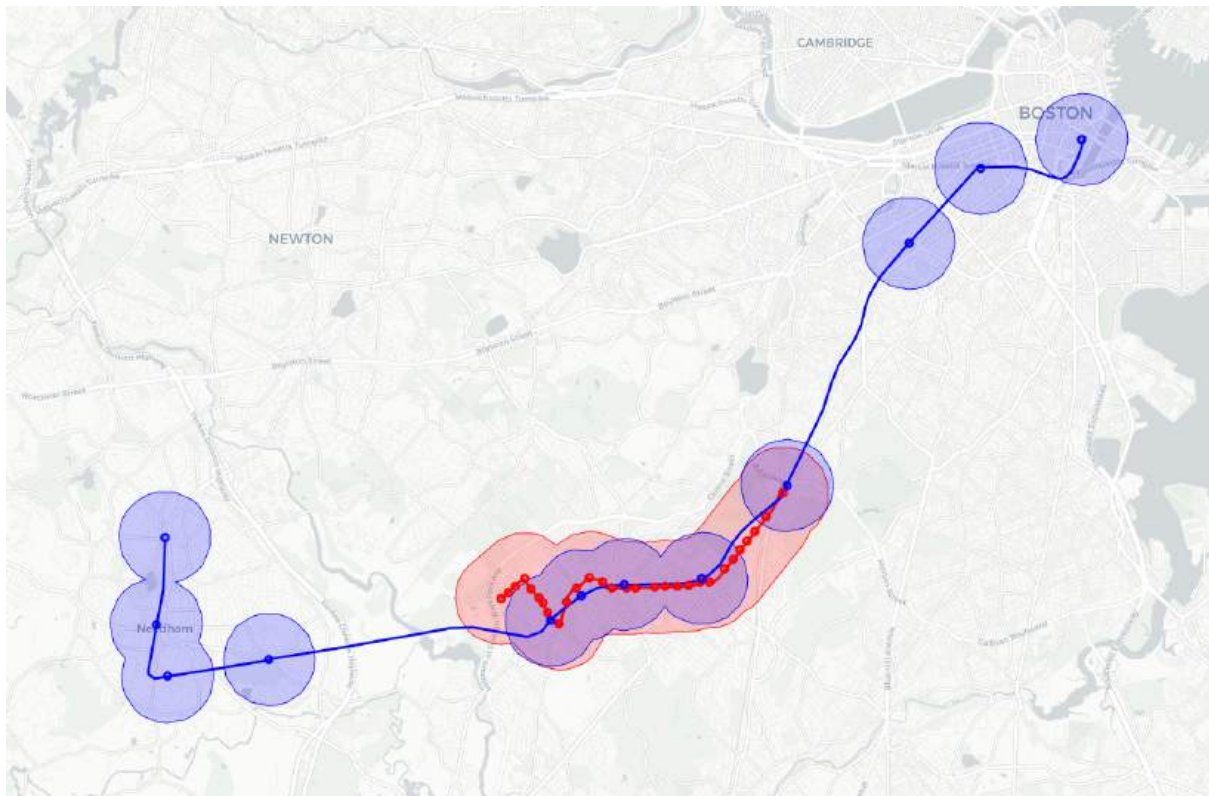


Figura 41: Rutas 37 y CR-Needham (65.0%)

2.6.7 Conclusiones sobre la duplicación de rutas

El análisis que realizamos cubre un amplio espectro de variables sobre el transporte:

- Límites de las áreas administrativas.
- Detección de hotspots con mayor densidad de tráfico.
- Áreas servidas por cada ruta mediante paradas.
- Superposición de áreas servidas por distintas rutas.

Sin embargo, esto no es suficiente para poder indicar con seguridad que dos rutas son duplicadas y que una puede ser eliminada. Esto se debe a que también debemos considerar información del contexto en el que surge este sistema de transporte, como por ejemplo:

- Datos demográficos, como densidad de población en cada área.
- Costumbres de las personas, los suburbios por ejemplo tienden a usar más autos privados.
- Tráfico en cada zona; hay rutas que pueden parecer duplicadas pero en realidad esa duplicación es necesaria para no sobrecargar ninguna una ruta particular.
- Patrones de transporte, por ejemplo entre qué zonas hay grandes grupos que se mueven.

Eso dicho, con el método de superposición de buffer de paradas se podría mejorar incluyendo mayor cantidad de filtros, por ejemplo para evitar incluir pares de rutas con distintos tipos de vehículos, o se podría probar utilizar distintos tamaños de buffer o porcentajes de intersección. De todos modos, para los fines académicos, este método mostró ser muy capaz de encontrar rutas con funciones similares, por más que estas no se intersecten directamente.

2.7 Análisis de cobertura del transporte público en colegios de la ciudad

En esta sección se analiza la accesibilidad al transporte público de los establecimientos educativos (desde jardín de infantes hasta secundaria) ubicados en la ciudad de Boston, utilizando la información del sistema de transporte planificado. Como primer paso, por fines académicos, restringimos el análisis únicamente a las escuelas situadas dentro de los límites de la ciudad de Boston, aunque se consideran todas las paradas disponibles en el conjunto de datos.

Luego, se observa el nivel de accesibilidad según el tipo de ruta, y luego se presentan dos rankings: el top 5 de colegios con menor accesibilidad y el top 5 con mayor accesibilidad. En cada caso, se explicita el criterio utilizado para definir qué se considera un colegio accesible o no accesible.

Es importante destacar que al analizar “qué tan cerca” o “qué tan lejos” se encuentra una parada de un colegio, utilizamos la distancia euclidiana (en línea recta). Si bien esta métrica es útil como aproximación, presenta algunas limitaciones relevantes:

- No considera la calidad del servicio (frecuencia, número de rutas, conexiones disponibles y áreas de cobertura de dichos servicios).
- No representa la accesibilidad real, ya que no contempla barreras físicas ni el camino efectivo más corto o rápido para llegar a la parada.

No obstante, como herramienta de caracterización espacial preliminar con fines académicos, esta métrica resulta válida para identificar establecimientos potencialmente más aislados en términos de cobertura geográfica directa del sistema de transporte.

Para llevar adelante este análisis, se utilizan los *scripts* y archivos GeoJSON ubicados en `scheduled/schools`.

2.7.1 Carga de los datos para realizar el análisis

El primer paso consiste en descargar el conjunto de datos correspondiente a los colegios (jardín de infantes y secundaria) del estado de Massachusetts, disponible en este [link](#). Este dataset se encuentra originalmente en formato Shapefile, por lo que es necesario convertirlo a formato GeoJSON para facilitar su procesamiento posterior. Esta conversión puede realizarse mediante el siguiente código en Python:

```
import geopandas as gpd

schools_gdf = gpd.read_file("schools/SCHOOLS_PT.shp")
schools_gdf = schools_gdf.to_crs(epsg=26986)
schools_gdf.to_file("schools.geojson", driver="GeoJSON")
```

El código transforma las geometrías al sistema de referencia EPSG:26986 y genera un archivo `schools.geojson` (disponible en la carpeta del repositorio) con los puntos correspondientes a cada colegio. Una vez generado el GeoJSON, se puede cargar la información en la base de datos PostgreSQL utilizando el este otro código:

```
import geopandas as gpd
from sqlalchemy import create_engine

if __name__ == "__main__":
    engine = create_engine("postgresql://postgres:postgres@localhost:5432/mbtagtfs")
    gdf = gpd.read_file("schools.geojson")
    gdf.columns = [col.lower() for col in gdf.columns]
    gdf.to_postgis("schools", engine, if_exists="replace", index=False)
```

Es importante señalar que este conjunto de datos contiene información de colegios correspondientes a todo el estado (aproximadamente 2500 colegios). Dado el gran volumen de datos y que además el presente análisis se enfoca exclusivamente en la ciudad de Boston, es necesario filtrar geográficamente los registros para considerar únicamente aquellos dentro del límite de la ciudad. Para ello, se utiliza para este análisis otro [GeoJSON con el polígono que representan los límites de la ciudad](#) (disponible en la carpeta del repositorio bajo el nombre `boston.geojson`). Este archivo también debe cargarse en la base

de datos, previo filtrado para conservar únicamente el polígono correspondiente al área principal (excluyendo islas u otras zonas periféricas). El siguiente código realiza dicho filtrado y carga el resultado en la tabla `boston_boundary`:

```
import geopandas as gpd
from sqlalchemy import create_engine

if __name__ == "__main__":
    engine =
    create_engine("postgresql://postgres:postgres@localhost:5432/mbtagtfs")

    gdf = gpd.read_file("boston.geojson")
    gdf.columns = [col.lower() for col in gdf.columns]
    gdf = gdf[gdf["objectid"] == 18].copy()
    gdf.rename(columns={"objectid": "id"}, inplace=True)
    gdf = gdf.to_crs(epsg=26986)
    gdf.to_postgis("boston_boundary", engine, if_exists="replace",
index=False)
```

El identificador `objectid = 18` corresponde al polígono principal que representa el área urbana de la ciudad de Boston. Con todo esto, podemos crear una vista `schools_in_boston` (168 tuplas) con los colegios dentro del polígono y utilizar el *script* `visualize_schools_in_boston.py` para visualizar el resultado en un mapa (Figura 42).

```
CREATE OR REPLACE VIEW schools_in_boston AS
SELECT s.*
FROM schools s
JOIN boston_boundary b ON b.id = 18 AND ST_Intersects(s.geometry,
b.geometry);
```

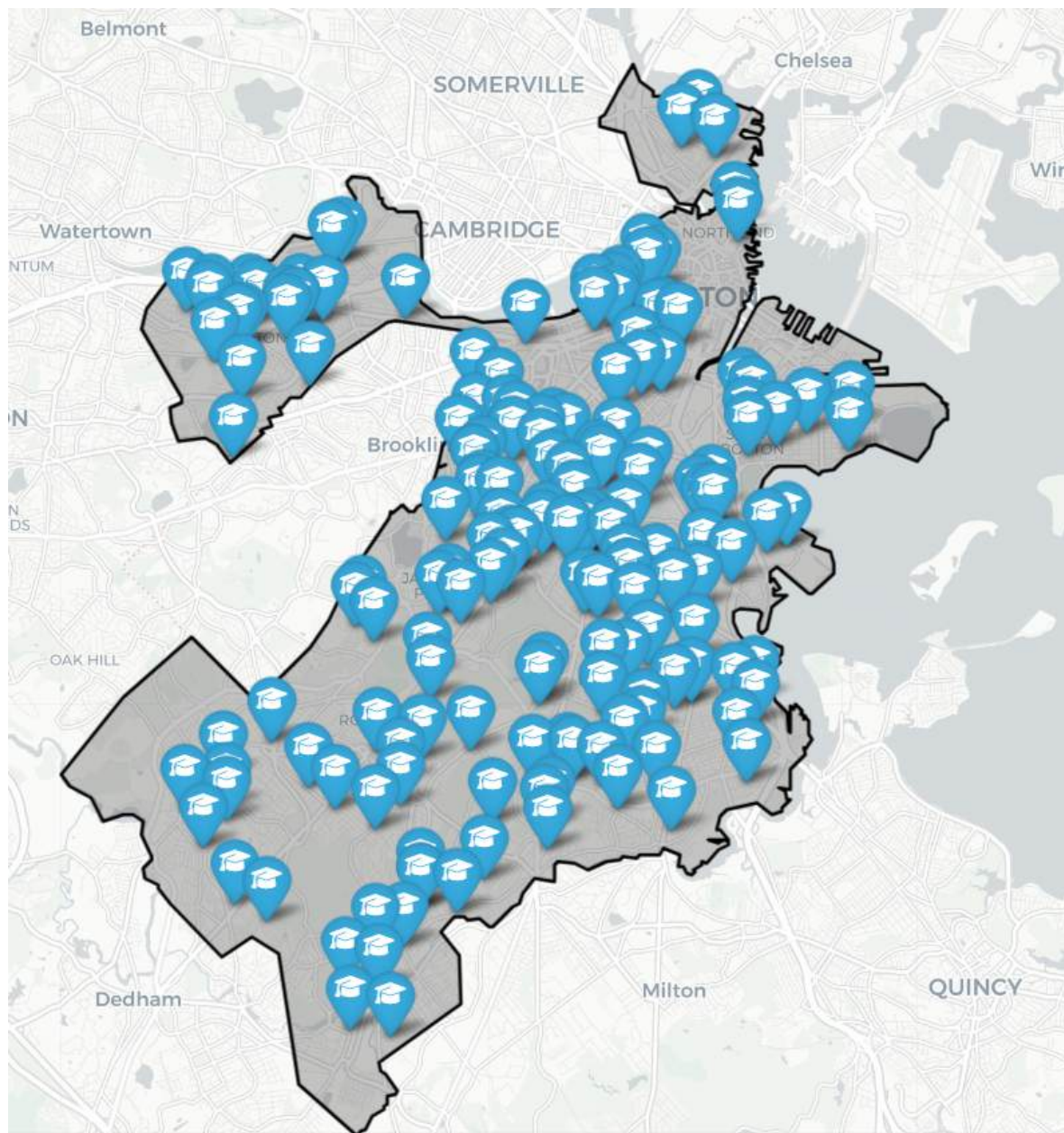


Figura 42: Colegios en la ciudad de Boston

2.7.2 Cobertura del transporte público por tipo de ruta

Podemos hacer una primera consulta para ver la distancia mínima entre cada colegio de la ciudad de Boston y la parada de transporte público más cercana con la siguiente consulta.

```
SELECT MAX(min_distance) AS required_distance_m
FROM (
  SELECT
    s.schid,
    MIN(ST_Distance(s.geometry, ST_Transform(t.stop_loc::geometry,
26986))) AS min_distance
  FROM schools_in_boston s
  JOIN stops t ON TRUE
  GROUP BY s.schid
) AS sub;
```

Se utiliza la vista `schools_in_boston` y se computa, para cada escuela, la menor distancia euclidiana a cualquier parada del sistema, transformando las geometrías a un sistema métrico (EPSG:26986) para obtener valores en metros. Finalmente, se extrae el valor máximo de todas esas distancias mínimas, lo que permite identificar cuál es el colegio más aislado respecto al transporte público. El resultado obtenido fue de aproximadamente 706 metros, lo que implica que todos los colegios de Boston tienen al menos una parada de transporte público dentro de ese radio, lo cual, en principio, evidencia una buena cobertura geográfica del sistema en relación con la localización de los establecimientos educativos. Ahora bien, sabemos por la sección 2.3.3 que tenemos diferentes tipos de ruta, por lo que este análisis macro no es muy representativo. La idea es hacer el mismo análisis, pero teniendo en cuenta este detalle.

```
WITH stops_with_route_types AS (
  SELECT DISTINCT
    s.stop_id,
    s.stop_loc,
    r.route_type
  FROM stops s
```

```

JOIN stop_times st ON s.stop_id = st.stop_id
JOIN trips t ON st.trip_id = t.trip_id
JOIN routes r ON t.route_id = r.route_id
)

SELECT MAX(min_distance) AS required_distance_m
FROM (
  SELECT
    s.schid,
    MIN(ST_Distance(s.geometry, ST_Transform(swrt.stop_loc::geometry,
26986))) AS min_distance
  FROM schools_in_boston s, stops_with_route_types swrt
  WHERE swrt.route_type = X
  GROUP BY s.schid
) AS sub;

```

Esta consulta permite identificar el colegio más alejado del sistema de transporte público para un tipo específico (bus, metro, tren o ferry), utilizando como métrica la mayor distancia mínima entre cada escuela y su parada más cercana de ese tipo.

Primero, se construye una subconsulta denominada `stops_with_route_types`, que asocia cada parada (*stop_id*) con su tipo de ruta (*route_type*) a partir de la unión de las tablas `stops`, `stop_times`, `trips` y `routes`.

Luego, para cada colegio de la vista `schools_in_boston`, se calcula la distancia mínima a las paradas correspondientes al tipo de servicio seleccionado (*route_type* = X). Para ello, se transforman las geometrías al sistema de referencia métrico EPSG:26986, lo que permite expresar las distancias en metros.

Finalmente, se toma el valor máximo entre todas esas distancias mínimas. Este valor representa el colegio con menor accesibilidad para el tipo de transporte analizado, y permite evaluar de forma específica la cobertura geográfica que dicho servicio ofrece a los establecimientos educativos de la ciudad. A continuación, se presentan los resultados obtenidos para cada tipo de ruta.

Tipo de ruta (identificador)	Resultado (metros)
Tram / Light rail (0)	~ 5811 m
Subway / Metro (1)	~ 6.463 m
Rail long-distance (2)	~ 3201 m
Bus (3)	~ 706 m
Ferry (4)	~ 11.383 m

Los resultados obtenidos permiten evaluar la cobertura del transporte público según el tipo de ruta en relación con los colegios de la ciudad de Boston. El sistema de bus presenta la mejor cobertura, con una mayor distancia mínima de solo 706 metros (que coincide con lo que vimos en la primera consulta) hasta la parada más cercana, lo que refleja su amplia distribución territorial.

En contraste, otros modos como el ferry (~11.383 m), el subte/metro (~6.463 m), el tren regional (~3.201 m) y el tranvía/light rail (~5.811 m) muestran una cobertura más localizada y/o limitada. Estos resultados destacan la fuerte dependencia del sistema educativo de Boston respecto del transporte en bus, lo cual resulta coherente con su alcance barrial como vimos en la sección 2.3.3.

A continuación, se presentan distintas visualizaciones (Figuras 43 a 47) en las que se muestran los colegios junto con las paradas utilizadas por cada tipo de servicio, con el objetivo de ofrecer una referencia espacial clara del análisis que hemos hecho hasta aquí.

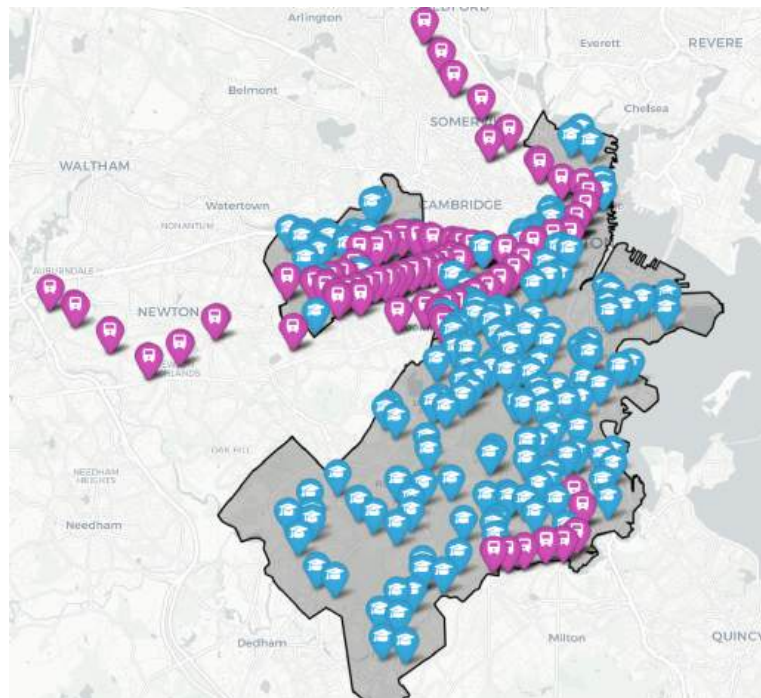


Figura 43: Visualización de colegios y paradas de tipo Tram / Light rail

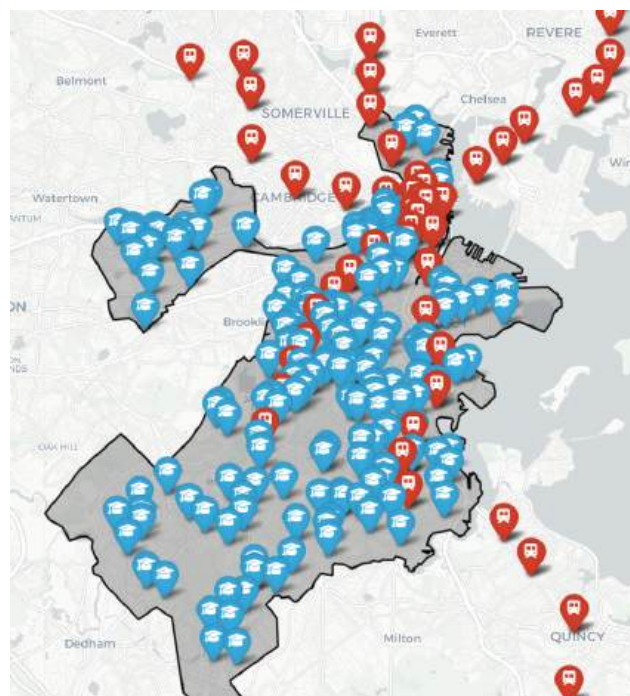


Figura 44: Visualización de colegios y paradas de tipo Subway / Metro

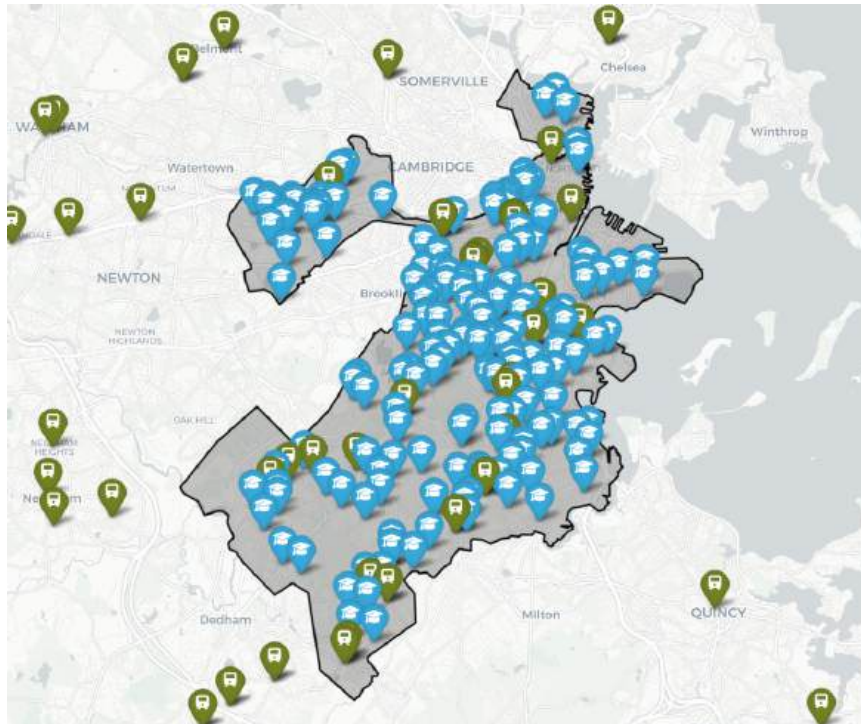


Figura 45: Visualización de colegios y paradas de tipo Rail Long Distance

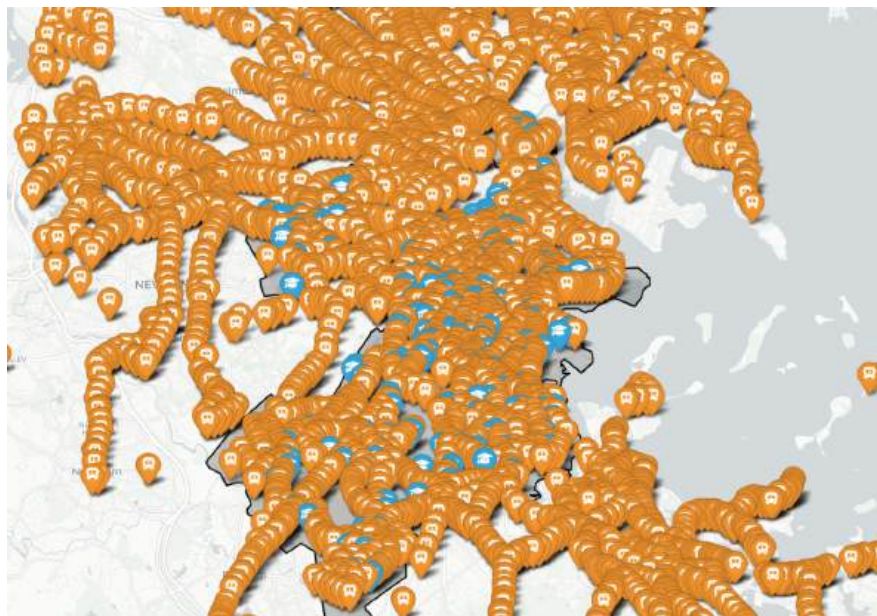


Figura 46: Visualización de colegios y stops de tipo Bus

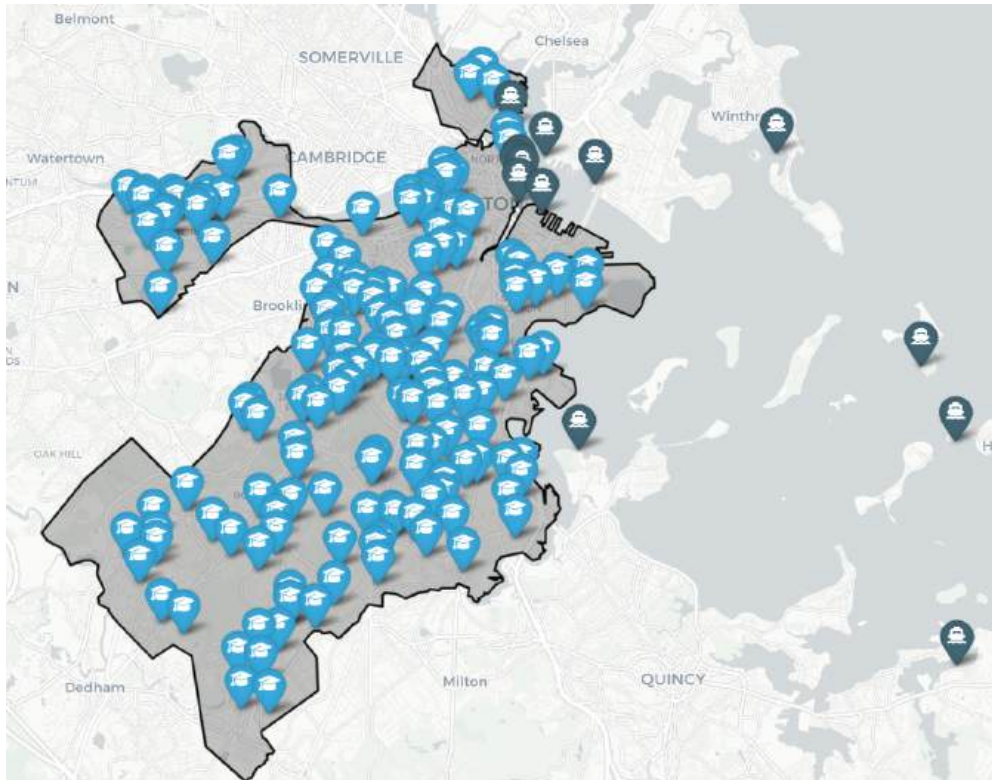


Figura 47: Visualización de colegios y stops de tipo Ferry

2.7.3 Top 5 colegios con menor accesibilidad

Vamos a definir un colegio con baja accesibilidad como aquel cuya parada de transporte público más cercana se encuentra a una distancia significativamente mayor respecto del resto, considerando la distancia euclidiana como métrica base. Con esto en mente, llevamos a cabo la siguiente consulta.

```
WITH closest_stops AS (
  SELECT DISTINCT ON (s.schid)
    s.schid,
    s.name,
    s.geometry AS school_geom,
    t.stop_id,
    t.stop_loc,
    ST_Distance(s.geometry, ST_Transform(t.stop_loc::geometry, 26986)) AS
dist_m
  FROM schools_in_boston s, stops t
  ORDER BY s.schid, ST_Distance(s.geometry,
ST_Transform(t.stop_loc::geometry, 26986))),
```



```
top5 AS (SELECT * FROM closest_stops ORDER BY dist_m DESC LIMIT 5)
SELECT * FROM top5;
```

schid	name	dist_m	stop_id	school_geom	stop_loc
00350891	Yeshiva Ohr Yisrael High School for Boys	706.6965756254781	1092	POINT (-71.15643063...	POINT (-71.151428 42.3...
00350184	Manning Elementary School	567.4784995894114	5243	POINT (-71.13149065...	POINT (-71.124871 42.3...
00350016	Mattahunt Elementary School	500.5030277761138	66460	POINT (-71.10303348...	POINT (-71.108216 42.2...
00350950	British International School of Boston	481.94019225747826	15241	POINT (-71.13009145...	POINT (-71.124253 42.3...
00350237	Mozart Elementary School	458.547683814156	606	POINT (-71.14105065...	POINT (-71.136949 42.2...

Figura 48: Resultado de la consulta anterior

Esta consulta identifica los 5 colegios de la ciudad de Boston cuya parada de transporte público más cercana está a mayor distancia en línea recta. Primero, en `closest_stops`, se calcula para cada colegio la parada más cercana utilizando `ST_Distance`, ordenando por distancia y seleccionando solo la más próxima mediante `DISTINCT ON (s.schid)`. Podemos graficar los resultados obtenidos en la Figura 48 con el *script* `visualize_isolated_schools.py`.

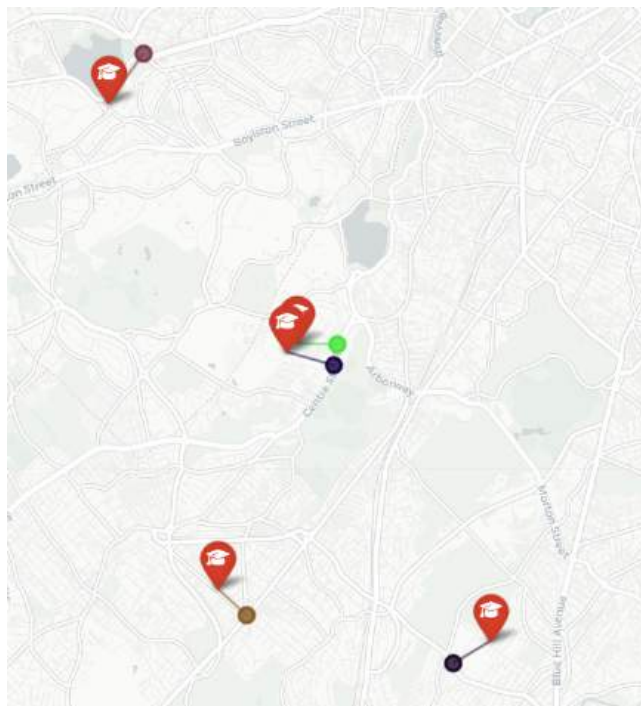


Figura 49: Colegios (aislados) según la distancia a la parada más cercana

A pesar de que la métrica utilizada es la distancia euclidiana (en línea recta), en este caso los resultados de la Figura 49 resultan bastante representativos de la realidad. Como puede observarse en las Figuras 50 y 51, las líneas que conectan cada colegio con su parada más cercana siguen una dirección cuasi-compatible con el trazado de las calles, lo que sugiere que el camino real que recorrerán los estudiantes no difiere sustancialmente de la distancia estimada. Esto refuerza la validez del análisis como aproximación espacial preliminar para este caso aunque como dijimos en la introducción de la sección, no es algo exhaustivo ni 100% representativo por la falta de comparación con otros factores.

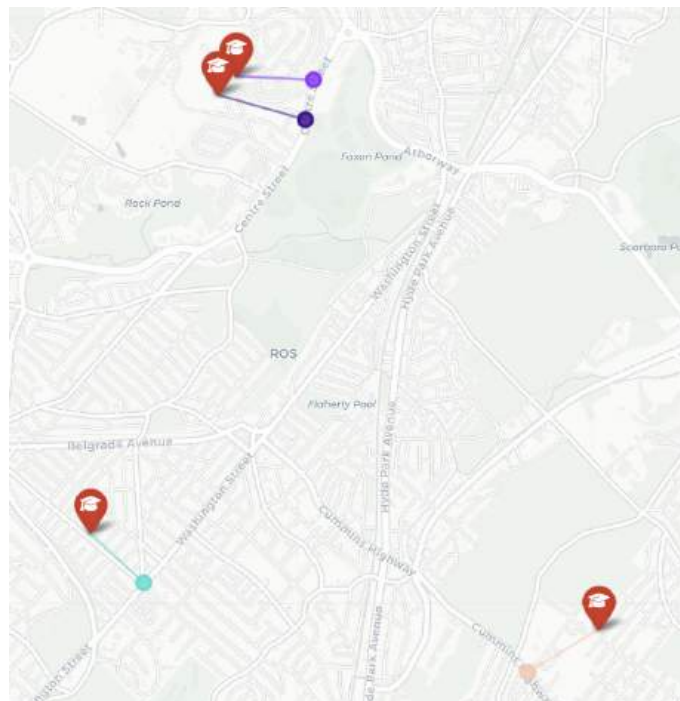


Figura 50: Figura 49 ampliada para visualizar mejor los resultados

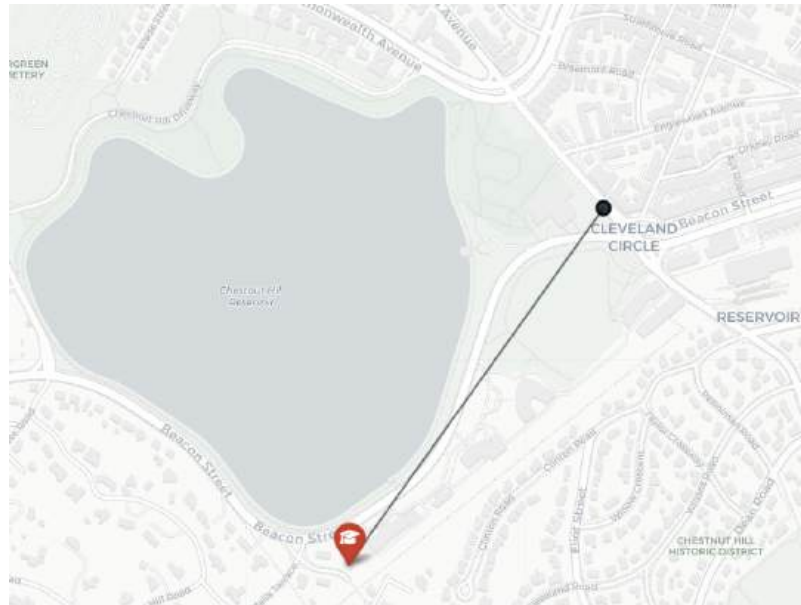


Figura 51: Figura 49 ampliada para visualizar mejor los resultados

2.7.4 Top 5 colegios con mayor accesibilidad

De manera análoga al análisis anterior, podemos identificar los colegios con mayor accesibilidad, es decir, aquellos cuya parada de transporte público más cercana se encuentra a la menor distancia en línea recta. A continuación, se presenta la consulta correspondiente junto con el *script* `visualize_accessible_schools.py`, que permite visualizar los resultados de la Figura 52 en el mapa.

```
WITH closest_stops AS (
  SELECT DISTINCT ON (s.schid)
    s.schid,
    s.name,
    s.geometry AS school_geom,
    t.stop_id,
    t.stop_loc,
    ST_Distance(s.geometry, ST_Transform(t.stop_loc::geometry, 26986)) AS
dist_m
  FROM schools_in_boston s, stops t
  ORDER BY s.schid, ST_Distance(s.geometry,
ST_Transform(t.stop_loc::geometry, 26986))
),
```

```
top5 AS (  
  SELECT * FROM closest_stops  
  ORDER BY dist_m ASC  
  LIMIT 5  
)  
SELECT * FROM top5;
```

schid	name	dist_m	stop_id	school_geom	stop_loc
00350729	Epiphany School	1.1306096455283468	node-smnl-farepaid	POINT (-71.065786617...	POINT (-71.065797 ...
00350725	Compass School	19.413890818881693	11512	POINT (-71.065960593...	POINT (-71.065751 ...
00350377	Higginson/Lewis K-8 School	27.05777650958916	1342	POINT (-71.086399584...	POINT (-71.085991 ...
00350014	Shaw Elementary School	27.981507518419587	443	POINT (-71.086840347...	POINT (-71.086842 ...
00350268	Dever Elementary School	31.94988994360597	140	POINT (-71.042740610...	POINT (-71.042799 ...

Figura 52: Resultado de la consulta anterior

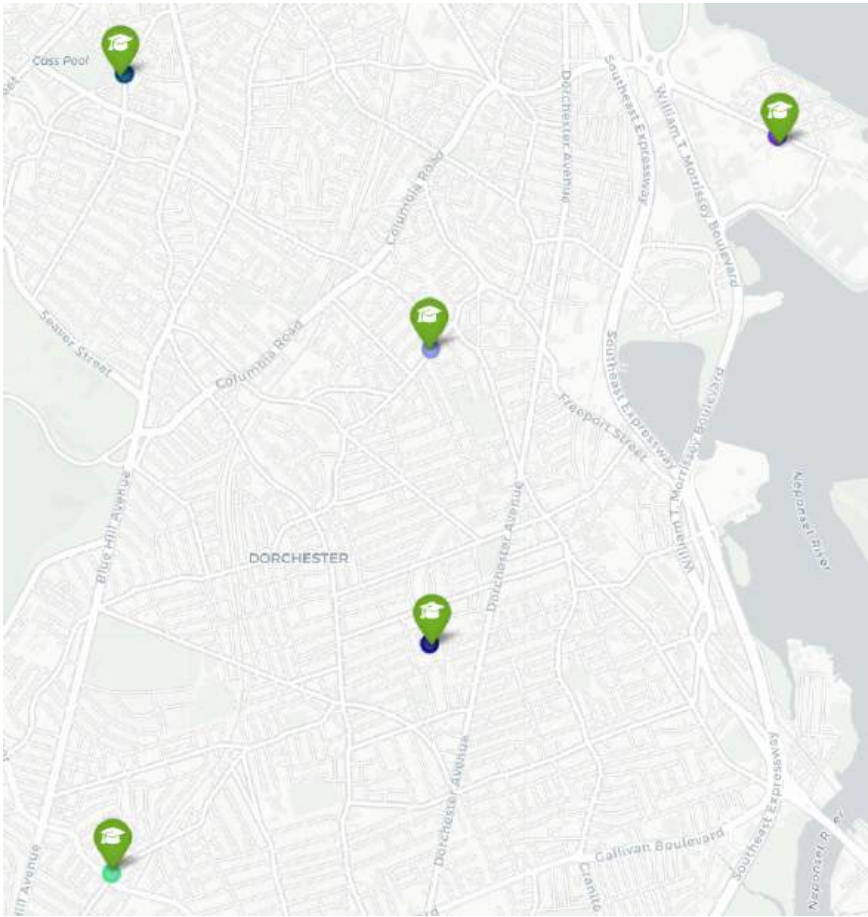


Figura 53 Colegios (accesibles) según la distancia a la parada más cercana.

Algo interesante que se desprende de la Figura 53 es que todos los colegios considerados accesibles tienen una parada a menos de 40 metros de distancia, lo cual representa una proximidad casi inmediata al transporte público. Esta cercanía favorece no solo el acceso de los estudiantes, sino también la conexión rápida con otras zonas de la ciudad. Además, se observa que estos establecimientos están distribuidos en diferentes sectores del barrio de Dorchester (céntrico), lo que sugiere que la buena cobertura no se limita a una zona puntual, sino que forma parte de una infraestructura más extendida en ese distrito.

3. GTFS Real Time

Hasta este punto, hemos procesado los datos estáticos del sistema y obtenido resultados que nos permitieron caracterizar el comportamiento esperado de los vehículos a partir de sus trayectorias planificadas. En esta sección, nos enfocaremos en el procesamiento y análisis de los datos **GTFS Real Time**, con el objetivo de recuperar, en una primera instancia, la posición de los vehículos en tiempo real y, a partir de ella, reconstruir sus trayectorias efectivamente recorridas.

Antes de avanzar, es conveniente introducir brevemente este estándar.

Los datos **GTFS Real Time** se distribuyen a través de *feeds* o flujos de datos que actualizan continuamente la información operativa del sistema de transporte público. Estos *feeds* están codificados en **Protocol Buffers (protobuf)**, un formato binario eficiente desarrollado por Google para facilitar el intercambio de datos estructurados. Un feed GTFS Real Time puede incluir tres tipos principales de actualizaciones:

- **Posiciones de vehículos ([Vehicle positions](#))**: Indican la posición en tiempo real de los vehículos.
- **Actualizaciones de viajes ([Trip updates](#))**: Informan sobre cambios en los recorridos planificados, como demoras, cancelaciones o desvíos.
- **Alertas de servicio ([Service alert](#))**: Comunican interrupciones, accidentes o modificaciones en las rutas.

Dado que nuestro objetivo es reconstruir las trayectorias reales de los vehículos, centraremos nuestro análisis en el consumo del *feed* correspondiente a las posiciones de los vehículos. Siguiendo la [documentación oficial del sistema de transporte público de Boston \(MBTA\)](#), es posible acceder al *endpoint* desde el cual se distribuye dicho *feed* en formato estándar Protobuf (versión 2.0), especificado por Google, sin ninguna API Key.

A continuación, se presenta un paso a paso del proceso seguido para consumir este *feed* y comprender su dinámica. Cabe destacar que no realizamos solicitudes constantes al *endpoint*, ya que esto podría conducir a la obtención de datos no actualizados. En su lugar, desarrollamos una estrategia que nos permitió asegurar que cada request realizada recuperará al menos una nueva muestra de datos garantizando la frescura de la información consumida.

3.1 Procesamiento del *feed* de las posiciones de los vehículos

Como se mencionó al final de la sección anterior, el objetivo de esta subsección es describir paso a paso cómo desarrollamos un cliente robusto para el sistema MBTA, capaz de recolectar grandes volúmenes de datos sobre las posiciones de los vehículos en tiempo real. Para eso, hemos seguido las recomendaciones del estandar en utilizar las librerías de Python `gtfs_realtime_pb2` de [google.transit](https://github.com/google/transit) y `requests` para el manejo de solicitudes HTTP.

Con estas dos librerías es posible construir un *script* que permite consumir el *feed* y parsear el contenido codificado en formato Protocol Buffers, dejándolo listo para su posterior procesamiento. El siguiente ejemplo representa el primer código funcional que desarrollamos, y nos fue de gran utilidad para comprender cómo estructurar correctamente la solicitud, interpretar la respuesta y extraer las posiciones de los vehículos a partir del *feed* de Vehicle Positions.

```
def fetch_feed(url: str) -> FeedMessage:
    feed = FeedMessage()
    try:
        response = requests.get(url)
        response.raise_for_status()
        feed.ParseFromString(response.content)
    except requests.exceptions.RequestException as e:
        raise ReferenceError(f"Error fetching GTFS-RT data: {e}")

    if not feed or not feed.entity:
        raise ReferenceError(f"No data received from GTFS-Real Time feed.
URL: {url}.")
    return feed

if __name__ == "__main__":
    feed = fetch_feed(
        url="https://cdn.mbta.com/realtime/VehiclePositions.pb"
    )
```


Siguiendo la [documentación](#) de buenas prácticas para la publicación de feeds GTFS Realtime, observamos un aspecto clave que puede ayudarnos a optimizar la recolección de datos: el uso de encabezados HTTP para detectar si el contenido del *feed* cambia (*i.e.* tiene nuevos datos). Según la documentación, se recomienda que los servidores que publican datos GTFS Realtime incluyan el encabezado `Last-Modified` en sus respuestas, y que los consumidores utilicen el encabezado `If-Modified-Since` en sus solicitudes. Este mecanismo permite al cliente consultar al servidor si el recurso ha cambiado desde la última vez que fue descargado. Si no ha cambiado, el servidor responde con un código 304 `Not Modified`, evitando así la transferencia innecesaria de datos.

Realizamos algunas pruebas y verificamos que el servidor del MBTA implementa correctamente el mecanismo de control de cambios mediante encabezados HTTP. Enviamos múltiples solicitudes en intervalos cortos (entre una décima y medio segundo) utilizando el encabezado `If-Modified-Since`, y observamos que el servidor responde con 304 `Not Modified` cuando no hay actualizaciones, y con 200 `OK` acompañado de un nuevo `Last-Modified` cuando el contenido del *feed* cambia. Esto fue incorporado en `fetch_feed()`, permitiéndonos recolectar datos evitando descargas y procesamiento innecesario.

```
def fetch_feed(url: str) -> FeedMessage | None:
    headers = {}

    # last_modified is a global variable
    if last_modified:
        headers["If-Modified-Since"] = format_datetime(last_modified )

    try:
        response = requests.get(url=url, headers=headers)
        response.raise_for_status()
    except requests.RequestException as e:
        raise ReferenceError(f"Error fetching GTFS-RT feed: {e}")

    if response.status_code == 304:
        print(
```

```
        f"Feed not modified since last fetch {last_modified }.\n        Skipping fetch."\n    )\n    return None\n\n    print("Feed updated. Parsing protobuf data.")\n    feed = FeedMessage()\n    feed.ParseFromString(response.content)\n\n    if not feed or not feed.entity:\n        raise ReferenceError("No entities found in GTFS-RT feed")\n\n    if "Last-Modified" in response.headers:\n        last_modified = parsedate_to_datetime(\n            response.headers["Last-Modified"]\n        )\n\n    return feed
```

Aunque la función `fetch_feed()` permite obtener un snapshot del *feed*, por sí sola no resuelve el problema completo de recolección de datos. En la práctica, lo que se necesita no es una sola descarga puntual, sino un proceso continuo que consulte el *feed* en intervalos regulares y acumule esa información para su posterior análisis.

Si mantuviéramos toda esta lógica dispersa en un solo *script*, el código rápidamente se volvería difícil de leer y poco flexible ante futuros cambios. Además, separar responsabilidades es clave: la recolección de datos no debería convivir con la lógica de visualización. Por eso, se optó por encapsular esta lógica dentro de una clase `MBTAClient`, con que expone un método llamado `collect_by_duration()` que se encarga de manejar toda la interacción con el *feed*: desde su descarga utilizando `fetch_feed()`, pasando por la lógica de recolección periódica y devolviéndolos los datos recolectados en un `DataFrame` de la librería *pandas*. Este cliente puede encontrarse en la carpeta `realtime/clients/mbta/mbta_client.py` y adjuntamos a continuación su código.

```

import time
import requests
import pandas as pd
from realtime.clients.mbtta.models.vehicle import Vehicle
from google.transit.gtfs_realtime_pb2 import FeedMessage
from email.utils import format_datetime, parsedate_to_datetime

class MBTAClient:

    def __init__(self, vehicle_protobuf_url: str):
        self.url = vehicle_protobuf_url
        self.last_modified = None

    def collect_by_duration(
        self, duration_minutes: int, interval_seconds: int
    ) -> pd.DataFrame:
        collected_data = []
        start_time = time.time()
        end_time = time.time() + duration_minutes * 60
        iteration = 0

        while time.time() < end_time:
            feed = self._fetch_feed()

            if feed:
                vehicles = Vehicle.from_feed(feed)
                collected_data.extend([v.to_dict() for v in vehicles])

            now = time.time()
            elapsed = now - start_time
            remaining = max(0, end_time - now)
            iteration += 1
            print(
                f"[Iteration {iteration}] Collected {len(collected_data)} "
                f"vehicle positions "
                f"| Elapsed: {elapsed:.1f}s | Remaining: {remaining:.1f}s"
            )

            time.sleep(interval_seconds)

        return pd.DataFrame(collected_data)

    def _fetch_feed(self) -> FeedMessage | None:
        headers = {}
        if self.last_modified:
            headers["If-Modified-Since"] =
format_datetime(self.last_modified)

```

```
try:
    response = requests.get(url=self.url, headers=headers)
    response.raise_for_status()
except requests.RequestException as e:
    raise ReferenceError(f"Error fetching GTFS-RT feed: {e}")

if response.status_code == 304:
    print(
        f"Feed not modified since last fetch {self.last_modified}.
Skipping fetch."
    )
    return None

print("Feed updated. Parsing protobuf data.")
feed = FeedMessage()
feed.ParseFromString(response.content)

if not feed or not feed.entity:
    raise ReferenceError("No entities found in GTFS-RT feed")

if "Last-Modified" in response.headers:
    self.last_modified = parsedate_to_datetime(
        response.headers["Last-Modified"]
    )
return feed
```

3.2 Procesar datos de vehículos a partir del feed

Para representar de forma estructurada la información de cada vehículo contenida en el feed GTFS Realtime de tipo *Vehicle Positions*, se definió un [dataclass](#) llamado `Vehicle` en `realtime/clients/mbta/models/vehicle.py`. Este encapsula los [atributos más relevantes](#) del vehículo, como su posición geográfica, `vehicle_id`, `trip_id`, etc.

El método de clase `from_feed()` de `Vehicle` permite construir una lista de objetos `Vehicle` a partir de un `FeedMessage`, mapeando cada entidad del feed a una instancia del modelo. Además, se implementó un método `to_dict()` que convierte cada objeto en un diccionario de Python.

```
@dataclass
class Vehicle:
    id: str
    trip_id: Optional[str]
    schedule_relationship: Optional[int]
    latitude: float
    longitude: float
    bearing: Optional[float]
    speed_kmh: Optional[float]
    current_status: Optional[int]
    timestamp: Optional[int]
    stop_id: Optional[str]
    vehicle_id: Optional[str]
    vehicle_label: Optional[str]
    vehicle_license_plate: Optional[str]
    congestion_level: Optional[int] = None
    occupancy_status: Optional[int] = None

    @classmethod
    def from_feed(cls, feed: FeedMessage) -> List["Vehicle"]:
        vehicles = []
        for entity in feed.entity:
            if entity.HasField("vehicle"):
                vehicle = entity.vehicle
                vehicle = cls(
                    id=entity.id,
                    trip_id=vehicle.trip.trip_id,
                    schedule_relationship=vehicle.trip.schedule_relationship,
```

```

        latitude=vehicle.position.latitude,
        longitude=vehicle.position.longitude,
        bearing=vehicle.position.bearing,
        speed_kmh=vehicle.position.speed * 3.6,
        current_status=vehicle.current_status,
        timestamp=vehicle.timestamp,
        stop_id=vehicle.stop_id,
        vehicle_id=vehicle.vehicle.id,
        vehicle_label=vehicle.vehicle.label,
        vehicle_license_plate=vehicle.vehicle.license_plate,
    )
    vehicles.append(vehicle)
return vehicles

def to_dict(self) -> dict:
    return {k: v for k, v in asdict(self).items() if v is not None}

```

Mediante la clase `MBTAClient`, que internamente utiliza la clase auxiliar `Vehicle` para interpretar cada entidad del feed, es posible construir un primer *script* básico que consulta el endpoint durante X cantidad de minutos en intervalos de Y segundos, procesa las posiciones de los vehículos y devuelve los resultados en un *DataFrame* de la librería *pandas*. Como primer paso exploratorio, volcamos estos datos en un archivo JSON llamado `vehicles_by_duration.json`, lo que nos permitió inspeccionar de forma clara el contenido del feed y validar que la estructura y los campos extraídos fueran correctos. Esa versatilidad la provee la librería *pandas* a través del método `to_json()`. Si quisiéramos podríamos obtener un Parquet, CSV o SQL utilizando los métodos de la librería `to_parquet()`, `to_csv()`, `to_sql()` respectivamente.

```

from realtime.clients.mbt.mbt_client import MBTAClient

if __name__ == "__main__":
    client = MBTAClient(
        vehicle_protobuf_url="https://cdn.mbt.com/realtime/VehiclePositions.pb"
    )
    df = client.collect_by_duration(duration_minutes=10, interval_seconds=0.5)
    df.to_json("vehicles_by_duration.json", orient="records", indent=4)

```


Este es un ejemplo de una entrada del archivo `vehicles_by_duration.json` correspondiente a un vehículo:

```
{
  "id": "y1455",
  "trip_id": "70016549",
  "schedule_relationship": 0,
  "latitude": 42.48224639892578,
  "longitude": -70.94418334960938,
  "bearing": 35.0,
  "speed_kmh": 0.0,
  "current_status": 1,
  "timestamp": 1752261076,
  "stop_id": "4535",
  "vehicle_id": "y1455",
  "vehicle_label": "1455",
  "vehicle_license_plate": ""
}
```

3.3 Cargar las posiciones de los vehículos depuradas en PostgreSQL

En la sección anterior desarrollamos un cliente robusto que permite obtener las posiciones de los vehículos en diferentes formatos, y visualizamos un ejemplo de la información recolectada en formato JSON.

El siguiente paso consiste en almacenar estos datos en la base de datos PostgreSQL para su posterior análisis. Los scripts mencionados a continuación se encuentran en la carpeta `realtime/feed`.

Para ello, basta con extender el código anterior con unas pocas líneas adicionales consolidando así nuestro `script load_raw_positions_to_database.py` que automatiza tanto la recolección como la inserción de los datos en la base de datos, utilizando el método `to_sql()` de pandas. Además, este mismo `script` guarda una copia de los datos en formato CSV, lo que permite reutilizar la información sin necesidad de volver a consultar el `feed`. En ese caso, puede utilizarse el script `load_raw_positions_csv_to_database.py` para cargar directamente el archivo CSV en la base de datos.

A continuación, se muestra el contenido del `script load_raw_positions_to_database.py`, utilizado para generar un archivo CSV llamado `vehicle_positions.csv`, el cual se encuentra disponible en la carpeta compartida de Google Drive.

```

from realtime.clients.mbtta.mbtta_client import MBTAClient
from realtime.clients.postgres.postgres_client import PostgresClient

if __name__ == "__main__":

    # Initialize the client with the GTFS-RT protobuf URL
    client = MBTAClient(
        vehicle_protobuf_url="https://cdn.mbtta.com/realtime/VehiclePositions.pb"
    )
    df = client.collect_by_duration(duration_minutes=190, interval_seconds=15)

    # Save retrieved data to CSV
    df.to_csv("vehicle_positions.csv", index=False)

    postgres_client = PostgresClient(
        db_user="postgres",
        db_pass="postgres",
        db_host="localhost",
        db_port="5432",
        db_name="mbtagtfs",
    )

    # Load data into PostgreSQL database
    df.to_sql(
        "vehicle_positions", postgres_client.engine, if_exists="replace",
        index=False
    )

```

Como se puede observar, decidimos recolectar datos hasta alcanzar los 190 minutos en intervalos de 15 segundos obteniendo un [CSV con aproximadamente 349.000 posiciones de vehículos](#). Esto se realizó el día sábado 12 de julio entre las 12:00 y 15:00 horas de Argentina aproximadamente (15:00 y 18:00 horas hora local de Boston).

Luego, de haber cargado la tabla, agregamos un campo geométrico que representa la posición de cada vehículo como un punto geoespacial a partir de los valores de latitud y longitud. Esos puntos serán contruidos utilizando el sistema de coordenadas SRID 26986, correspondiente a la proyección oficial del estado de Massachusetts.

```
ALTER TABLE vehicle_positions ADD COLUMN geom geometry(Point, 26986);
UPDATE vehicle_positions
SET geom = ST_Transform(ST_SetSRID(ST_Point(longitude, latitude), 4326),
26986);
```

Al analizar los datos recolectados, observamos que para un mismo *trip_id*, *vehicle_id* y posición (*geom*), existen múltiples registros asociados al mismo *timestamp*. Esto indica que algunos puntos están repetidos tanto espacial como temporalmente, lo cual genera conflictos al momento de construir trayectorias temporales con MobilityDB. La siguiente consulta permite identificar esos casos:

```
SELECT trip_id, vehicle_id, geom,
COUNT(*) AS total_rows,
COUNT(DISTINCT timestamp) AS different_timestamps,
COUNT(*) - COUNT(DISTINCT timestamp) AS repeated_timestamps
FROM vehicle_positions
GROUP BY trip_id, vehicle_id, geom
HAVING COUNT(DISTINCT timestamp) > 1;
```

	trip_id	vehicle_id	geom	total_rows	different_timestamps	repeated_timestamps
1	68964186	dLF 958	POINT (-70.9199218...	15	2	13
2	68992095	y1334	POINT (-71.0554946...	7	2	5
3	68992095	y1334	POINT (-71.0556106...	8	2	6
4	68992097	y1341	POINT (-71.0415496...	2	2	0
5	68992097	y1341	POINT (-71.0425262...	3	2	1
6	68992097	y1341	POINT (-71.0463638...	4	2	2
7	68992097	y1341	POINT (-71.0560913...	23	2	21
8	68992097	y1341	POINT (-71.0554946...	5	2	3
9	68992097	y1341	POINT (-71.0556106...	4	2	2
10	68992098	y1337	POINT (-71.0560913...	40	3	37
11	68992098	y1337	POINT (-71.0558853...	2	2	0
12	68992098	y1337	POINT (-71.0556106...	3	2	1
13	68992100	y1341	POINT (-71.0203620...	6	2	4
14	68992103	y1336	POINT (-71.0560913...	51	3	48
15	68992104	y1341	POINT (-71.0463638...	2	2	0
16	68992104	y1341	POINT (-71.0466842...	2	2	0
17	68992104	y1341	POINT (-71.0554946...	16	2	14
18	68992104	y1341	POINT (-71.0556106...	3	2	1
19	68992105	y1334	POINT (-71.0421981...	3	2	1
20	68992105	y1334	POINT (-71.0463638...	3	3	0

Figura 54: Resultado de la consulta anterior

Esto motivó su limpieza mediante la creación de la tabla `cleaned_vehicle_positions`, en la que se descartan estos casos:

```

CREATE TABLE cleaned_vehicle_positions AS
SELECT DISTINCT ON (vp.trip_id, vp.vehicle_id, vp.timestamp)
  vp.id,
  vp.trip_id,
  r.route_type,
  vp.schedule_relationship,
  vp.latitude,
  vp.longitude,
  vp.bearing,
  vp.speed_kmh,
  vp.current_status,
  vp.timestamp,
  vp.stop_id,
  vp.vehicle_id,
  vp.vehicle_label,
  vp.vehicle_license_plate,
  vp.geom
FROM vehicle_positions vp
LEFT JOIN trips t ON vp.trip_id = t.trip_id
LEFT JOIN routes r ON t.route_id = r.route_id
WHERE vp.trip_id IS NOT NULL
  AND vp.timestamp IS NOT NULL
  AND vp.latitude IS NOT NULL
  AND vp.longitude IS NOT NULL
  AND vp.geom IS NOT NULL
  AND vp.stop_id IS NOT NULL
  AND vp.vehicle_id IS NOT NULL
ORDER BY vp.trip_id, vp.vehicle_id, vp.timestamp, vp.id;

SELECT COUNT(*) from cleaned_vehicle_positions;
--- 239210

```

Además, en la tabla `cleaned_vehicle_positions` buscamos matchear el *trip_id* con su correspondiente *route_type* (nos será útil en la sección de *map matching*). El objetivo ahora es realizar una simplificación para trabajar con un único vehículo por ruta. Para ello, primero identificamos los *trip_id* con más de un vehículo:

```

SELECT trip_id, COUNT(DISTINCT vehicle_id) AS vehicle_count
FROM cleaned_vehicle_positions
GROUP BY trip_id
HAVING COUNT(DISTINCT vehicle_id) > 1
ORDER BY vehicle_count DESC;

```

Como resultado, se obtienen 36 casos donde un mismo *trip_id* está vinculado a dos vehículos distintos. Para mantener un único vehículo por recorrido, seleccionamos, para cada *trip_id*, aquel *vehicle_id* que posee la mayor cantidad de puntos registrados en la tabla *cleaned_vehicle_positions*. Esto se logra mediante la siguiente consulta:

```
CREATE TABLE cleaned_vehicle_positions_filtered AS
SELECT cvp.*
FROM cleaned_vehicle_positions cvp
JOIN (
    WITH vehicle_counts AS (
        SELECT
            trip_id,
            vehicle_id,
            COUNT(*) AS point_count
        FROM cleaned_vehicle_positions
        GROUP BY trip_id, vehicle_id
    ),
    ranked AS (
        SELECT *,
            ROW_NUMBER() OVER (PARTITION BY trip_id ORDER BY
point_count DESC) AS rn
        FROM vehicle_counts
    )
    SELECT trip_id, vehicle_id
    FROM ranked
    WHERE rn = 1
) dominant_vehicles
ON cvp.trip_id = dominant_vehicles.trip_id
AND cvp.vehicle_id = dominant_vehicles.vehicle_id;
```

```
SELECT COUNT(*) from cleaned_vehicle_positions_filtered;
-- 238578
```


3.4 Visualización de las trayectorias reales

En esta sección se introduce una problemática frecuente al trabajar con datos de posiciones GPS: los puntos recolectados no siempre se ajustan correctamente a las calles representada en los mapas, lo que da lugar a trayectorias que, por ejemplo, se desvían de las calles, atraviesan edificios o presentan comportamientos poco realistas.

Para resolver esta inconsistencia visual y analítica, es necesario aplicar técnicas de *map matching*, que permiten alinear las posiciones observadas con la infraestructura vial real. Como primer paso, se presentará un ejemplo de trayectorias sin aplicar *map matching*, con el objetivo de visualizar claramente el desajuste. A continuación, se abordará esta problemática en profundidad, proponiendo el uso de la herramienta Valhalla en combinación con un algoritmo de interpolación temporal, que nos permitirá reconstruir trayectorias más realistas y útiles para el análisis. En esta sección estaremos usando los scripts ubicados en la carpeta `realtime/mapmatching`.

Antes de avanzar, es necesario habilitar la extensión MobilityDB en nuestra base de datos PostgreSQL, ya que esta nos permitirá construir trayectorias espacio-temporales mediante el uso de objetos *tgeompoint*. Para ello, basta con ejecutar la siguiente instrucción SQL:

```
CREATE EXTENSION IF NOT EXISTS mobilitydb;
```

3.4.1 Visualización de las trayectorias sin map matching

Para obtener un primer vistazo de las trayectorias capturadas desde la API, construimos la siguiente tabla:

```
CREATE TABLE raw_actual_trips AS
SELECT
  trip_id,
  vehicle_id,
  route_type,
  tgeompointseq(
    array_agg(tgeompoint(geom, to_timestamp(timestamp)) ORDER BY
to_timestamp(timestamp))
  ) AS trip
FROM cleaned_vehicle_positions_filtered
GROUP BY trip_id, vehicle_id, route_type;

ALTER TABLE raw_actual_trips ADD COLUMN trajectory geometry;
UPDATE raw_actual_trips SET trajectory = trajectory(trip);
```

Esta consulta genera la tabla `raw_actual_trips`, que contiene un *trip* por cada *trip_id*, *vehicle_id* y *route_type* construida a partir de los datos ya depurados en `cleaned_vehicle_positions_filtered`. Utilizamos la función `tgeompointseq` de MobilityDB para crear secuencias de puntos con su marca temporal mediante un orden creciente de tiempo. Finalmente, agregamos la columna *trajectory*, que representa la trayectoria puramente espacial asociada a cada viaje, permitiendo su posterior visualización.

```
SELECT COUNT(*) from actual_trips;
-- 2581

SELECT COUNT(*)
FROM raw_actual_trips
WHERE ST_GeometryType(trajectory) = 'ST_LineString'
-- 2363
```

Si ejecutamos el script `visualize_raw_trajectories.py` obtenemos una vista de las trayectorias como se puede observar en la Figura 55.

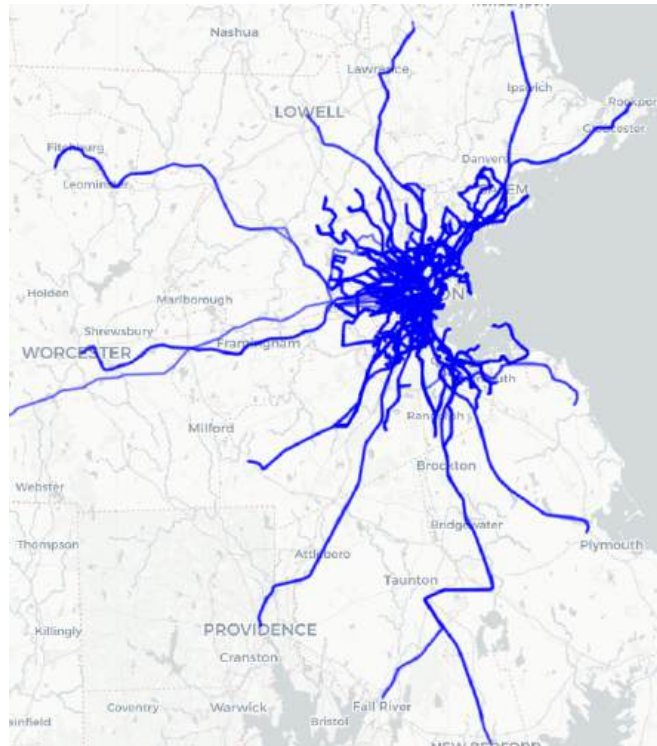


Figura 55: Visualización de las rutas obtenidas a partir de la tabla raw_actual_trips

Sin embargo, aunque las trayectorias parecen precisas a simple vista se observa que no se alinean con los límites reales de las calles, como por ejemplo en la Figura 56. Por este motivo, es necesario aplicar una técnica de *map matching*, que permite ajustar las trayectorias a estos límites.

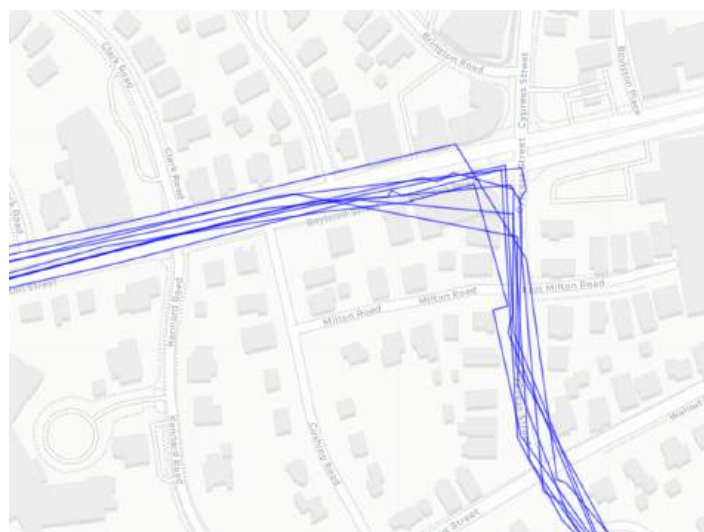
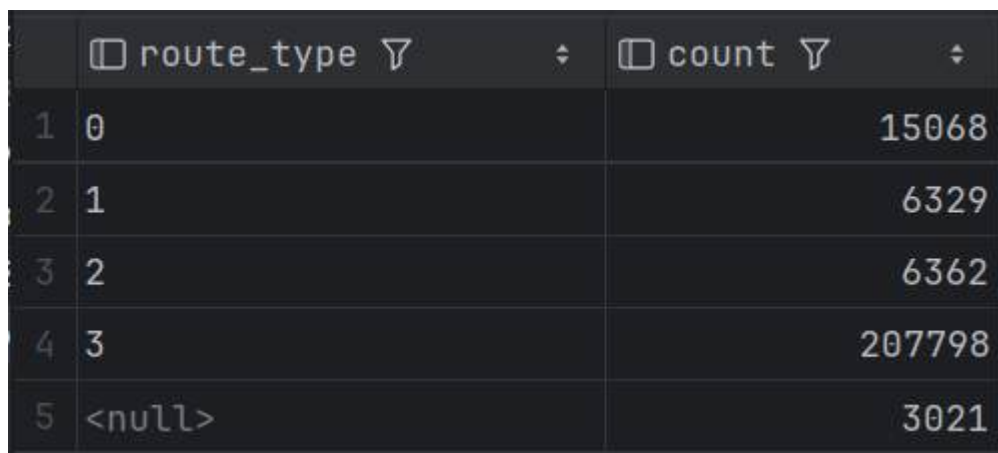


Figura 56: Trayectoria cruda desviada del trazado vial real (vista con zoom).

Ahora bien, como vimos en la sección 2.3.3, el *dataset* tiene *trips* asociadas a diferentes tipos de ruta (Bus, Metro, Tren, Tranvía, entre otros). Ejecutando la siguiente consulta podemos visualizar la distribución de los *trips real time* según el tipo de ruta.

```
SELECT route_type, COUNT(*) AS count
FROM cleaned_vehicle_positions_filtered
GROUP BY route_type
ORDER BY route_type;
```



	route_type	count
1	0	15068
2	1	6329
3	2	6362
4	3	207798
5	<null>	3021

Figura 57: Cantidad de registros por tipo de ruta

Podemos observar en la Figura 57 que existen rutas clasificadas como Tram/Light rail (0), Metro (1), Tren (2), Bus (3), y otras sin clasificar (por falta de correspondencia con el GTFS Schedule). Como la herramienta de *map matching* que utilizaremos en la siguiente sección (Valhalla) solo permite trabajar con trayectos de tipo Bus, nos restringimos a trabajar únicamente ese subconjunto. Cabe destacar que esta decisión no compromete la representatividad del análisis, ya que los trayectos de buses representan aproximadamente el 87 % de los registros del dataset como se puede observar en la Figura 57.

Con el objetivo de analizar un caso puntual para mostrar en funcionamiento la técnica de *map matching* y luego generalizarla, buscamos los buses con mayor cantidad de

puntos distintos registrados en la tabla `cleaned_vehicle_positions_filtered`. Para ello, ejecutamos la siguiente consulta:

```
SELECT cv.trip_id,
       cv.vehicle_id,
       COUNT(*) AS total_points,
       COUNT(DISTINCT cv.geom) AS distinct_points
FROM cleaned_vehicle_positions_filtered cv
JOIN (
  SELECT t.trip_id
  FROM trips t
  JOIN routes r ON t.route_id = r.route_id
  WHERE r.route_type = '3'
) AS bus_trips
ON cv.trip_id = bus_trips.trip_id
GROUP BY cv.trip_id, cv.vehicle_id
ORDER BY distinct_points DESC, total_points DESC;
```

Esta consulta devuelve varias tuplas, ordenadas por la cantidad de puntos espaciales distintos y, en caso de empate, por la cantidad total de registros. A partir de este resultado seleccionamos al azar el *trip_id* 69980438 y *vehicle_id* y3329, correspondiente a un recorrido con 250 puntos geométricos distintos, sobre el cual aplicaremos el proceso de *map matching*.

Si actualizamos el *script* `visualize_raw_trajectory.py` descrito previamente para que filtre por el *trip_id* y *vehicle_id* correspondiente, la geometría resultante muestra la trayectoria cruda, la cual no respeta los límites de las calle (Figuras 58 y 59) ni las rotondas.

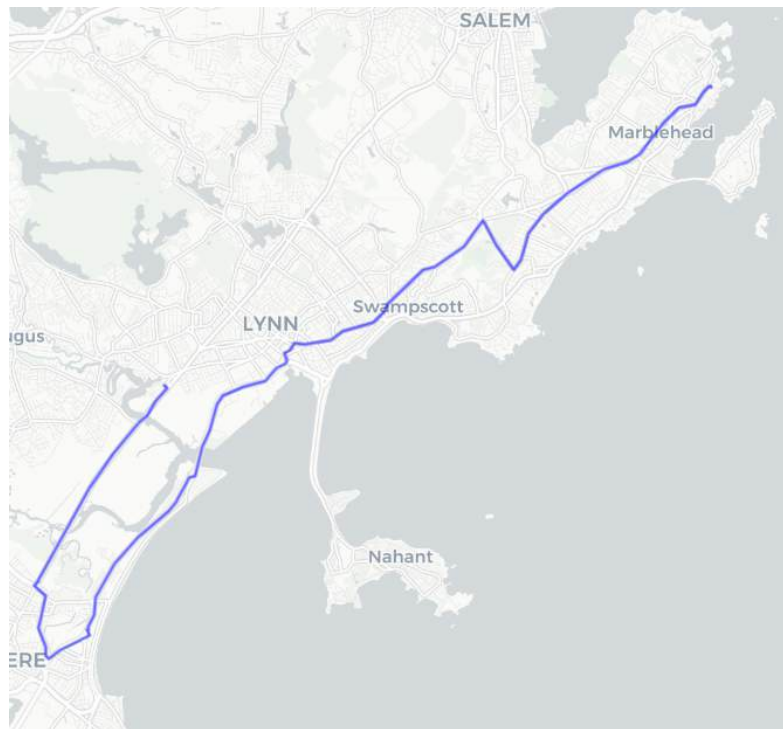


Figura 58: Recorrido del bus con *trip_id* = 69980438 sin *map matching*

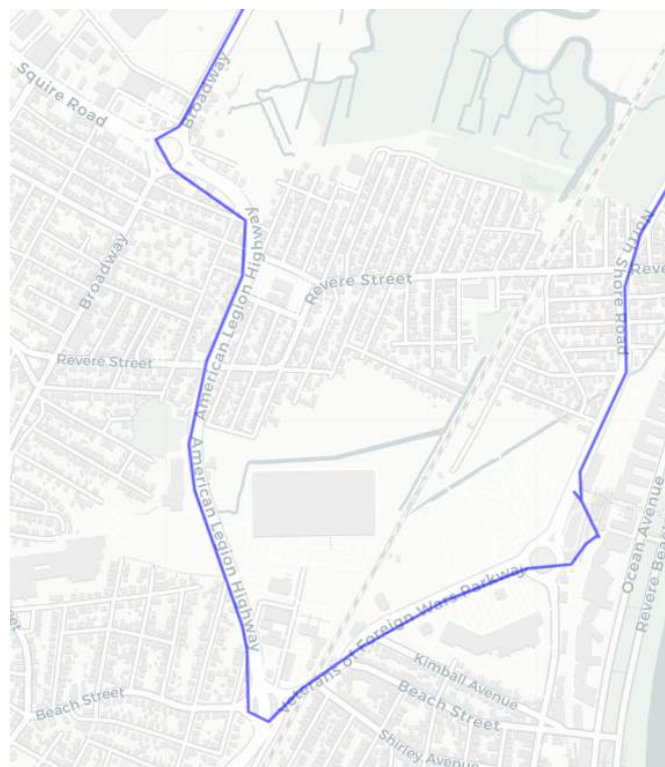


Figura 59: Vista ampliada donde se observa que la trayectoria sale de los límites de la calles y rotondas.

3.4.2 Map matching usando Valhalla

Como su nombre lo indica, el *map-matching* es un proceso para ajustar coordenadas geográficas a un mapa digital. Utilizando como referencia el capítulo 9 del libro del curso, para hacer *map matching* utilizamos la herramienta *open source* llamada [Valhalla](#). Valhalla es un motor de enrutamiento compuesto por un conjunto de librerías diseñado para trabajar con datos de OpenStreetMap, que ofrece funcionalidades como cálculo de matrices de tiempo y distancia, generación de isócronas, muestreo de elevación, *map matching* y optimización de recorridos (problema del viajante).

En nuestro caso, usamos su módulo específico Meili, el cual resuelve el problema de *map matching* mediante un enfoque probabilístico: primero identifica los segmentos de calle más cercanos dentro de un radio determinado y luego, aplicando el algoritmo de Viterbi (basado en modelos ocultos de Markov), infiere la secuencia de tramos más probable que recorrieron los puntos GPS.

Valhalla se ejecuta como un servidor al que se accede mediante una API REST. Para configurarlo, se puede seguir la documentación oficial en el repositorio para compilar el código o usar una imagen de Docker y levantar dicho servidor. Para el *setup* es necesario cargar un mapa, que suele descargarse desde sitios como Geofabrik en formato *protobuf*. Por suerte, para quienes quieren probar Valhalla sin montar su propio servidor, existe una opción a modo de demostración pública alojada por [FOSSGIS que incluye un mapa global](#). En nuestro caso, optamos por utilizar dicho servidor. Sin embargo, el servidor público [impone un límite de uso](#) de 1 solicitud por segundo por usuario y 100 solicitudes por segundo en total para todos los usuarios. Estos límites permiten experimentar con la API sin sobrecargar el servicio. Según la [documentación de la API](#), existen dos endpoints para hacer *map matching*:

1. **/trace_route** reconstruye la ruta más probable que siguió un vehículo a lo largo de las calles. El resultado es una secuencia continua de segmentos que simula una ruta de navegación tradicional.
2. **/trace_attributes** también realiza map matching, pero en lugar de devolver una ruta como tal, entrega información detallada sobre cada segmento del recorrido real, como el nombre de la calle, tipo de vía o velocidad máxima permitida.

En nuestro caso, dado que nos interesa reconstruir las trayectorias efectivas de los vehículos para obtener una representación geoespacial coherente, utilizamos el *endpoint* **/trace_route**. Este *endpoint* corresponde al que se ofrece públicamente en el servidor demo de FOSSGIS bajo el link http://valhalla1.openstreetmap.de/trace_route y está documentado oficialmente en [este enlace](#) que originalmente está en alemán, pero puede traducirse dentro del navegador con la opción de traducción de Google al inglés.

El body de la request al servidor de Valhalla admite múltiples parámetros que influyen en el comportamiento del algoritmo, tales como `search_radius`, `turn_penalty_factor`, entre otros. Si bien más adelante se detallarán los valores elegidos para cada uno de ellos, resulta útil, de forma análoga a lo realizado en la Sección 3.1 con MBTA, desarrollar un cliente específico para Valhalla que nos permita interactuar con este servicio de manera flexible y controlada.

A continuación, se describen las decisiones de implementación adoptadas en el diseño de este cliente, incluyendo consideraciones importantes como los límites de frecuencia impuestos por el servidor (por ejemplo, cantidad máxima de requests por segundo y por usuario), que mencionamos así como otros mecanismos de control para asegurar una recolección estable de los datos.

3.4.2.1 Implementación del cliente Valhalla

Con el objetivo de facilitar la interacción con el servidor público de Valhalla, se desarrolló un cliente denominado `ValhallaClient`, ubicado dentro de la carpeta `realtime/clients/valhalla`. Este cliente construye correctamente los pedidos a la API (en este proyecto, lo acotamos al *endpoint* `/trace_route`) y gestiona el envío de solicitudes con todos los parámetros necesarios. El diseño modular del cliente permite extenderlo fácilmente para interactuar con otros *endpoints* de la API de Valhalla (por ejemplo, extenderlo a `/trace_attributes`).

Los pedidos a la API, por sugerencia del libro del curso, se solicitan en formato `osrm` (*Open Source Routing Machine*), un estándar ampliamente usado en el cálculo de rutas sobre redes de calles. Este formato permite, entre otras cosas, acceder a la geometría del recorrido como un `polyline` codificado, el cual puede decodificarse fácilmente para reconstruir un `LineString`. Los parámetros del *body* de la *request* que permite ajustar el algoritmo de *map matching* están encapsulados en clases específicas (`Costing`, `ShapeMatch`, `Directions`, `Options`, etc.), definidas en la carpeta `clients/valhalla/models`. Para este informe, se incluye únicamente el código del componente `ValhallaClient`, ya que representa la lógica principal de interacción con el servidor Valhalla. Por claridad y concisión, no se incluye el código de las clases que encapsulan el comportamiento de los parámetros del *body*, aunque están disponibles en el repositorio.

```
import requests
from typing import List
from realtime.clients.valhalla.models.measure_with_time import Measure
from realtime.clients.valhalla.models.costing import Costing
from realtime.clients.valhalla.models.shape_match import ShapeMatch
from realtime.clients.valhalla.models.directions import Directions
from realtime.clients.valhalla.models.options import Options
```

```

from realtime.clients.valhalla.models.osrm_response import OSRMResponse
from realtime.clients.valhalla.decorator import retry_on_failure

class ValhallaClient:
    def __init__(self, base_url: str):
        if not base_url:
            raise ValueError("Base URL must be provided.")
        self.base_url = base_url.rstrip("/")

    @retry_on_failure(max_attempts=3, backoff=2)
    def trace_route(
        self,
        measures: List[Measure],
        costing: Costing,
        shape_match: ShapeMatch,
        directions: Directions,
        options: Options,
        parse_tracepoint=False,
    ) -> dict:

        if not measures:
            raise ValueError("At least one measure must be provided.")

        if not costing:
            raise ValueError("Costing must be specified")

        if len(measures) < 2:
            raise ValueError(f"Not enough points to process with
Valhalla\n")

        shape = [m.to_dict() for m in measures]

        payload = {
            "shape": shape,
            "shape_match": shape_match.value,
            "costing": costing.value,
            **directions.to_dict(),
            **options.to_dict(),
        }

        if payload.get("format") != "osrm":
            raise ValueError(
                "Unsupported payload. format: osrm is supported at the
moment"
            )

        print(

```

```

        "Sending request to /trace_route with payload:",
        {k: v for k, v in payload.items() if k != "shape"},
    )

    response = requests.post(
        f"{self.base_url}/trace_route",
        json=payload,
        headers={"Content-Type": "application/json"},
    )
    response.raise_for_status()
    return OSRMResponse.from_valhalla_response(
        response.json(), parse_tracepoints=parse_tracepoint
    )

```

Como se mencionó, el cliente Valhalla tiene limitaciones en términos de *request* por segundo. Es por eso que el método `trace_route` dentro del `ValhallaClient` tiene un [decorator](#) `retry_on_failure()` que reintentar automáticamente la solicitud en caso de errores en la *request*, siempre que el fallo no provenga del lado del cliente, la lógica se encuentra en `realtime/clients/valhalla/decorator.py`.

Además, la función `trace_route()` retorna un objeto `OSRMResponse` (ubicado en `clients/valhalla/models`) que resume la información relevante [contenida en la respuesta](#) de la API de Valhalla.

```

from dataclasses import dataclass
from polyline import decode
from shapely import LineString

@dataclass
class OSRMResponse:
    best_matching_index: int
    geometry: LineString
    confidence: float
    duration_in_seconds: float # in seconds
    distance_in_meters: float # in meters

    @classmethod
    def from_valhalla_response(cls, response: dict) -> "OSRMResponse":
        matchings = response.get("matchings", None)
        tracepoints = response.get("tracepoints", None)

```

```

        if not matchings or not tracepoints:
            raise ValueError(
                "Invalid Valhalla response: missing 'matchings' or
'tracepoints'"
            )

        selected_index, selected_matching = max(
            enumerate(matchings),
            key=lambda x: (x[1].get("confidence", 0.0), x[1].get("weight",
0.0)),
        )

        if selected_matching is None or selected_index is None:
            raise ValueError("No valid matching found in the response")

        decoded_coords = decode(
            expression=selected_matching["geometry"], precision=6,
            geojson=True
        )
        geometry = LineString(decoded_coords)

        if not geometry.is_valid:
            raise ValueError("Decoded geometry is not valid")

        return cls(
            geometry=geometry,
            confidence=selected_matching.get("confidence", 0.0),
            duration_in_seconds=selected_matching.get("duration", 0.0),
            distance_in_meters=selected_matching.get("distance", 0.0),
            best_matching_index=selected_index,
        )

```

Este fragmento de código define `OSRMResponse`, diseñada para estructurar y procesar la respuesta del endpoint `/trace_route` de Valhalla, cuando se solicita en formato OSRM. Su propósito principal es identificar el matching más representativo y decodificar la geometría ajustada (*geometry*).

- `best_matching_index`: índice del matching más adecuado dentro del arreglo `matchings`, elegido según mayor `confidence` y, en caso de empate, mayor `weight`.

- **geometry:** `LineString`, generada al decodificar la *polyline* de la geometría del `matching` seleccionado.
- **confidence, duration_in_seconds, distance_in_meters:** métricas agregadas del recorrido `map-match`ado.

3.4.2.2 Visualización del bus trip de la sección 3.4.1

Con la implementación del cliente `ValhallaClient`, es posible tomar la ruta del bus analizada en la Sección 3.4.1 (correspondiente al `trip_id` **69980438**) y ejecutar el script `visualize_particular_bus_trajectory.py`. Este script permite visualizar la geometría original de la trayectoria (sin *map matching*, en color azul) y la trayectoria ajustada mediante *map matching* (en color rojo).

Internamente, el *script* realiza una consulta a la base de datos para obtener los puntos GPS del bus junto con sus *timestamps* de la tabla `cleaned_vehicle_positions_filtered`. A partir de esta información, construye una lista de objetos `MeasureWithTime` (que encapsulan posición y tiempo), y los utiliza para invocar el método `trace_route()` del `ValhallaClient`.

El cliente genera una solicitud al endpoint `/trace_route` del servidor público de Valhalla, configurada con los siguientes parámetros:

- **Perfil de movilidad:** `bus`
- **Modo de ajuste:** `map_snap`
- **Formato de salida:** `osrm`
- **search_radius:** 100 metros. Este parámetro define el radio de búsqueda para identificar los segmentos de calle más cercanos a cada punto. Valores más altos

aumentan la cobertura pero también el tiempo de procesamiento. El valor de 100 es el máximo ofrecido para ese parámetro.

- **turn_penalty_factor:** penaliza los giros pronunciados o inconsistentes entre segmentos consecutivos. Ayuda a evitar trayectorias que retroceden o se comportan de forma errática. Según la documentación oficial, se recomienda un valor mínimo de 500.

Este primer *script* fue fundamental para validar el funcionamiento del cliente Valhalla y explorar el impacto de diferentes parámetros. A partir de él, desarrollamos versiones extendidas para aplicar el proceso a múltiples buses, lo cual se detalla en la sección siguiente. A continuación, se presentan los resultados obtenidos en esta primera prueba sobre el bus seleccionado e incluso el [html](#) en la carpeta de Drive (Figuras 60 y 61).

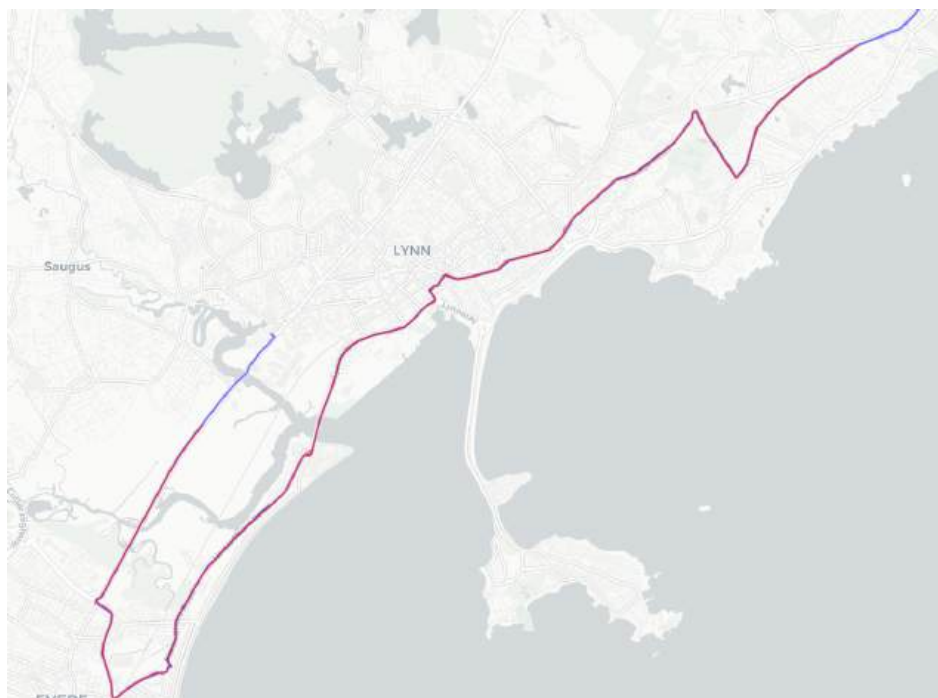


Figura 60: Visualización del *trip* con *map matching*. En rojo se observa la ruta con *map matching* y en azul sin *map matching*.

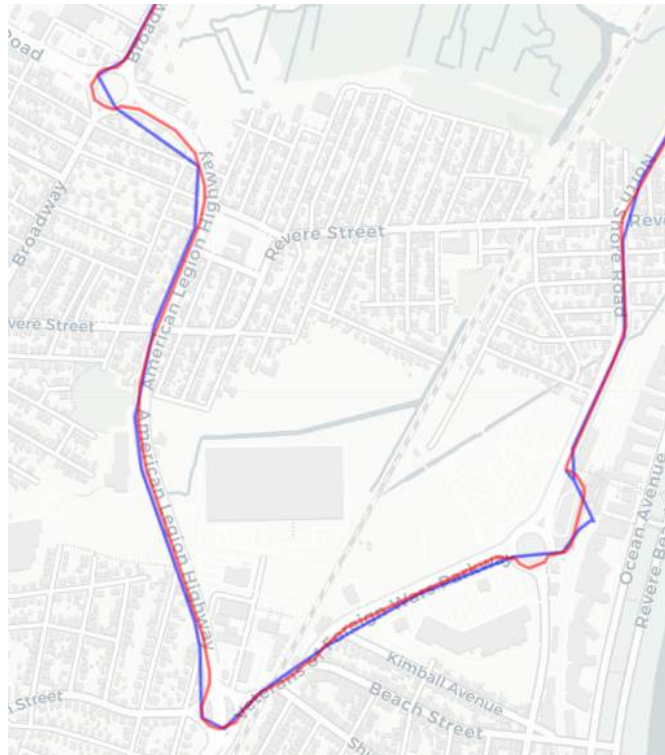


Figura 61: Zoom en la zona de análisis presentada en la sección 3.4.1. En rojo se observa la ruta con *map matching* y en azul sin *map matching*.

3.4.2.3 Visualización de todos los bus trips

Podemos extender el enfoque anterior para aplicar *map matching* sobre todos los viajes disponibles en la tabla `cleaned_vehicle_positions_filtered` para tipo bus. Para garantizar que los resultados sean significativos y que el servicio de Valhalla pueda procesarlos correctamente, incorporamos algunas restricciones de filtrado:

- Cada trip debe contener al menos dos puntos GPS para que Valhalla pueda procesarlo.
- La distancia mínima entre algún par de puntos del trayecto debe ser de al menos 100 metros. Esto es para evitar tener trips que no sean en absoluto representativos.

Este último no debe confundirse con el parámetro `search_radius = 100`, que define el radio (en metros) en el que Valhalla busca coincidencias sobre las calles.

El código correspondiente se encuentra en `visualize_map_matched_bus_trajectories.py`. Tal como está implementado, el *script* procesa aproximadamente 1822 trayectorias por segundo (duración estimada: ~30 minutos). Puede observarse que con el trabajo hecho en las secciones anteriores, el mismo fue rápido de implementar.

```
from numpy import array, linalg
from typing import List
from folium import Map, PolyLine
from geopandas import GeoDataFrame
from pandas import DataFrame
from shapely import Point
from clients.postgres.postgres_client import PostgresClient
from clients.valhalla.valhalla_client import ValhallaClient
from clients.valhalla.models.measure_with_time import MeasureWithTime
from clients.valhalla.models.costing import Costing
from clients.valhalla.models.shape_match import ShapeMatch
from clients.valhalla.models.directions import Directions
from clients.valhalla.models.options import Options
from clients.valhalla.models.osrm_response import OSRMResponse

if __name__ == "__main__":

    postgres_client = PostgresClient(
        db_user="postgres",
        db_pass="postgres",
        db_host="localhost",
        db_port="5432",
        db_name="mbtagtfs",
    )

    valhalla_client =
ValhallaClient(base_url="http://valhalla1.openstreetmap.de")

    m = Map(location=[42.3601, -71.0589], tiles="CartoDB positron",
zoom_start=12)

    # Query to get all trips with route_type = bus
    points: DataFrame = postgres_client.query(
        sql=""
        SELECT
            trip_id AS trip_id,
```

```

        vehicle_id AS vehicle_id,
        longitude AS lon,
        latitude AS lat,
        timestamp AS time
    FROM cleaned_vehicle_positions_filtered cvpf
    WHERE route_type = '3'
"""
)

grouped_points = points.groupby("trip_id")
total_trips = len(grouped_points)

max_distance_between_points = 100 # meters
for i, (trip_id, group) in enumerate(grouped_points):
    vehicle_id = group["vehicle_id"].iloc[0]

    print(
        f"Processing trip {i + 1}/{total_trips}: {trip_id} for
vehicle {vehicle_id}"
    )
    gdf = GeoDataFrame(
        geometry=[Point(lon, lat) for lon, lat in zip(group["lon"],
group["lat"])],
        crs="EPSG:4326",
    ).to_crs(
        epsg=26986
    )

    if gdf.empty or not gdf.is_valid.all():
        print(f"Skipped trip {trip_id} (invalid geometry)\n")
        continue

    # Calculate the maximum distance between points
    coords = array([(geom.x, geom.y) for geom in gdf.geometry])
    dist_matrix = linalg.norm(coords[:, None, :] - coords[None, :,
:], axis=-1)
    max_distance_meters = dist_matrix.max()

    if max_distance_meters < max_distance_between_points:
        print(
            f"Skipped trip {trip_id} (too short
{max_distance_meters:.2f} meters)\n"
        )
        continue

    measures: List[MeasureWithTime] = [

```

```

        MeasureWithTime(lon=row["lon"], lat=row["lat"],
time=row["time"])
        for _, row in group.iterrows()
    ]

    if len(measures) < 2:
        print(
            f"Skipped trip {trip_id} (not enough points to process
with Valhalla)\n"
        )
        continue

    try:
        output: OSRMResponse = valhalla_client.trace_route(
            measures=measures,
            costing=Costing.BUS,
            shape_match=ShapeMatch.MAP_SNAP,
            directions=Directions().set_format("osrm"),
            options=Options()
                .set_search_radius(100)
                .set_turn_penalty_factor(500)
                .set_use_timestamps(True),
        )
    except Exception as e:
        print(f"Error processing trip {trip_id}: {e}\n")
        continue

    gdf_matched = GeoDataFrame(geometry=[output.geometry],
crs="EPSG:4326")
    for _, row in gdf_matched.iterrows():
        coords = [(lat, lon) for lon, lat in row.geometry.coords]
        PolyLine(
            coords,
            color="red",
            weight=2,
            opacity=0.6,
            popup=f"Matched trajectory for {vehicle_id} on trip
{trip_id}",
        ).add_to(m)

    print(f"Processed trip {trip_id} successfully\n")

    m.save("bus_map_matched_geometries.html")

```

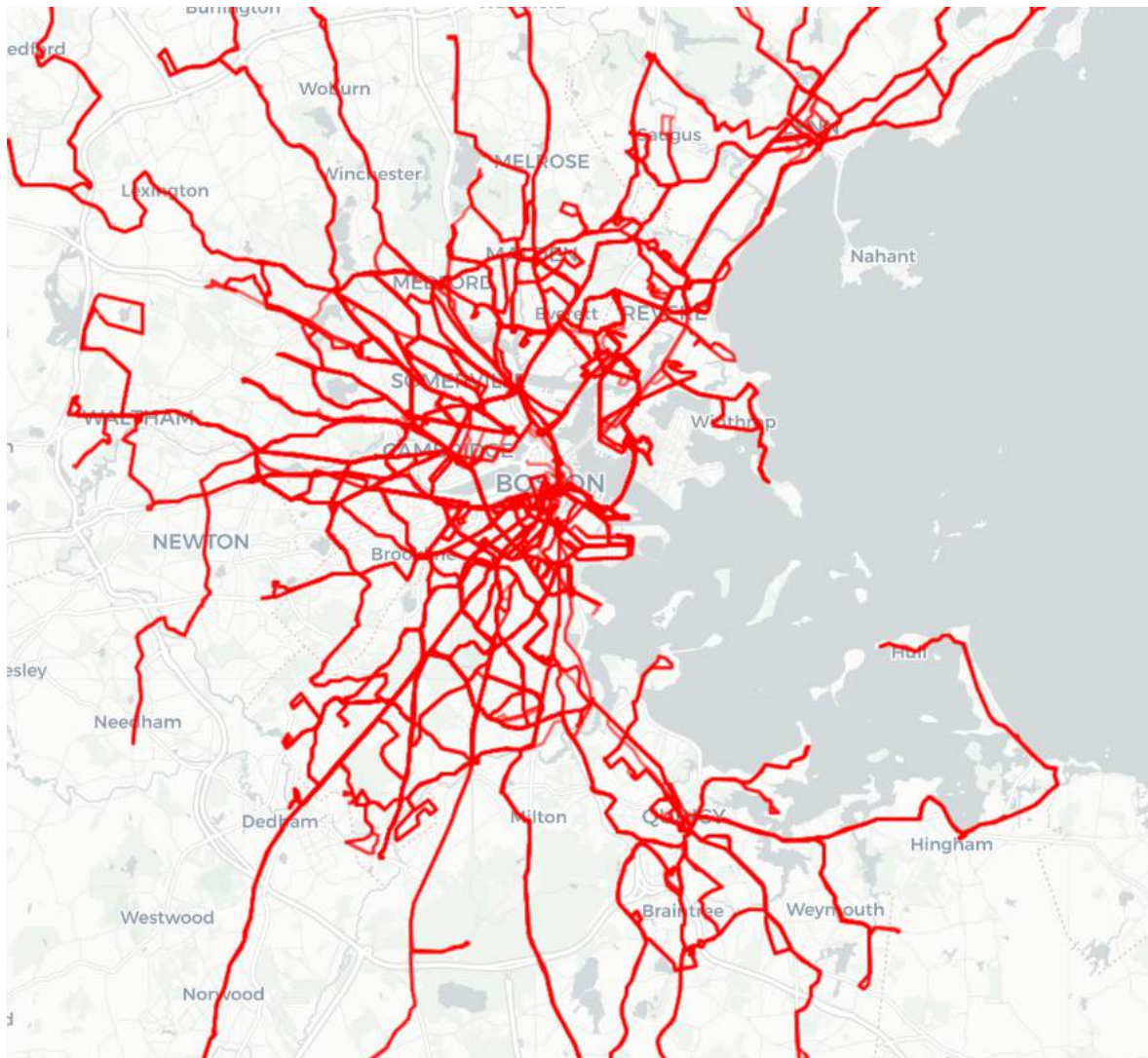


Figura 62: Visualización de las rutas de tipo bus luego de aplicar *map matching*

El resultado de la Figura 62 no representa los *trips ajustados* que cargaremos en la base de datos. Es una figura representativa para mostrar que fue posible visualizar todas las rutas utilizando el cliente que implementamos sin inconvenientes. En la siguiente sección abordaremos la problemática de los trips ajustados y de las modificaciones que tuvimos que hacer en el *body* de la *request* para obtener mejores resultados.

3.5 La problemática del armado de los *bus trips* “ajustados”

En la sección anterior llevamos adelante el proceso de *map matching* utilizando el servicio `/trace_route` de Valhalla. A partir de los puntos GPS capturados en tiempo real en las secciones 3.1, 3.2 y 3.3, pudimos obtener su geometría ajustada que representa la trayectoria más probable seguida por cada bus, corrigiendo las imprecisiones del GPS para alinearlas con las calles en las que transitan. Finalmente, estas trayectorias ajustadas fueron visualizadas en el mapa validando que nuestro cliente implementado funciona para realizar pedidos para ajustar trayectorias.

Ahora, el nuevo desafío que enfrentamos, es la construcción de los *trips* “ajustados”, es decir, trayectorias que además de la posición contengan la dimensión temporal. Valhalla, a pesar de devolver una geometría ajustada y, por separado, los puntos ingresados por *input* ajustados (llamados *tracepoints* en la documentación oficial), no asocia *timestamps* a estos puntos. Esto impide utilizar directamente su salida como una secuencia espacio-temporal en MobilityDB. Incluso, hay veces en que la cantidad de puntos ajustados no coincide con la cantidad de puntos que fueron ingresados como *input*, lo que complica aún más la situación.

Investigamos la documentación oficial de Valhalla y distintos *issues* en su repositorio, y observamos que aunque el servicio permite configurar parámetros como *use_timestamps*, *begin_time* o *durations*, no hay forma de que se incluya información temporal en la respuesta. Esta limitación ha sido reconocida por múltiples usuarios de la comunidad. En otras palabras, Valhalla realiza una corrección espacial, pero no mantiene, ni reconstruye, ni facilita la dimensión temporal de los puntos procesados.

Frente a esta situación, tomamos la decisión de trabajar con los puntos ajustados que devuelve Valhalla y asignarles, uno a uno, los *timestamps* reales que nosotros ya tenemos y que fueron utilizados como entrada en la *request*. Este enfoque nos permite

conservar la información temporal obtenida del *feed*, al tiempo que trabajamos con geometrías espacialmente corregidas. Sin embargo, para que esta estrategia sea válida, es necesario que la cantidad de puntos ajustados obtenidos coincida exactamente con la cantidad de puntos originales que pasamos como *input* en la *request*. Por ese motivo, vamos a conservar únicamente aquellos viajes en los que el número de puntos de salida de Valhalla coincida con el número de puntos enviados como entrada.

Una vez que contamos con los puntos ajustados (*tracepoints*) y los *timestamps* originales, una opción sería asignar esos tiempos directamente a los puntos ajustados y proyectarlos sobre la línea ajustada (*polyline*). Sin embargo, los *tracepoints* no representan la geometría final: cada uno se corresponde con un punto de entrada y puede o no formar parte de la geometría construida por Valhalla (esto se puede verificar del output de la API). Usarlos como base geométrica llevaría a una trayectoria de menor resolución que no coincida con las calles.

Por este motivo, adoptamos el enfoque inverso: usamos los *tracepoints* como referencias temporales y asignamos *timestamps* a cada punto de la geometría ajustada generada por Valhalla. Esto se logra proyectando los puntos de la geometría sobre los segmentos definidos por las referencias temporales e interpolando sus tiempos en función de la distancia acumulada. En otras palabras, dicho mal y pronto, buscamos "agregarle tiempo a los puntos del LineString".

Como dijimos, nuestra propuesta se basa exclusivamente en los tiempos asignados a los *tracepoints* y una [interpolación lineal a lo largo de una geometría lineal y no cerrada](#). Para eso, presentamos la fórmula a utilizar:

$$t = t_1 + \frac{d-d_1}{d_2-d_1} \times (t_2 - t_1)$$

- t : es el *timestamp* interpolado que queremos asignar a un punto perteneciente al *LineString*.
- t_1, t_2 : son los *timestamps* originales de los puntos de anclaje o referencia (tracepoints) que se usan para obtener t .
- d : es la *distancia proyectada* desde el inicio del *LineString* hasta el punto sobre dicha geometría al que queremos asignarle un *timestamp*. Representa cuánto “avanzamos” sobre la geometría desde su inicio hasta ese punto. En otras palabras, es la distancia acumulada del punto dentro de la línea.
- d_1, d_2 : son las *distancias proyectadas* sobre el *LineString* de los puntos de anclaje p_1 y p_2 , respectivamente.

Los puntos de anclaje se ordenan inicialmente de menor a mayor según su distancia proyectada sobre el *LineString*, y se utilizan para definir los segmentos dentro de los cuales se intenta ubicar cada punto a interpolar. En caso de que el segmento definido por los puntos de anclaje sea inválido, ya sea porque son iguales o porque el punto del *LineString* a interpolar se encuentra más adelante, se avanza al siguiente segmento. Además, si bajo esa condición, el segundo punto de anclaje p_2 no coincide con el último punto interpolado, y su *timestamp* es mayor al del último tiempo interpolado, se lo agrega explícitamente al resultado como si fuera un punto interpolado. Esto se debe a que, aunque los *tracepoints* pueden o no formar parte de la geometría ajustada, representa un instante relevante capturado en el *protobuf* de MBTA. Omitirlos implicaría perder una referencia espacio-temporal significativa que contribuye a preservar la coherencia temporal del recorrido y de lo que hemos *sampleado* en la sección 3.1. Para hacer todo esto, haremos mucho uso de la librería de Python llamada *shapely* que nos permite manipular geometrías fácilmente.

Finalmente, los resultados de la interpolación serán cargados en la tabla `map_matched_bus_trips`, donde este último es un *tgeompoint* construido a partir de

dichos puntos. Todos los scripts de esta sección están implementados en
realtime/interpolated

```
CREATE TABLE map_matched_bus_trips (
  trip_id TEXT NOT NULL,
  vehicle_id TEXT NOT NULL,
  trip tgeompoint NOT NULL,
  PRIMARY KEY (trip_id, vehicle_id)
);
```

3.5.1 Modificando el cliente Valhalla para parseo de tracepoints

Tal como lo indica el título de la sección, tenemos que modificar la clase
OSRMResponse implementada en la sección 3.4.2.1 para que ahora incluya los puntos
ajustados en la respuesta para luego implementar la lógica de interpolación temporal y
posteriormente la carga en la base de datos.

```
from dataclasses import dataclass
from typing import List
from polyline import decode
from shapely import LineString, Point

@dataclass
class OSRMResponse:
    best_matching_index: int
    geometry: LineString
    confidence: float
    duration_in_seconds: float # in seconds
    distance_in_meters: float # in meters
    tracepoints: List[Point]

    @classmethod
    def from_valhalla_response(
        cls, response: dict, parse_tracepoints: bool = False
    ) -> "OSRMResponse":
        matchings = response.get("matchings", None)
        raw_tracepoints_data = response.get("tracepoints", None)

        if not matchings or not raw_tracepoints_data:
            raise ValueError(
                "Invalid Valhalla response: missing 'matchings' or 'tracepoints'"
            )
```

```

    )

    selected_index, selected_matching = max(
        enumerate(matchings),
        key=lambda x: (x[1].get("confidence", 0.0), x[1].get("weight", 0.0)),
    )

    if selected_matching is None or selected_index is None:
        raise ValueError("No valid matching found in the response")

    decoded_coords = decode(
        expression=selected_matching["geometry"], precision=6, geojson=True
    )
    geometry = LineString(decoded_coords)

    if not geometry.is_valid:
        raise ValueError("Decoded geometry is not valid")

    tracepoints = []
    if parse_tracepoints:
        for tracepoint_data in raw_tracepoints_data:
            if tracepoint_data is None:
                continue
            location = tracepoint_data.get("location", None)
            if location is None:
                continue
            lon, lat = location # Valhalla returns [lon, lat]
            tracepoints.append(Point(lon, lat)) # Shapely expects (x=lon, y=lat)

    return cls(
        geometry=geometry,
        confidence=selected_matching.get("confidence", 0.0),
        duration_in_seconds=selected_matching.get("duration", 0.0),
        distance_in_meters=selected_matching.get("distance", 0.0),
        best_matching_index=selected_index,
        tracepoints=tracepoints,
    )

```

Como puede observarse en el fragmento anterior, se incorporó el campo *tracepoints* como una lista de *Point* de la biblioteca *shapely*, representando los puntos ajustados de la trayectoria. La clase ya contaba con un atributo con la geometría ya decodificada de *LineString*, también de *shapely* (ver sección 3.4.2.1). Además, se agregó un parámetro opcional *parse_tracepoints* al método de clase. Este parámetro permite controlar si se desea parsear o no dichos puntos, ya que su procesamiento puede resultar costoso en

términos de cómputo cuando la cantidad de puntos es elevada. De este modo, evitamos sobrecargar innecesariamente el flujo de trabajo previamente realizado en la sección 3.4.2.

3.5.2 Algoritmo de interpolación temporal

El algoritmo de interpolación fue descrito con amplios detalles en la sección 3.5 y puede encontrarse en el archivo `interpolation.py` de la carpeta `valhalla`. Como dijimos, el mismo devuelve una serie de puntos asociados a su *timestamp* (representado en la clase *tgeompoint*).

```
from typing import List
from shapely import LineString, Point
from math import inf
from realtime.clients.valhalla.models.tgeompoint import TGeomPoint

# https://spin.atomicobject.com/interpolate-along-linestring/
def assign_timestamps_to_linestring(
    anchor_points: List[Point],
    anchor_times: List[int],
    geometry: LineString,
) -> List[TGeomPoint]:
    """
    Assigns interpolated timestamps to each vertex of a LineString using known
    anchor points as time references.
    For each vertex in the geometry, the function locates the anchor segment it
    falls into and linearly interpolates its timestamp.
    """

    # Ensure the geometry has no self-intersections.
    # If not linear geometries, projection distances along the Linestring may
    become ambiguous.
    #
    # https://shapely.readthedocs.io/en/2.1.1/reference/shapely.LineString.html#shapely
    # .LineString.interpolate
    #
    # https://shapely.readthedocs.io/en/2.1.1/reference/shapely.LineString.html#shapely
    # .LineString.project
    if not geometry.is_simple:
        raise ValueError(
            "Geometry is not simple. It contains self-intersections. This can
            lead to incorrect projections or interpolations results"
        )
```

```

if len(anchor_points) < 2 or len(anchor_times) < 2:
    raise ValueError("At least two anchors with timestamps are required.")

# Project anchor points onto the Linestring and pair with timestamp
# Sort by distance along the Linestring
anchors = [
    (point, geometry.project(point), timestamp)
    for point, timestamp in zip(anchor_points, anchor_times)
]
anchors.sort(key=lambda x: x[1])

# Results
result = []

# Variables to help in the algorithm
previous_interpolated_time, previous_interpolated_point = -inf, None

# Start with the first segment
start_point, start_dist, start_time = anchors.pop(0)
end_point, end_dist, end_time = anchors.pop(0)

for coord in geometry.coords:
    current_point = Point(coord)
    current_dist = geometry.project(current_point)

    # Advance segment if current point is beyond it or if it's invalid
    while len(anchors) > 0 and (
        start_point.equals_exact(end_point) or current_dist > end_dist
    ):
        # Store p2 to maintain consistency with retrieved data in protobuf
        if (
            not end_point.equals_exact(previous_interpolated_point)
            and end_time > previous_interpolated_time
        ):
            result.append(TGeomPoint(end_point, end_time))
            previous_interpolated_time = end_time
            previous_interpolated_point = end_point
            start_point, start_dist, start_time = end_point, end_dist, end_time
            end_point, end_dist, end_time = anchors.pop(0)

    # Interpolate only if the current point is within a valid segment
    if not start_point.equals_exact(end_point) and current_dist <= end_dist:

        # Interpolation formula
        interpolated_time = start_time + (
            (current_dist - start_dist) / (end_dist - start_dist)
        ) * (end_time - start_time)

```

```

        if interpolated_time < previous_interpolated_time:
            raise ValueError(
                f"Non-increasing timestamp at distance {current_dist:.2f}."
            )
        elif interpolated_time == previous_interpolated_time:
            continue

        result.append(TGeomPoint(current_point, interpolated_time))
        previous_interpolated_time = interpolated_time
        previous_interpolated_point = current_point

    return result

```

3.5.3 Carga de los bus trips ajustados

El último paso, es cargar los puntos interpolados en la base datos en la tabla que mencionamos al inicio de esta sección. Dejamos el código final que puede encontrarse en el `script load_interpolated_trajectories_to_database.py`. De todas formas, hemos dejado un [sql](#) en el link de Drive que permite crear la tabla y sus datos para agilizar la carga.

```

from typing import List
from pandas import DataFrame
from realtime.clients.postgres.postgres_client import PostgresClient
from realtime.clients.valhalla.valhalla_client import ValhallaClient
from realtime.clients.valhalla.models.measure_with_time import
MeasureWithTime
from realtime.clients.valhalla.models.costing import Costing
from realtime.clients.valhalla.models.shape_match import ShapeMatch
from realtime.clients.valhalla.models.directions import Directions
from realtime.clients.valhalla.models.options import Options
from realtime.clients.valhalla.models.osrm_response import OSRMResponse
from realtime.clients.valhalla.interpolation import
assign_timestamps_to_linestring
from realtime.clients.valhalla.models.tgeompoint import TGeomPoint
from realtime.clients.valhalla.utils import is_trip_long_enough

if __name__ == "__main__":

```

```

postgres_client = PostgresClient(
    db_user="postgres",
    db_pass="postgres",
    db_host="localhost",
    db_port="5432",
    db_name="mbtagtfs",
)

valhalla_client =
ValhallaClient(base_url="http://valhalla1.openstreetmap.de")

points: DataFrame = postgres_client.query(
    sql="""
SELECT
    trip_id AS trip_id,
    vehicle_id AS vehicle_id,
    longitude AS lon,
    latitude AS lat,
    timestamp AS time
FROM cleaned_vehicle_positions_filtered cvpf
WHERE route_type = '3'
    AND trip_id NOT IN (
        SELECT trip_id FROM map_matched_bus_trips
    )
    """
)

grouped_points = points.groupby("trip_id")
total_trips = len(grouped_points)
successfully_processed = 0
for i, (trip_id, group) in enumerate(grouped_points):
    vehicle_id = group["vehicle_id"].iloc[0]

    print(
        f"Processing trip {i + 1}/{total_trips}: {trip_id} for
vehicle {vehicle_id}"
    )

    if not is_trip_long_enough(group, min_distance_meters=100.0):
        print(f"Skipped trip {trip_id} (too short)\n")
        continue

    measures: List[MeasureWithTime] = [
        MeasureWithTime(lon=row["lon"], lat=row["lat"],
time=row["time"])

```



```

        for _, row in group.iterrows()
    ]

    try:
        output: OSRMResponse = valhalla_client.trace_route(
            measures=measures,
            costing=Costing.BUS,
            shape_match=ShapeMatch.MAP_SNAP,
            directions=Directions().set_format("osrm"),
            options=Options()
                .set_search_radius(100)
                .set_turn_penalty_factor(10000)
                .set_use_timestamps(True),
            parse_tracepoint=True,
        )
    except Exception as e:
        print(f"Error processing trip {trip_id}: {e}\n")
        continue

    if len(output.tracepoints) != len(measures):
        print(f"Skipped trip {trip_id} (tracepoints count
mismatch)\n")
        continue

    try:
        interpolated: List[TGeomPoint] =
assign_timestamps_to_linestring(
            anchor_points=output.tracepoints,
            anchor_times=group["time"].astype(int).tolist(),
            linestring=output.geometry,
        )
    except Exception as e:
        print(f"Interpolation failed for trip {trip_id}: {e}\n")
        continue

    try:
        postgres_client.execute(
            sql="""
INSERT INTO map_matched_bus_trips (trip_id, vehicle_id,
trip)

VALUES (:trip_id, :vehicle_id, tgeompoint :trip)
""",
            params={
                "trip_id": trip_id,
                "vehicle_id": vehicle_id,
                "trip": f"SRID=4326; [{','.join([str(tgp) for tgp in

```

```

interpolated]]]]",
        },
    )
    except Exception as e:
        print(f"Failed to insert trip {trip_id}: {e}\n")
        continue

    successfully_processed += 1
    print(
        f"Successfully processed trip {trip_id}
({successfully_processed}/{total_trips})\n"
    )

    print(f"Successfully processed
{successfully_processed}/{total_trips} trips")

```

De las 1822 trayectorias procesadas con el código previamente descrito, en un primer intento logramos interpolar correctamente 930 trayectorias. La mayoría de los descartes se debieron a los siguientes tipos de errores: *requests* muy pesadas que el servidor no la puede procesar por la alta cantidad de puntos, LineStrings con autointersecciones y discrepancias entre la cantidad de puntos de inputs y los *tracepoints*

Dado que este número representaba poco más del 50% del total, nos propusimos mejorar la tasa de éxito. Comenzamos ajustando los parámetros de Valhalla. Mantuvimos el *search_radius* en 100 metros, ya que valores menores reducen significativamente la precisión del ajuste. En cambio, nos enfocamos en modificar el parámetro *turn_penalty_factor* que consideramos el más influyente en la calidad del resultado.

Luego de múltiples pruebas, encontramos que incrementar significativamente el valor de *turn_penalty_factor* nos permitió mejorar sustancialmente los resultados. Con un valor 20 veces mayor al predeterminado (10.000), logramos aumentar la cantidad de trayectorias exitosas de 975 a aproximadamente 1128. Al seguir aumentando ese parámetro desde 10.000 (por ejemplo, hasta 150.000), el número de trayectorias exitosas descendía, sugiriendo la existencia de un punto óptimo intermedio y además muchas rutas terminaban ajustándose mal. Cabe destacar que, al tratarse de una API pública alojada por FOSSGIS,

desconocemos los valores internos de parámetros como delta, gamma y sigma utilizados por la aplicación, lo que limita la posibilidad de un ajuste fino más preciso y nos obliga a operar por prueba y error (además de que se trata de un algoritmo probabilístico).

3.5.4 Visualización de los bus trips ajustados

A partir de lo hecho en la sección anterior, podemos agregar la columna de trayectoria geom para los *trips* de los buses interpolados y visualizarlas con el *script* llamado `visualize_interpolated_trajectories.py`

```
ALTER TABLE map_matched_bus_trips ADD COLUMN geom geometry;  
UPDATE map_matched_bus_trips SET geom = trajectory(trip);
```

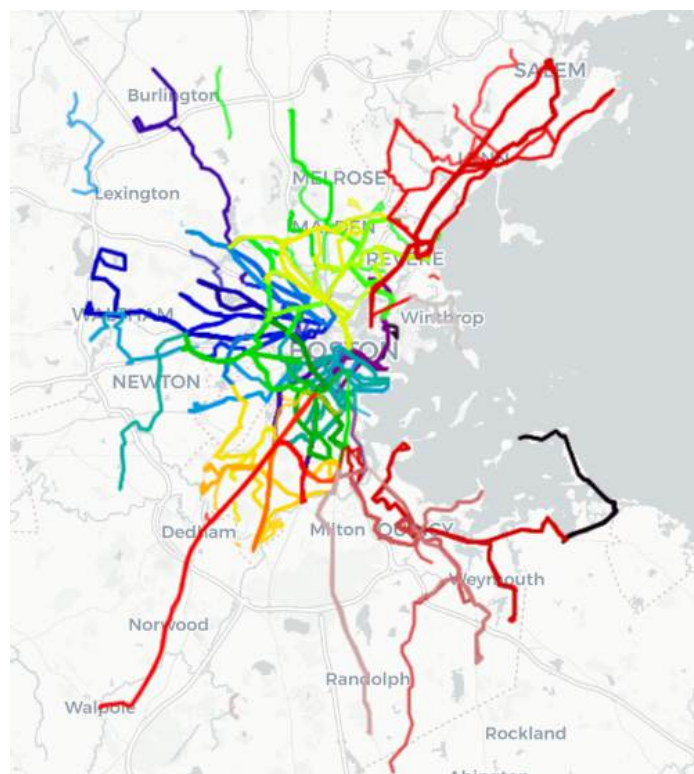


Figura 63: Visualización de las trayectorias registradas de los buses en Boston entre las 15:00 y las 18:00 (hora local)

Tal como se analizó en la Sección 2.3.3 (Figura 5), se identificaron 835 viajes planeados. En la Figura 63 se visualizan los 1128 viajes obtenidos, superando así la cantidad planeada.

3.6 Construcción de segmentos

Siguiendo la metodología descrita en la Sección 11.4.5 del libro de la materia, en esta sección se busca generar los segmentos reales recorridos por los vehículos a partir de sus trayectorias. Para eso, se utilizaron las funciones `nearestApproachInstant` y `nearestApproachDistance`, que permiten estimar con precisión tanto el instante como la posición más cercana de un vehículo respecto a cada una de las paradas planificadas y así obtener las paradas asociadas a cada viaje. Luego con eso, es posible obtener los segmentos para cada viaje. Todos los *scripts* utilizados en esta sección se encuentran en `realtime/segments`.

3.6.1 Preprocesamiento de los trips ajustados

El primer paso es reproyectar las trayectorias de la tabla `map_matched_bus_trips` al sistema de coordenadas EPSG:26986, correspondiente a Massachusetts. Esta transformación permitió aplicar correctamente las funciones `nearestApproachInstant` y `nearestApproachDistance`. Para ello, se crearon dos nuevas columnas que luego fueron completadas utilizando las funciones `tgeompointseq` y `trajectory`.

```
ALTER TABLE map_matched_bus_trips
  ADD COLUMN trip_26986      tgeompoint,
  ADD COLUMN trajectory_26986 geometry(LineString, 26986);

UPDATE map_matched_bus_trips
SET trip_26986 = tgeompointseq(ARRAY(
  SELECT tgeompoint(ST_Transform(getvalue(inst), 26986),
                    gettimestamp(inst))
  FROM unnest(instants(trip)) AS inst
  ORDER BY gettimestamp(inst)
));

UPDATE map_matched_bus_trips
```

```
SET trajectory_26986 = trajectory(trip_26986);
```

3.6.2 Creación de la tabla *trip_stops*

A partir del preprocesamiento de los *trips* ajustados, se creó la tabla *trip_stops*, cuya finalidad es vincular cada trayecto real con las paradas planificadas más próximas. Para establecer esta relación, se utilizó la función *nearestApproachInstant*, que identifica el instante de la trayectoria más cercano a una parada específica. Además, se incorporó un filtro de distancia con *nearestApproachDistance* para evitar asociaciones incorrectas en casos donde la trayectoria estuviera incompleta o demasiado alejada de la parada.

```
CREATE TABLE trip_stops AS
WITH Temp AS (
  SELECT
    m.trip_id,
    ad.shape_id,
    s.stop_id,
    s.stop_name,
    ad.stop_sequence,
    s.stop_loc::geometry,
    ad.t_arrival AS schedule_time,
    nearestApproachInstant(
      m.trip_26986,
      ST_Transform(s.stop_loc::geometry, 26986)
    ) AS stop_instant
  FROM map_matched_bus_trips m
  JOIN arrivals_departures ad ON m.trip_id = ad.trip_id
  JOIN stops s ON ad.stop_id = s.stop_id
  WHERE ad.date = '2025-07-12'::timestamp
  AND nearestApproachDistance(
    m.trip_26986,
    ST_Transform(s.stop_loc::geometry, 26986)
  ) < 20
)
SELECT
  trip_id,
```

```
shape_id,  
stop_id,  
stop_name,  
stop_sequence,  
stop_loc,  
schedule_time,  
getTimestamp(stop_instant) AS actual_time,  
ST_Transform(getValue(stop_instant), 4326) AS trip_geom  
FROM Temp;
```

La consulta construye la tabla `trip_stops`, que toma muestras de las trayectorias reales en las ubicaciones de las paradas definidas por el GTFS Schedule. Para ello, se genera previamente una tabla temporal que integra tres fuentes: `map_matched_bus_trips` (trayectorias interpoladas), `arrivals_departures` (paradas programadas) y `stops` (geometría de las paradas). También se recupera el atributo *shape_id*, clave para identificar los viajes que recorren un mismo segmento. Esta tabla ofrece una vista detallada de las diferencias entre los datos programados y los reales de los viajes, lo que permite analizar demoras, llegadas anticipadas o desvíos respecto al recorrido planificado.

Cabe destacar que en la consulta se emplea un *magic number* que actúa como umbral de distancia para considerar válida la asociación entre una parada y la trayectoria real del viaje. Inicialmente utilizamos un valor de 10 metros, tal como sugiere el libro de la materia, pero observamos que en varios casos no se lograba establecer la asociación con ciertas paradas. Por ello, realizamos pruebas comparativas utilizando umbrales de 10 y 20 metros, visualizando en cada caso la trayectoria real del viaje (en azul), las paradas programadas (puntos rojo) y las paradas detectadas (marcadores en verde).

Decidimos no utilizar umbrales mayores a 20 metros para evitar asociaciones incorrectas, ya que esto podría comprometer la precisión del análisis. Consideramos que 20 metros nos resulta una distancia suficientemente acotada y razonable como para capturar más casos. Si bien los casos que se presentan a continuación fueron útiles para definir este umbral, un análisis verdaderamente representativo requeriría una muestra más amplia de

viajes. En este trabajo, sin embargo, se optó por un análisis detallado de tres trayectos distintos que, aunque no buscan ser exhaustivos, permiten ilustrar de forma clara el enfoque adoptado desde una perspectiva académica. Se muestran ejemplos en los que un umbral de 10 metros no permite detectar correctamente ciertas paradas, y cómo estas son recuperadas al ampliar el umbral a 20 metros (Figuras 64 a 69).

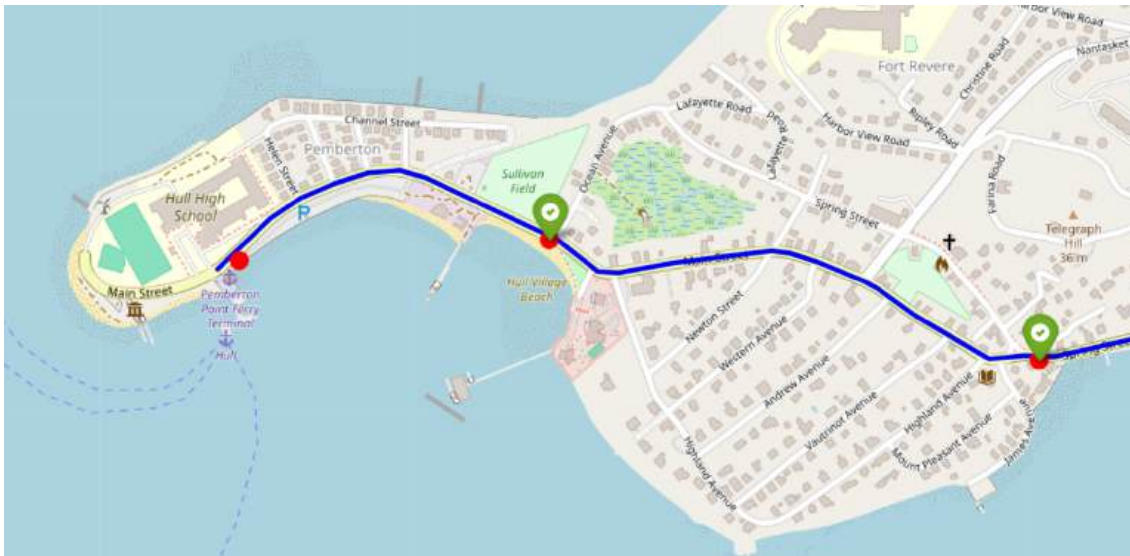


Figura 64: Visualización de una porción del viaje 68964189 utilizando un umbral de 10 metros

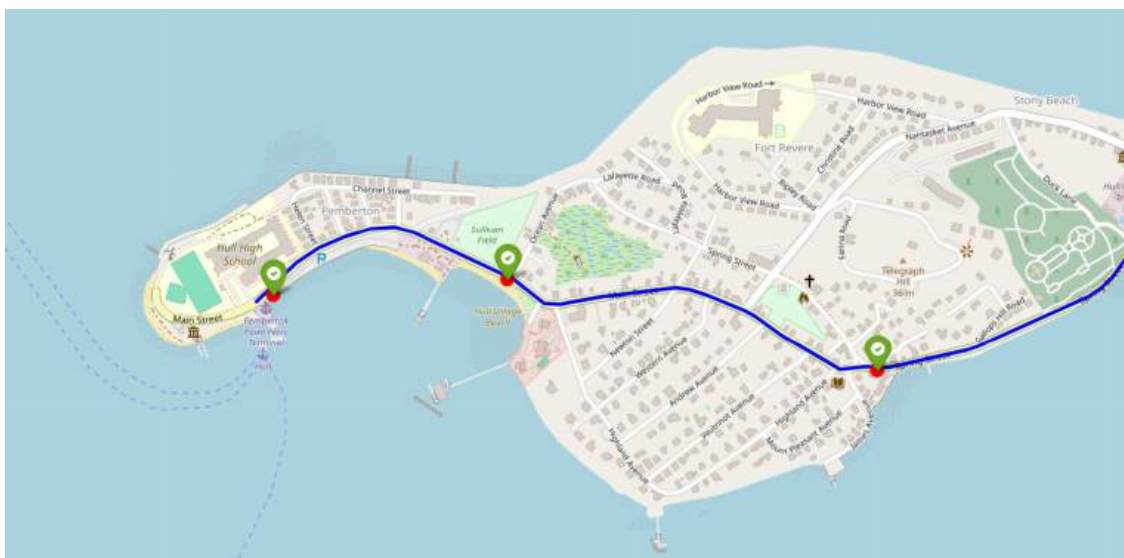


Figura 65: Visualización de una porción del viaje 68964189 utilizando un umbral de 20

metros

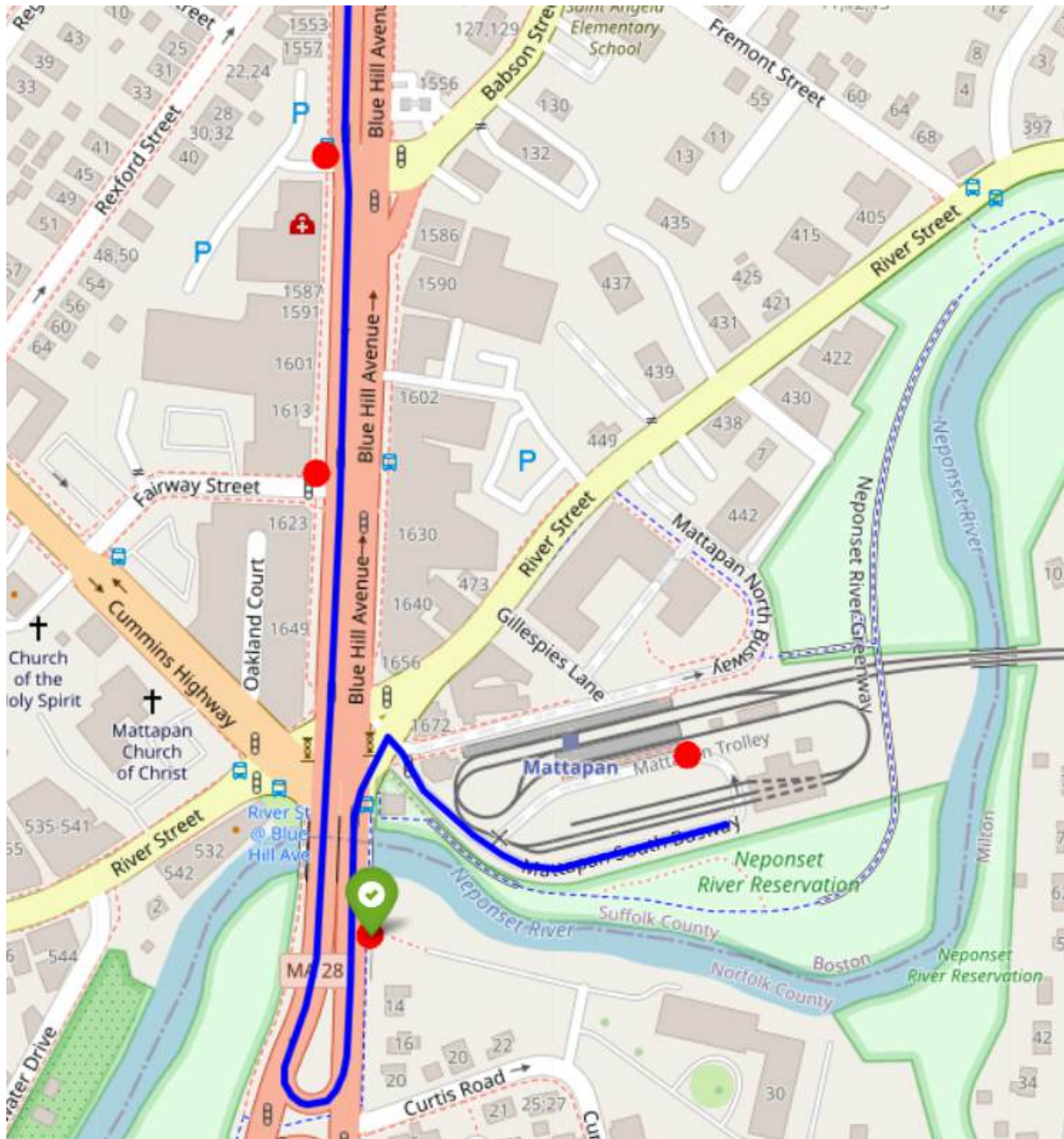


Figura 66: Visualización de una porción del viaje 68992342 utilizando un umbral de 10 metros

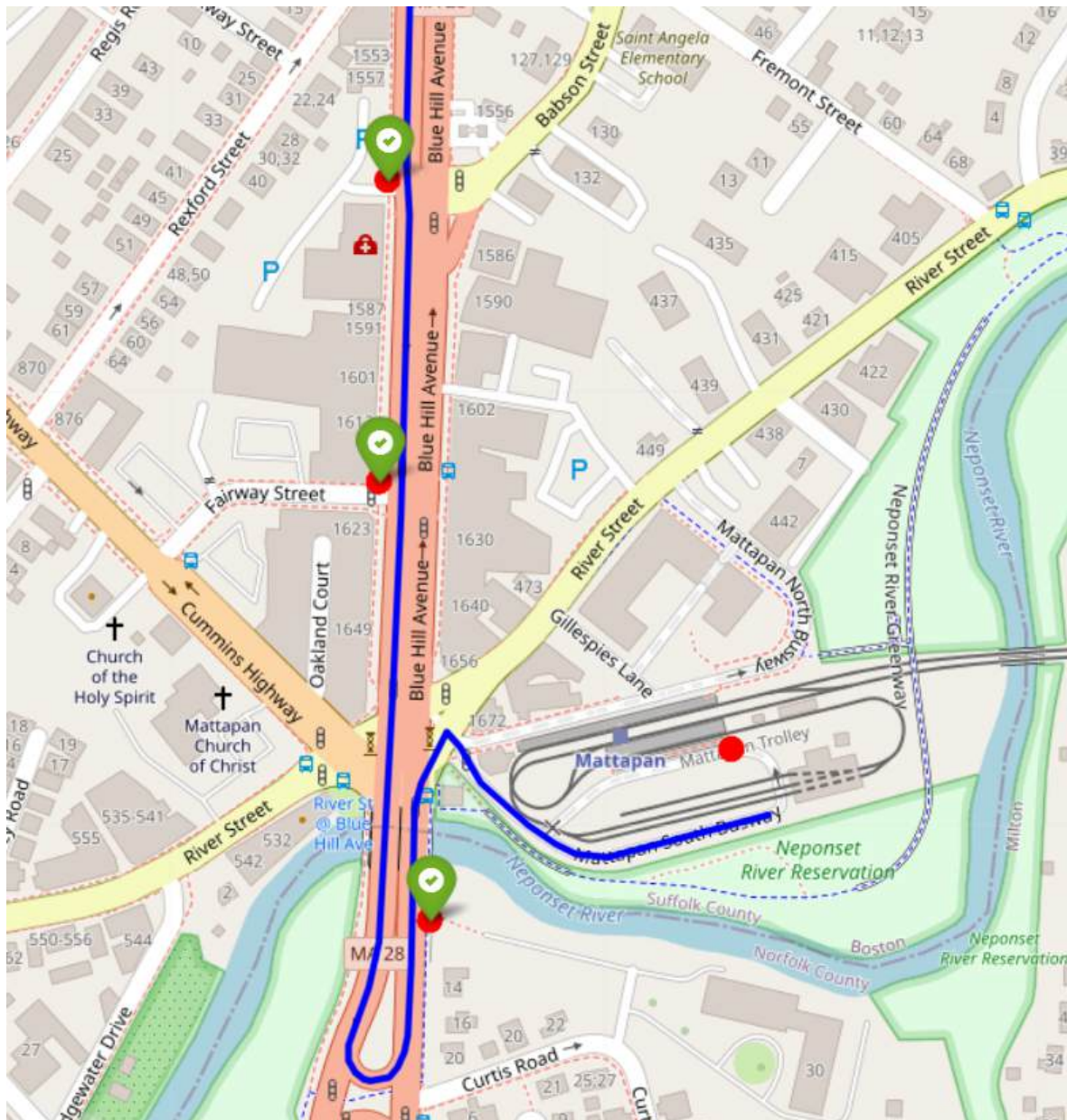


Figura 67: Visualización de una porción del viaje 68992342 utilizando un umbral de 20 metros

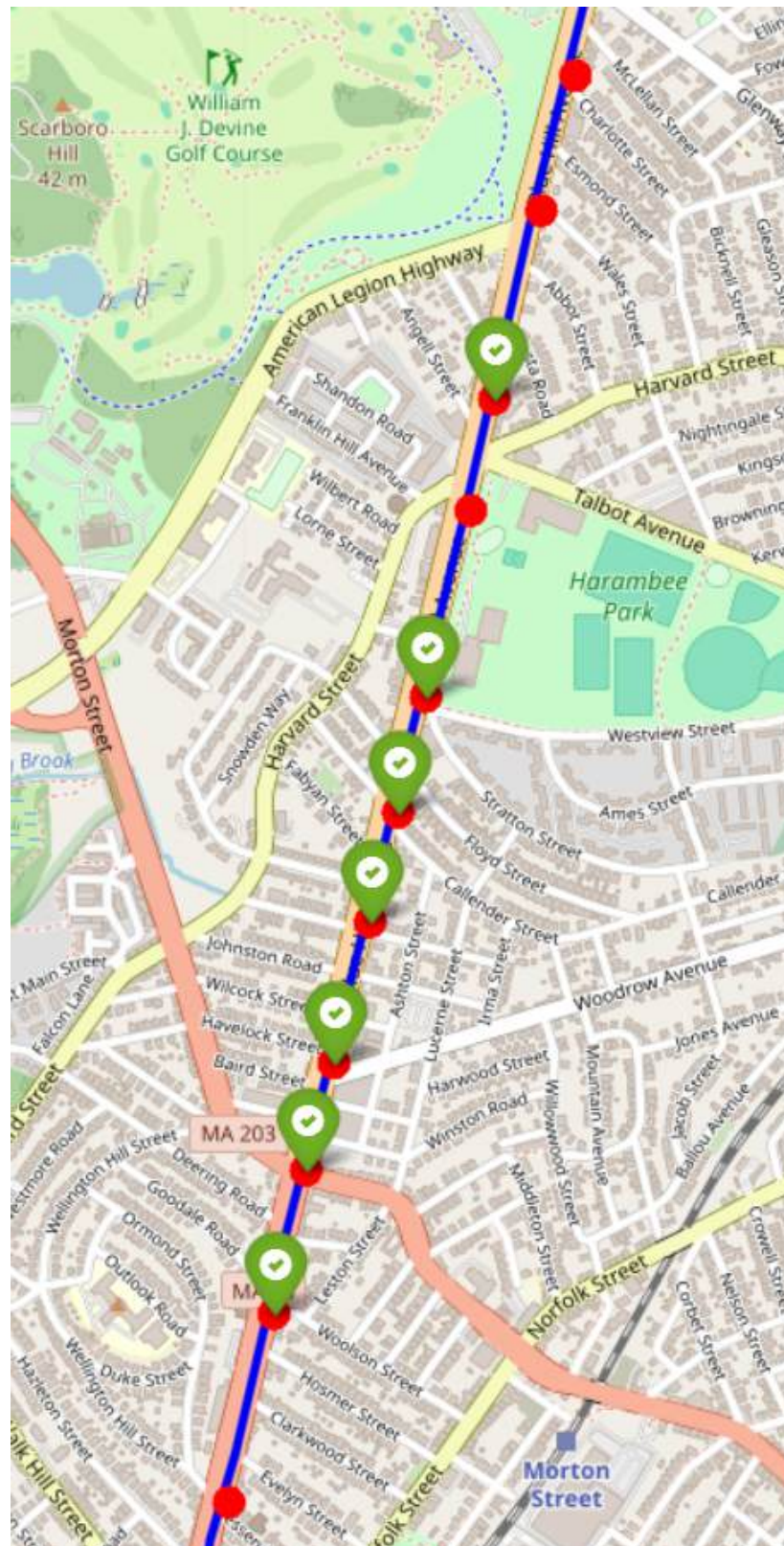


Figura 68: Visualización de una porción del viaje 68992346 utilizando un umbral de 10 metros

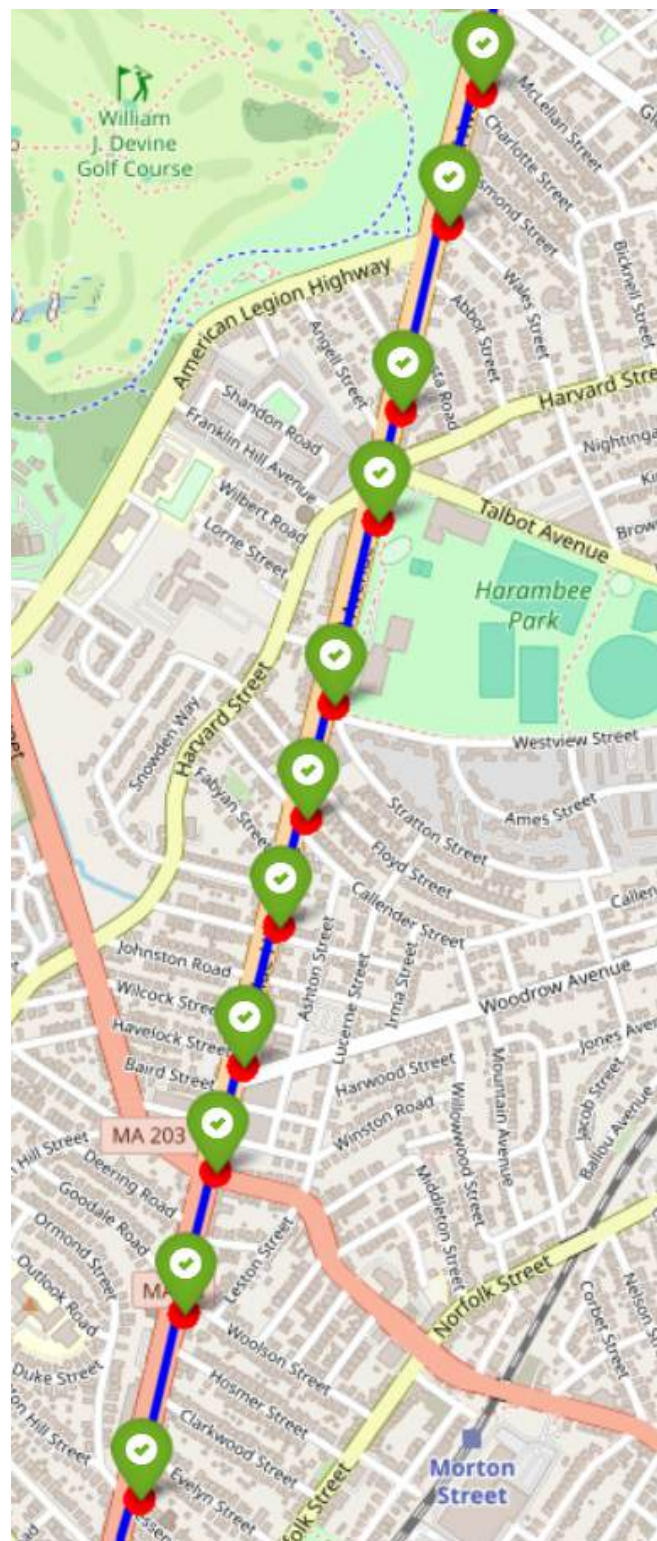


Figura 69: Visualización de una porción del viaje 68992346 utilizando un umbral de 20 metros

Con la tabla `trip_stops` ya generada utilizando el umbral definido, nos propusimos analizar cuántos `trip_id` distintos lograron ser correctamente representados. Observamos que la tabla `map_matched_bus_trips` contiene 1128 viajes distintos como mencionamos en la sección 3.5.3, mientras que `trip_stops` alcanza los 1096. Esta diferencia sugiere que hay de 32 viajes cuyas trayectorias reales no logran coincidir con al menos una parada planificada dentro del umbral definido. En estos casos, cualquier intento de comparar retrasos o velocidades pierde validez, ya que se estarían utilizando referencias temporales de una ruta distinta, lo cual no resulta representativo del comportamiento real del vehículo. Por lo tanto, resulta razonable excluir estos viajes del análisis. No obstante, dado que la cantidad de viajes afectados es reducida, en este trabajo optamos por no eliminarlos de la tabla.

3.6.3 Creación de la tabla `trip_segments`

A partir de la tabla `trip_stops`, se puede generar una nueva tabla que represente los segmentos reales recorridos entre paradas consecutivas que la llamaremos `trip_segments`. Para ello, se recurre a la función de ventana `LAG`, que permite identificar la parada anterior dentro de cada viaje. Esto posibilita formar pares de paradas (origen y destino) para cada segmento, incluyendo tanto los horarios programados como los horarios reales asociados.

```
CREATE TABLE trip_segments AS
SELECT
    trip_id,
    shape_id,
    stop_id AS start_stop_id,
    schedule_time AS start_time_schedule,
    actual_time AS start_time_actual,
    LAG(stop_id) OVER (
        PARTITION BY trip_id ORDER BY stop_sequence
    ) AS end_stop_id,
    LAG(schedule_time) OVER (
```

```
        PARTITION BY trip_id ORDER BY stop_sequence
    ) AS start_time_schedule,
    LAG(actual_time) OVER (
        PARTITION BY trip_id ORDER BY stop_sequence
    ) AS start_time_actual
FROM trip_stops;
```


4. Comparación de datos planificados y en tiempo real

Una vez recolectados, depurados y preparados de forma independiente los datos del sistema planificado (*scheduled*) y del sistema en tiempo real (*real time*), en esta sección se procede a comparar ambos conjuntos. Para ello, se utilizan las trayectorias procesadas y los segmentos definidos en las secciones anteriores, con el objetivo de analizar cómo se comportó efectivamente el sistema frente a lo esperado. Se utilizó como referencia el libro del curso a partir de la sección 11.4.5 hasta la sección 11.5 inclusive para llevar a cabo las secciones 4.1 y 4.2.

El análisis se enfoca en el día 12 de julio de 2025 entre las 15:00 y las 18:00 (hora local de Boston), correspondiente al intervalo entre las 12:00 y las 15:00 en Argentina. Este enfoque permite observar las diferencias en velocidades y tiempos de recorrido, proporcionando así una medida del rendimiento real del servicio de transporte. Todos los scripts se encuentran en `realtime/comparison`.

4.1 Comparación de velocidades

Para evaluar un primer aspecto del sistema de transporte, se llevó a cabo una comparación entre las velocidades promedio registradas en tiempo real y las velocidades promedio planificadas para cada segmento de viaje. Este análisis permite identificar diferencias entre el comportamiento esperado de los vehículos y su rendimiento real durante el período analizado con datos en tiempo real.

Los datos planificados utilizados provienen de la vista materializada `segment_speed_kmh`, presentada en la sección 2.5. Dado que esta vista incluye información de múltiples días y horarios no relevantes para este estudio (la segunda semana de julio 2025), se aplicó un filtro temporal para restringir el análisis a la franja horaria y día de interés. Las velocidades reales fueron calculadas a partir de la tabla `trip_segments`, construida previamente en la sección 3.6.3. Con esto, se generó una nueva vista materializada

avg_speed_diff_per_segment, que permite comparar, para cada segmento, la velocidad promedio planificada con la velocidad observada en tiempo real dentro del intervalo definido.

```
CREATE MATERIALIZED VIEW avg_speed_diff_per_segment AS

-- Calculate the scheduled average speed for each segment during the analyzed time interval
WITH scheduled_segment_speeds AS (SELECT distance_m AS length,
                                         from_stop_id,
                                         to_stop_id,
                                         geometry,
                                         from_stop_name,
                                         to_stop_name,
                                         AVG(speed) AS scheduled_avg_speed
                                   FROM segment_speed_kmh
                                   WHERE t_arrival >= '2025-07-12 14:50:00 America/New_York'
                                      AND t_arrival < '2025-07-12 18:10:00 America/New_York'
                                   GROUP BY distance_m,
                                         from_stop_id,
                                         to_stop_id,
                                         geometry,
                                         from_stop_name,
                                         to_stop_name)

-- Combine with real observed velocities for each segment
SELECT AVG(s.length / EXTRACT(EPOCH FROM (t.end_time_actual - t.start_time_actual))) * 3.6)
AS real_avg_speed,
   s.scheduled_avg_speed,
   s.geometry,
   s.from_stop_id,
   s.to_stop_id,
   s.from_stop_name,
   s.to_stop_name
FROM trip_segments t
   JOIN scheduled_segment_speeds s
   ON t.start_stop_id = s.from_stop_id
   AND t.end_stop_id = s.to_stop_id
WHERE t.start_time_actual IS NOT NULL
   AND t.end_time_actual IS NOT NULL
   AND EXTRACT(EPOCH FROM (t.end_time_actual - t.start_time_actual)) > 0
GROUP BY s.geometry,
   s.from_stop_id,
   s.to_stop_id,
   s.from_stop_name,
```

```
s.to_stop_name,  
s.scheduled_avg_speed;
```

La vista contiene, para cada segmento de viaje, la velocidad promedio observada en tiempo real y la velocidad promedio planificada, junto con la geometría del segmento, los identificadores de parada y los nombres de las paradas de origen y destino. Esto nos va a permitir realizar un análisis espacial y cuantitativo.

4.1.1 Análisis cuantitativo

Previo al análisis espacial, es conveniente cuantificar cuántos segmentos evidencian discrepancias entre la velocidad planificada y la velocidad observada, así como la magnitud de esas diferencias. Para ello, se elaboró un histograma que representa la distribución de las variaciones de velocidad calculadas como la diferencia entre la velocidad real y la planificada en cada segmento. Este enfoque permite visualizar el comportamiento general del sistema en términos de desviaciones respecto a lo esperado.

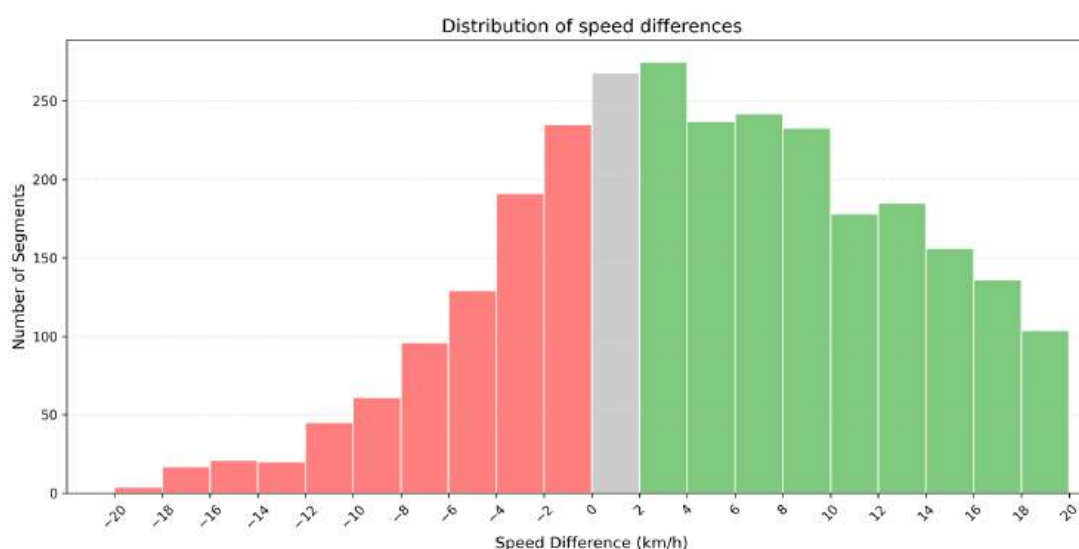


Figura 70: Histograma de diferencias de velocidad promedio por segmento para el 12 de julio de 2025 entre las 15:00 y las 18:00 (hora local de Boston)

El histograma de la Figura 70 presenta una distribución asimétrica con sesgo hacia la derecha. Esto sugiere que, en términos generales, los vehículos tienden a circular a una

velocidad ligeramente superior a la planificada. El valor más frecuente (la moda) se encuentra aproximadamente entre los 2 km/h y los 4 km/h, lo que indica que, en promedio, los vehículos circulan entre 2 y 4 km/h más rápido de lo previsto. Este histograma se generó con el *script* `visualize_speed_diff_histogram.py`.

4.1.2 Análisis espacial

Para complementar el análisis cuantitativo realizado anteriormente, en esta sección representamos espacialmente las velocidades y su diferencia. De esta forma, no solo observamos cuánto difieren las velocidades reales de las planificadas en promedio, sino también dónde ocurren esas diferencias dentro de la red.

Para ello, ejecutamos el script `visualize_speed_diff.py`, que genera tres mapas: uno con las velocidades planificadas, otro con las velocidades reales observadas y un tercero que muestra la diferencia entre ambas para cada segmento. Estas visualizaciones corresponden al mismo intervalo temporal en cuestión.

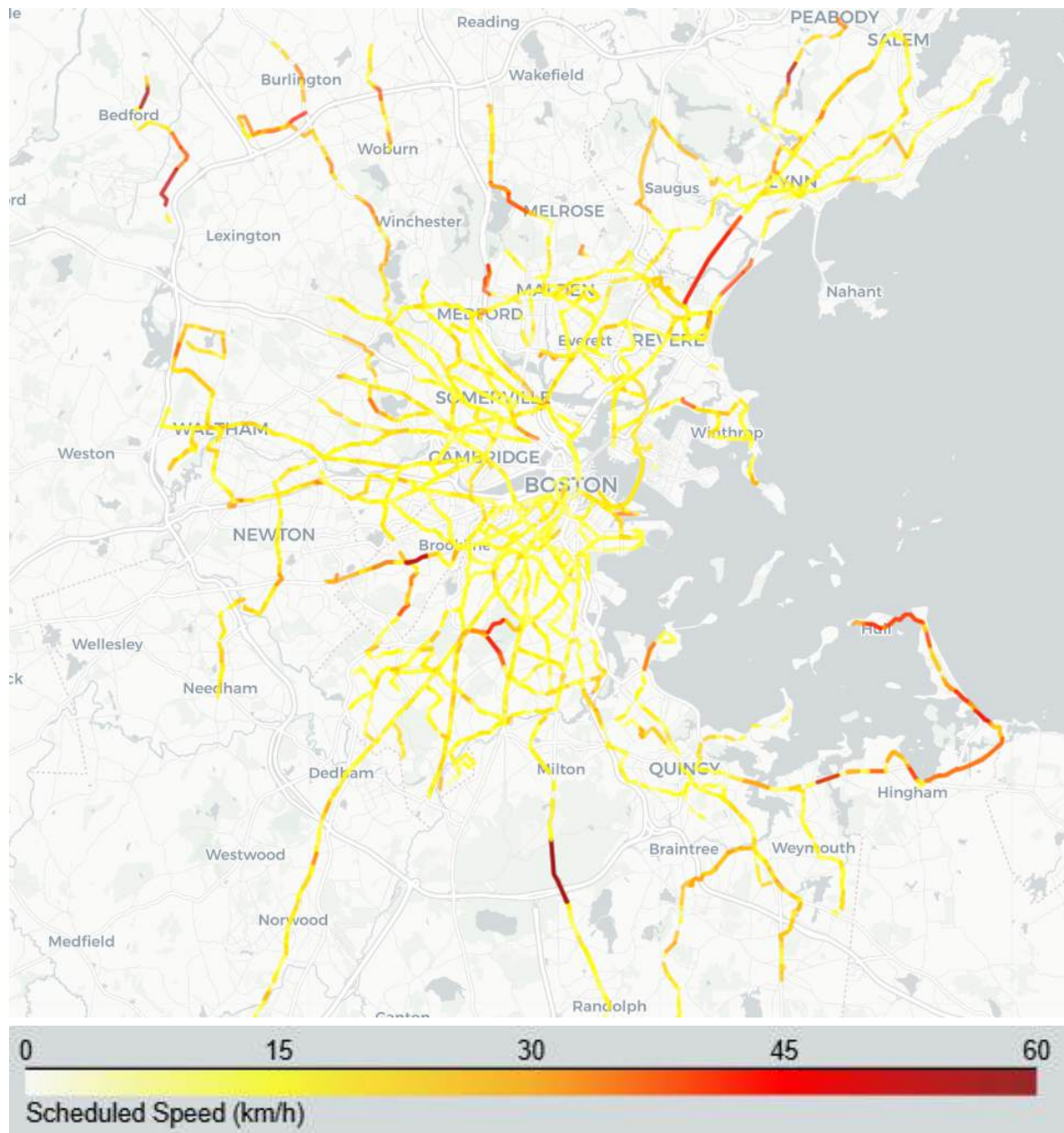


Figura 71: Velocidad promedio planificada para cada segmento

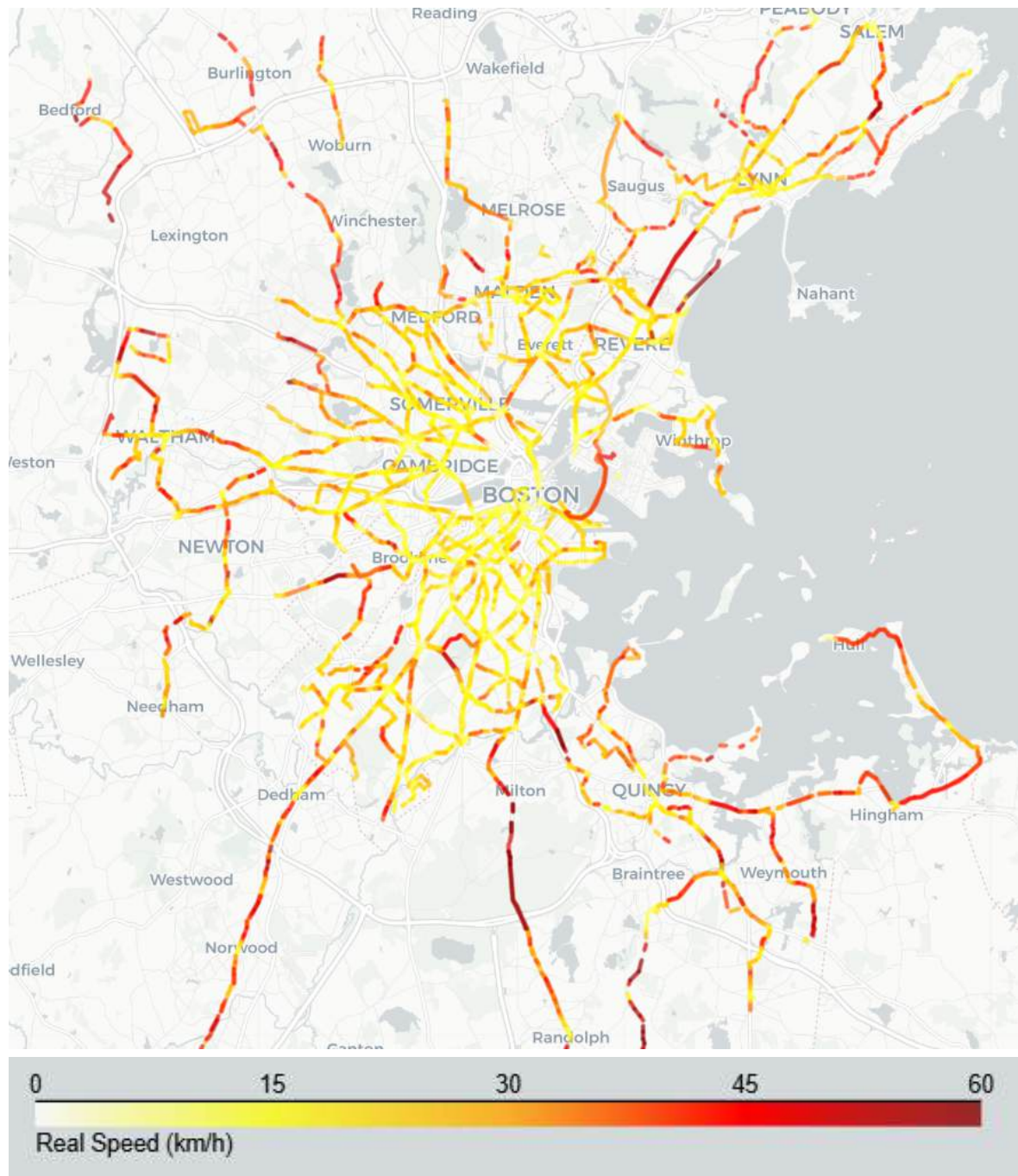


Figura 72: Velocidad promedio real observada para cada segmento



Figura 73: Diferencia entre la velocidad real y planificada para cada segmento

El análisis espacial de las diferencias de velocidad entre lo observado y lo planificado (Figura 73) refuerza las conclusiones obtenidas previamente en el análisis cuantitativo. Tal como evidenciaba el histograma, donde la moda se ubicó entre los 2 y 4 km/h con un claro sesgo hacia la derecha, los mapas muestran una marcada prevalencia de segmentos con velocidades reales superiores a las planificadas, representados en tonos

cálidos, lo que sugiere que los vehículos tienden a circular más rápido de lo previsto en gran parte del sistema.

4.2 Comparación de la demora

En esta sección se estudian las discrepancias (*delay*) entre los tiempos planificados y los tiempos efectivamente observados durante la operación del servicio. El análisis se realiza de dos maneras. A nivel de segmento, se compara el tiempo estimado para recorrer el trayecto entre dos paradas consecutivas con el tiempo real registrado. A nivel de parada, se contrasta la hora programada de arribo con la hora efectiva de llegada del vehículo.

4.2.1 Demora a nivel parada

Para llevar a cabo este análisis, simplemente tenemos que evaluar si los vehículos llegan antes o después de lo planificado. Para ello, podemos calcular el campo `delay` en la tabla `trip_stops` obtenida en la sección 3.6.3 como la diferencia entre el tiempo observado y el planificado, expresado en minutos para cada parada.

```
ALTER TABLE trip_stops ADD COLUMN delay numeric;  
UPDATE trip_stops  
SET delay = EXTRACT(EPOCH FROM actual_time - schedule_time) / 60;
```

Una vez calculado este campo, es posible obtener la demora promedio por parada mediante la siguiente consulta:

```
SELECT stop_id, stop_loc, AVG(delay) as avg_delay  
FROM trip_stops  
GROUP BY stop_id, stop_loc;
```

De este modo, se obtiene una estimación general del desempeño del sistema en cada parada, lo que permite identificar paradas donde las demoras tienden a ser sistemáticamente mayores o menores a lo esperado. Podemos utilizar esta query para representar el resultado espacialmente a través del *script* `visualize_stops_delay.py`.

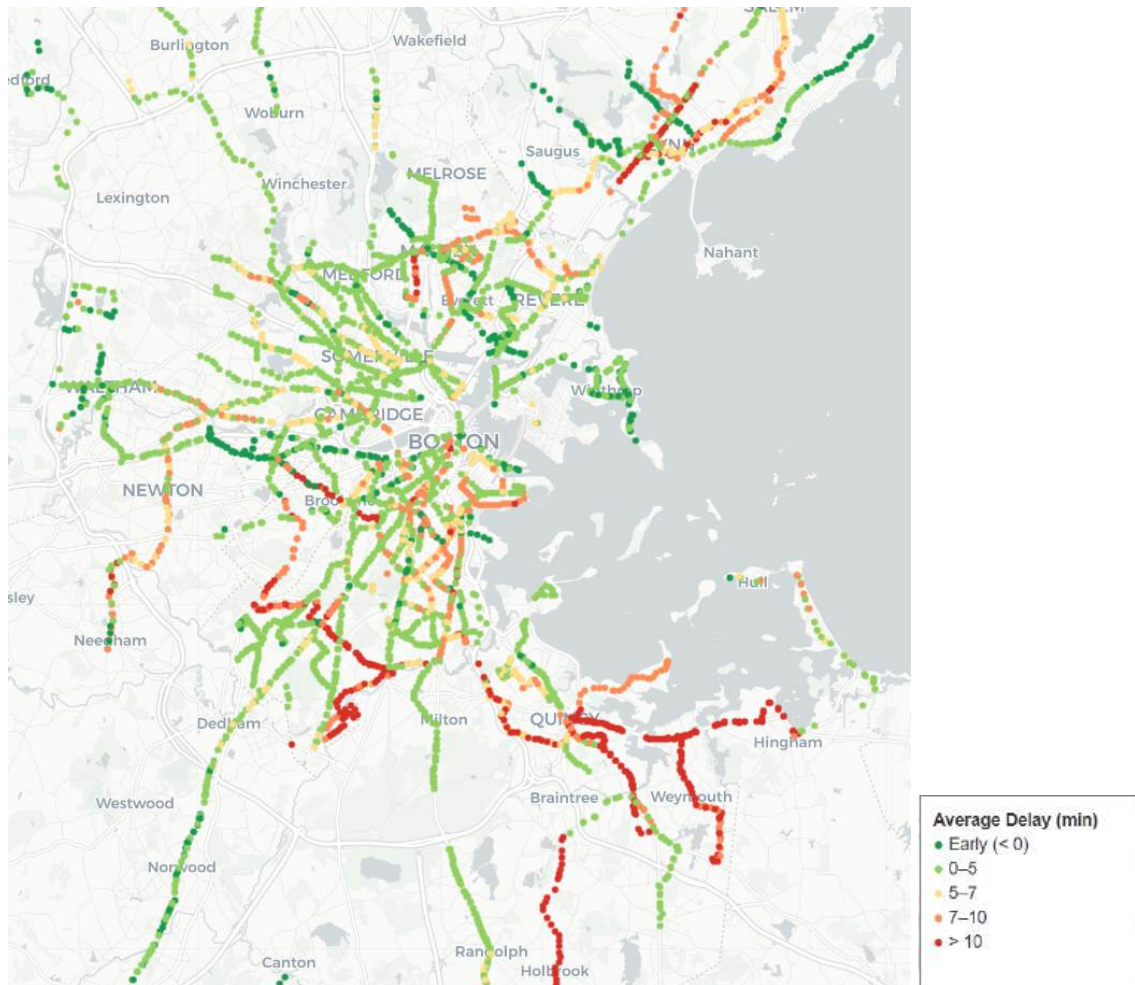


Figura 74: Paradas coloreadas según el retraso promedio en minutos observado

La Figura 74 muestra que, en términos generales para el día que estamos analizando, el sistema de transporte público de Boston operó con puntualidad en la mayor parte del área metropolitana, registrando valores promedio de demora bajos o incluso adelantos respecto al horario programado. Sin embargo, se identifican concentraciones significativas de demoras en la zona sur, donde numerosos puntos presentan valores superiores a los 10 minutos de retraso promedio. Esto sugiere posibles cuellos de botella o situaciones puntuales que afectaron la regularidad del servicio en ese sector durante la jornada analizada.

4.2.2 Demora a nivel segmento

Siguiendo el enfoque del libro del curso, en esta sección se analiza cómo funciona el sistema en cada tramo del recorrido, es decir, entre pares de paradas consecutivas. Este tipo de análisis permite observar no solo si un vehículo llega tarde a una parada, sino también cómo se comporta en el camino entre paradas. De esta forma, se puede identificar en qué tramos del *trip* hay demoras.

Para llevar adelante este análisis, el primer paso consiste en construir la tabla `trips_joins_segments`, que vincula cada segmento con los tiempos planificados y reales necesarios para recorrerlo. A su vez, se calcula la demora por segmento como la diferencia entre ambos tiempos, y se incorporan datos adicionales relevantes, como la geometría del tramo y la información asociada a las paradas de origen y destino.

```
CREATE TABLE trips_join_segments AS (  
  SELECT  
    t.trip_id,  
    t.shape_id,  
    s.distance_m,  
    t.start_time_actual,  
    t.end_time_actual,  
    EXTRACT(EPOCH FROM (t.end_time_actual - t.start_time_actual)) AS  
elapsed_time_actual,  
    EXTRACT(EPOCH FROM (t.end_time_schedule - t.start_time_schedule)) AS  
elapsed_time_schedule,  
    s.geometry AS geometry,  
    t.start_stop_id,  
    t.end_stop_id  
  FROM trip_segments t JOIN segments s  
    ON t.shape_id = s.shape_id  
    AND t.start_stop_id = s.start_stop_id  
    AND t.end_stop_id = s.end_stop_id  
  WHERE  
    t.start_time_actual IS NOT NULL  
    AND t.end_time_actual IS NOT NULL  
    AND EXTRACT(EPOCH FROM (t.end_time_actual - t.start_time_actual)) > 0  
);
```

Una vez creada esta tabla, se puede ejecutar el script `visualize_segments_delay.py`, que genera un mapa interactivo con información sobre cada segmento analizado para ver si tiene o no demora. La consulta que alimenta este mapa se detalla a continuación:

```
WITH delay_metrics_per_segment AS (  
  SELECT  
    start_stop_id,  
    end_stop_id,  
    geometry,  
    COUNT(*) AS no_trips_seg,  
    AVG(elapsed_time_actual) AS avg_actual,  
    AVG(elapsed_time_schedule) AS avg_schedule  
  FROM trips_join_segments  
  GROUP BY start_stop_id, end_stop_id, geometry  
)  
  
SELECT  
  geometry,  
  no_trips_seg,  
  CASE WHEN avg_actual - avg_schedule > 0 THEN 1 ELSE 0 END as has_delay  
FROM delay_metrics_per_segment;
```

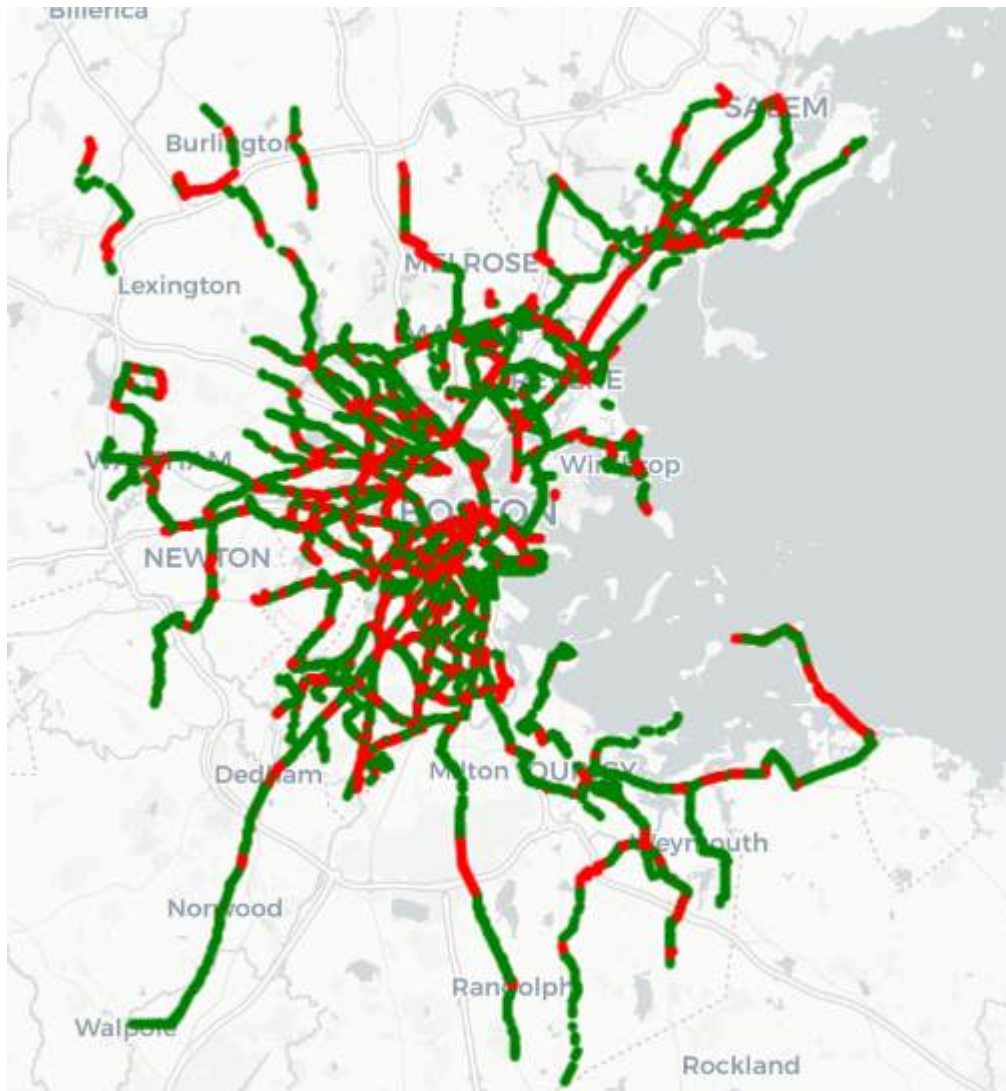


Figura 75: Segmentos según si tienen delay (en rojo) o no (en verde).

Aunque se identifican zonas en la Figura 75 donde los segmentos con demora coinciden con paradas que registran altos valores de delay, la correspondencia entre ambos no es completamente directa. En varias áreas del mapa se observan tramos contiguos con comportamientos dispares: algunos muestran un rendimiento mejor al planificado (coloreados en verde), mientras que otros cercanos evidencian demoras significativas (coloreados en rojo). Esto sugiere que el desempeño del sistema puede variar notablemente incluso dentro de una misma zona.

4.2.3 Demora de vehículos puntuales a nivel parada y segmentos

En esta sección se retoman dos trips analizados previamente en la Sección 3.6.2 (IDs: 68992342, 68964189), con el objetivo de ilustrar de manera más concreta algunos de los patrones observados en los análisis agregados de las secciones anteriores.

Dado el volumen de segmentos y paradas, ciertas comparaciones pueden perderse al visualizarse de forma global. Por este motivo, se desarrolló un *script* específico que toma esos recorridos y genera un mapa comparativo: uno a nivel de paradas y otro a nivel de segmentos, permitiendo así una evaluación más clara y detallada del comportamiento de los vehículos en cada caso particular.



Figura 76: Delay a nivel segmento y parada del *trip* 68992342

En la Figura 76 se observa que las primeras 9 paradas consecutivas presentan demoras. Sin embargo, de los 8 segmentos que conectan esas paradas, solo 3 muestran demoras, mientras que los restantes se recorren en tiempo o incluso más rápido de lo planificado. Esto sugiere que, aunque el vehículo llega tarde a las paradas, su desempeño entre ellas no necesariamente es deficiente.

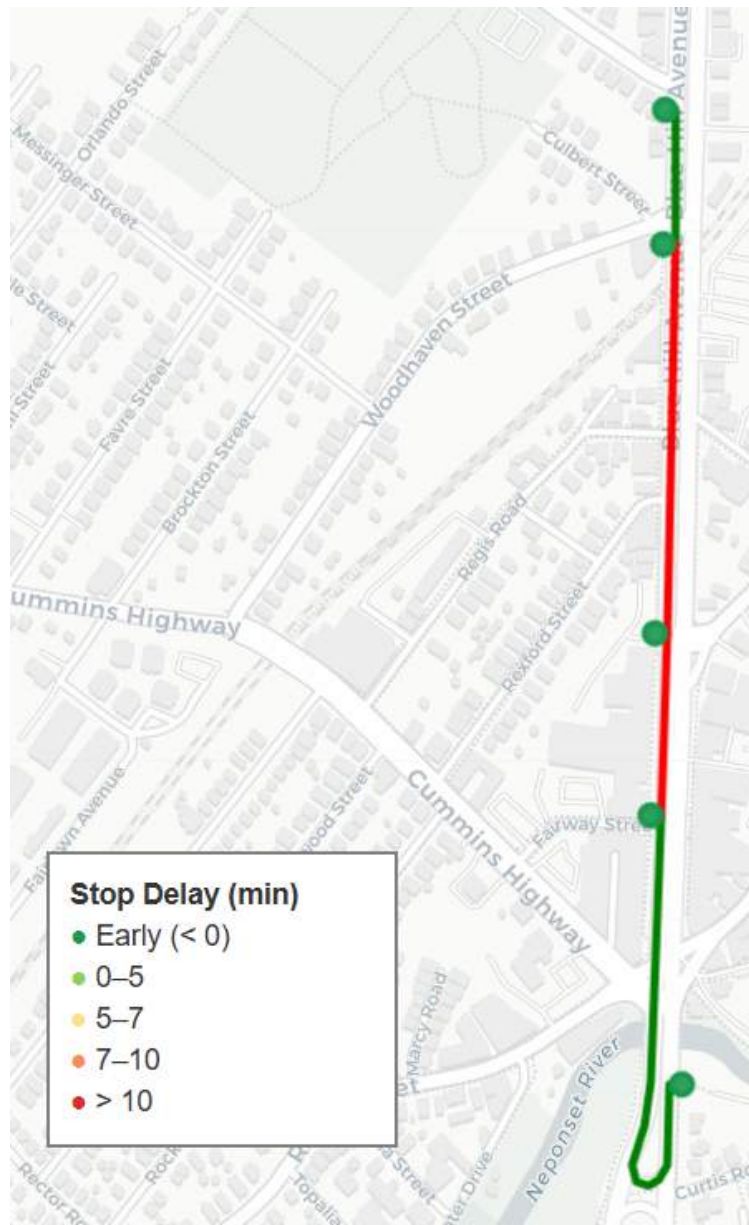


Figura 77: Delay a nivel segmento y parada del *trip* 68992342

En la Figura 77, se observa que el vehículo llega anticipadamente a todas las paradas del recorrido. Sin embargo, dos de los segmentos intermedios presentan demoras, lo que sugiere que, a pesar de esos retrasos, el vehículo había acumulado suficiente adelanto al inicio (o logró recuperarlo más adelante), permitiéndole llegar puntualmente a las siguientes paradas.

5. Conclusión

A lo largo del trabajo, se logró implementar una arquitectura sólida para la carga, procesamiento y análisis de datos GTFS estáticos y en tiempo real del sistema de transporte público de la ciudad de Boston. El uso de herramientas modernas como `gtfs-via-postgres`, `gtfs-functions`, `MobilityDB` y `Valhalla` permitió realizar un análisis comparativo detallado entre los trayectos planificados y los efectivamente realizados por los vehículos, así como también visualizar demoras y velocidades.

Uno de los principales desafíos del proyecto estuvo en el manejo de los datos en tiempo real. Particularmente, la etapa de map matching representó una dificultad significativa: el proceso de ajuste de trayectorias a la red vial utilizando el servidor público de `Valhalla` resultó complejo, llevo mucho tiempo, y, en muchos casos, los resultados obtenidos no fueron del todo coherentes. Este problema se vio agravado por limitaciones propias del servidor (como el límite de solicitudes por segundo), por inconsistencias en la calidad de los puntos GPS recolectados y por la necesidad de realizar múltiples pruebas para ajustar parámetros como el `turn_penalty_factor`.

A pesar de estos obstáculos, el trabajo permitió extraer conclusiones valiosas sobre el sistema de transporte de Boston y sentó una base replicable para realizar estudios similares en otras ciudades. El código, la documentación y los resultados obtenidos están disponibles públicamente, facilitando su uso en futuras investigaciones o proyectos académicos relacionados.

X. Referencias

- [1] J. Godfrid, P. Radnic, A.A. Vaisman, and E. Zimányi. Analyzing public transport in the city of Buenos Aires with MobilityDB. *Public Transport*, 14(2):287–321, 2022
- [2] Sakr, M., Vaisman, A., & Zimányi, E. (2025). *Mobility Data Science: From Data to Insights*. Springer.
- [3] Mobility Data. Reference - General Transit Feed Specification (Schedule). Link disponible en la web: <https://gtfs.org/documentation/schedule/reference/>
- [4] Mobility Data. Reference - General Transit Feed Specification (Real Time). . Link disponible en la web: <https://gtfs.org/documentation/realtime/reference/>
- [5] Public-Transport. gtfs-via-postgres: Process GTFS static/schedule by importing it into a postgresql database, GitHub. Link disponible en la web: <https://github.com/public-transport/gtfs-via-postgres>
- [6] Bondify. gtfs_functions: Package with useful functions to create geo-spatial visualizations from a GTFS, GitHub. Link disponible en la web: https://github.com/Bondify/gtfs_functions
- [7] Descripción General de GTFS Static. Google. Link disponible en la web: <https://developers.google.com/transit/gtfs?hl=es-419>
- [8] Descripción General de GTFS Realtime. Google. Link disponible en la web: <https://developers.google.com/transit/gtfs-realtime?hl=es-419>
- [9] GTFS Schedule Data, MBTA. Link disponible en la web: <https://www.mbta.com/developers/gtfs>
- [10] GTFS Real Time Data, MBTA. Link disponible en la web: <https://www.mbta.com/developers/gtfs-realtime>